Faculty of Sciences Department of Computer Sciences

Master thesis

Procedural Generation of Solvable Cooperative Levels for the Laser Learning Environment

Author:
Hugo Charels
hugo.charels@ulb.be

Supervisors:
Tom Lenaerts
Yannick Molinghen

Academic year 2025-2026

Abstract

This thesis develops a SAT-based framework for the procedural generation of solvable, cooperative levels for the Laser Learning Environment (LLE), a multi-agent reinforcement learning benchmark. We formalise bounded-horizon LLE solvability as a propositional satisfiability decision problem and prove the reduction correct; we introduce a strict-beam-semantics counterfactual that turns the **cooperation-required** property into a second decidable problem on the same level; and we further classify cooperative levels by the structure of their inter-agent dependencies into five profiles: asymmetric, mutual, chain, distributed, and fully coupled. These decision procedures are embedded inside a family of procedural generators (Random, Constrained Random, Constructive, Level-6-Style) that emit only levels certified by the solver to be solvable and, on demand, to require cooperation, possibly of a specific profile (one of the five above). The empirical evaluation compares two SAT encodings, characterises per-generator rejection rates and cooperation-profile distributions, and shows that off-the-shelf value-based cooperative-MARL algorithms (IQL, VDN, QMIX) can learn certified cooperative levels at 5×5 and that enlarging the generated training pool restores generalisation. A final set of curriculum experiments finds that curriculum scheduling offers no advantage over direct training on a reachable target, and that no curriculum — including a staged budget of two million steps — lets these algorithms solve the mutually-cooperative LLE Level 6, on which direct training also scores zero; we trace this negative result to the base task being unlearnable by these value-based methods rather than to a failure of curriculum design.

Keywords: multi-agent reinforcement learning, procedural content generation, SAT solving, cooperative levels, Laser Learning Environment, curriculum learning.

Contents

Chapter 1

Introduction

1.1 Context

Cooperative Multi-Agent Reinforcement Learning (MARL) studies settings in which several agents must learn behaviours whose value appears only at the team level. In such settings, the environment is not a neutral container: it determines which coordination patterns are possible, which ones are necessary, and how difficult they are to discover.

This dependence on environment structure is especially strong in sparse-reward cooperative tasks. When reward is issued only after the team objective has been achieved, two failure modes make a training level useless. An unsolvable level provides no learning signal at all. A level solvable without cooperation defeats the purpose, since we are training the agents precisely to learn how to cooperate.

We instantiate this work on the Laser Learning Environment (LLE), a cooperative MARL benchmark whose mechanics are formalised in Section 3.1; its hand-crafted Level 6 serves as the canonical hard target throughout the thesis. We expect this methodology (coupling procedural generation with a formal verification oracle) to generalise to any MARL setting whose target properties are expressible as decision problems, but we evaluate it on LLE only and leave broader transfer to future work.

1.2 Motivation

In cooperative environments whose mechanics create explicit inter-agent dependencies, level design is particularly challenging. A useful training level should not merely appear cooperative, it should provably contain the intended coordination structure.

Procedural Content Generation (PCG) offers a way to scale level creation beyond what manual design allows, but only if the generated instances satisfy the properties that matter for training. Two properties are central. First, a level must be **solvable**. Second, success should **require cooperation** in the specific sense induced by the environment's mechanics. Without those guarantees, PCG risks producing levels that are invalid, trivial, or misaligned with the training objective.

Generated levels are also useful beyond standalone training. The same knobs that control a single level also define a natural difficulty axis: grid size, number of agents, number of lasers, wall budget, cooperation profile, and time horizon (the bounded number of joint steps allowed for a solution). A formally-verified generator is therefore a candidate building block for **curriculum learning**: rather

than train directly on a hard hand-crafted target, an agent can be trained on a sequence of progressively harder generated levels.

The thesis delivers three contributions and an empirical study. The contributions are a bounded-horizon SAT encoding of LLE solvability with a correctness proof and complexity placement; a strict-semantics counterfactual that turns the informal "cooperation required" intuition into a decidable property, refined into five increasingly rich cooperation profiles; and a solver-in-the-loop generator family that accepts only certified levels and exposes the structural axes above as user-facing parameters. The empirical study covers SAT-encoding cost, generator acceptance rates and profile distributions, learnability of generated cooperative levels for off-the-shelf MARL, and a curriculum-transfer study toward LLE Level 6.

Beyond the thesis itself, the SAT-based solver, the cooperation detector, and the procedural generator family have been contributed upstream to the official `laser-learning-environment` library[1] (released on PyPI as `laser-learning-environment[generator]`, version 2.9.0 and onwards), where they are exposed as the public functions `lle.solve`, `lle.is_cooperative`, `lle.cooperation_level`, and `lle.generate`.

1.3 Research questions and scope

This thesis addresses one overarching question: how can we automatically generate levels for a cooperative MARL environment that are provably solvable and provably require the target cooperative interaction, and how can such levels be used to support MARL training? We decompose this question into six research questions:

- **RQ1:** How can we formally verify that a level is solvable by the agents?
- **RQ2:** How can we formally verify that solving a level genuinely requires cooperation between agents, rather than allowing independent solutions?
- **RQ3:** How can formal verification be embedded inside a procedural generator so that every accepted level comes with certified properties?
- **RQ4:** Can we control the cooperation structure of generated levels by targeting specific profiles such as asymmetric, mutual, chain, distributed, or fully coupled dependencies?
- **RQ5:** Can MARL agents trained exclusively on procedurally generated cooperative levels reach a non-trivial greedy success rate on a held-out pool of generated levels at a fixed training budget, and how does that performance vary across IQL, VDN, and QMIX?
- **RQ6:** Does a staged curriculum of generated levels of growing geometric and cooperative complexity yield a higher greedy success rate on hand-crafted LLE Level 6 than a baseline trained directly on Level 6 (or on the union of all stage pools) at the same total training budget?

The thesis instantiates this framework on LLE, focusing on the **exit-reaching task**: each agent must reach an exit tile. The formal model covers agent movement, inter-agent blocking interactions, and a bounded-horizon solvability criterion. Collectible gems and the incentive-scoring layer that rewards them are outside the scope of this thesis. Void tiles are not handled in our model either; to use the solver on a level containing void tiles, the user must first replace each void tile with a wall, which yields the same verdict. These restrictions are deferred to future work.

[1] https://pypi.org/project/laser-learning-environment/

1.4 Thesis structure

The remainder of this thesis is organised as follows. Chapter 2 positions the work relative to cooperative MARL benchmarks, procedural content generation, and compilation-based planning literature. Chapter 3 fixes the technical setting: the LLE mechanics, the Boolean satisfiability background, and the formal problem statement that defines bounded-horizon solvability as well as the cooperation requirement.

The three contribution chapters then develop the original technical content of the thesis in logical order. Chapter 4 introduces the SAT-based solver: a propositional encoding of bounded-horizon LLE solvability, with correctness proofs and a complexity-theoretic positioning. Chapter 5 builds on that encoding to define the cooperation detector based on a strict counterfactual semantics, and extends it with a profile analyser that classifies the kind of cooperation a level exhibits. Chapter 6 then uses both decision procedures as acceptance oracles inside a family of procedural generators that produce levels certified to satisfy the advertised properties.

Chapter 7 reports the empirical evaluation: the SAT encoding comparison, generator acceptance rates and cooperation-profile distributions, the learnability of generated cooperative levels for off-the-shelf MARL algorithms, the effect of training-pool size on generalisation, and the curriculum-transfer study toward the hand-crafted LLE Level 6. Chapter 8 concludes with a summary and directions for future work.

Chapter 2

Related work

2.1 Cooperative MARL and coordination-critical benchmarks

This thesis is motivated by a benchmark-design question within cooperative Multi-Agent Reinforcement Learning (MARL): how can one generate training instances whose coordination structure is both non-trivial and formally controlled?

In the fully cooperative setting, all agents optimise a shared return, so the quality of the environment matters as much as the quality of the learning algorithm. If the environment admits only trivial solutions, it does not meaningfully test coordination. If it contains unsolvable instances, it provides no useful training signal at all. For the present thesis, the central issue is therefore not MARL in general, but the design of instances in which inter-agent dependence is both structurally present and formally decidable.

The general framework for sequential multi-agent decision-making originates in stochastic games [1], generalised to the Markov game model that has become the standard formalism for MARL [2]. The single-agent reinforcement-learning machinery on which cooperative MARL builds is covered comprehensively in [3].

Two algorithm families dominate the cooperative MARL literature. **Value-decomposition** methods factor a shared team value into per-agent components to enable decentralised execution: VDN [4] uses an additive decomposition, and QMIX [5] generalises this to a state-dependent monotonic mixing network. **Centralised-critic actor-critic** methods take a different route, training a joint critic that conditions on full state at training time while each agent retains a local actor at execution time: MADDPG [6] is the canonical deterministic-policy instance, COMA [7] refines the credit-assignment signal with a counterfactual baseline, and MAVEN [8] extends QMIX with latent-variable exploration to escape the monotonicity bottleneck on coordination-critical tasks.

Both families perform well when individual contributions are roughly additive, and both struggle on the coordination-critical, low-reward bottlenecks that LLE is designed to expose [9]. The empirical chapter of this thesis (Chapter 7) accordingly uses three points along the value-decomposition spectrum, namely independent Q-learning (IQL, no credit assignment), VDN (additive), and QMIX (monotonic mixing), to train on generated cooperative levels and on the curriculum-transfer target.

2.2 The Laser Learning Environment

The Laser Learning Environment (LLE), whose mechanics we formalise in Section 3.1, was introduced precisely to study coordination-critical multi-agent tasks [9]. It sits alongside other cooperative-MARL benchmarks such as SMAC [10] (StarCraft micro-management with partially observable team play) and Overcooked [11] (a constrained cooperative kitchen with temporal synchronisation), but differs in that its hardness comes from explicit **inter-agent blocking** rather than from partial observability or large action spaces.

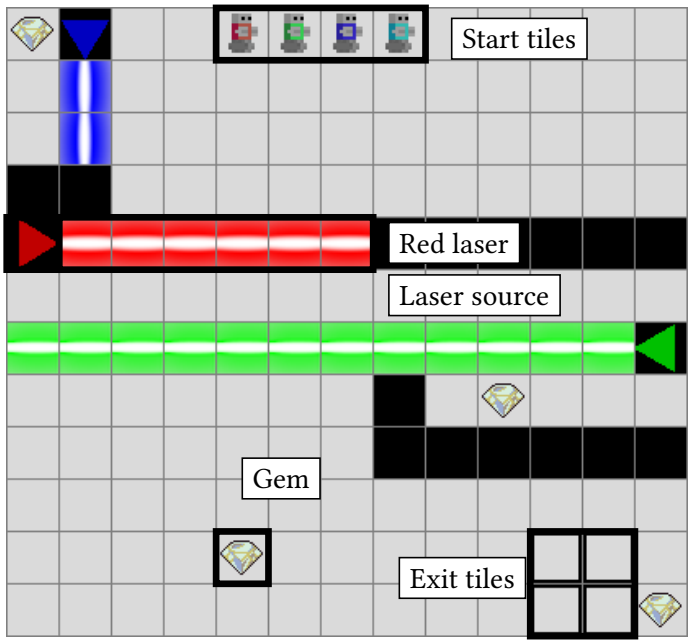

Figure 1: LLE Level 6, the canonical hard target used throughout this thesis. Agents must reach the exit tiles. Laser beams block movement for non-immune agents, but an immune agent can block its own beam by occupying a cell along its path, opening a route for a teammate.

The LLE paper identifies three properties that make the benchmark difficult for value-based MARL methods: **perfect coordination**, **interdependence**, and **zero-incentive dynamics** [12]. Together, these properties create bottlenecks in which one agent must perform a locally unrewarded action that enables another agent to progress.

This framing is directly relevant here. The thesis does not try to improve MARL algorithms on LLE; it addresses an upstream question the benchmark paper leaves open: how can we generate LLE levels guaranteed to be solvable and to contain the beam-blocking dependency the benchmark relies on?

The LLE paper therefore plays two roles in this thesis. First, it justifies why LLE is an interesting target domain. Second, it provides the conceptual vocabulary used here to discuss cooperation: the key object is not generic teamwork, but a concrete interdependence mechanism created by coloured lasers and same-colour blocking.

2.3 Dependency structures in cooperative MARL

The same-colour beam-blocking mechanism on which LLE relies admits a "cooperation required / not required" verdict per level, but cooperative behaviour at finer granularity can vary considerably: in a level with several agents and several lasers, helping relations can form a one-way edge, a mutual pair, a directed chain, a fan-in (one beneficiary, multiple helpers), or a fully connected graph. The

cooperation profile analyser of Chapter 5 extracts this dependency graph from a SAT model of a solution and labels it as, respectively, **asymmetric**, **mutual**, **chain**, **distributed**, or **fully coupled**. The structural categories themselves are standard graph-theory vocabulary; to our knowledge, this is the first taxonomy that recovers such structures from a SAT certificate of solvability in LLE. The prior MARL literature considers related but distinct structural notions.

The closest precedent is the **coordination graph** introduced by C. Guestrin, M. G. Lagoudakis, and R. Parr [13], which decomposes a joint Q-function over agent subsets connected by hyper-edges. Coordination graphs are a representational tool: they assume the structure is known a priori and use it to make value computation tractable. Our taxonomy operates in the opposite direction. The structure is not given; it is **recovered** from a SAT certificate of solvability, and the label is a property of the level produced rather than a modelling assumption made about the agents. The two notions are therefore complementary: a coordination graph tells the **learner** which agent subsets interact, while our profile tells the **level designer** what kind of cooperation a generated instance contains.

A more recent line of work by U. Biswas, V. Palod, S. Bhambri, and S. Kambhampati [14] operates in the same spirit: they define **constructive interdependence**, a metric that measures how much agents rely on each other's actions to achieve a shared goal, over a STRIPS planning formalism, and use it to evaluate cooperation in human-AI teams on the Overcooked domain. Like our cooperation-profile taxonomy, their metric is recovered from a formal model rather than assumed; unlike ours, it scores the cooperative behaviour that emerges between trained agents and humans on a fixed task, rather than classifying the structural dependency a generated level requires.

2.4 Procedural generation under structural constraints

Procedural Content Generation (PCG) is useful in reinforcement-learning settings because it can replace a small fixed benchmark set with a larger and more diverse stream of instances [15]. Within PCG, search-based methods (surveyed by J. Togelius, G. N. Yannakakis, K. O. Stanley, and C. Browne [16]) phrase content creation as an optimisation problem over a content space, which is conceptually close to the solver-driven acceptance loop adopted in this thesis. The closest precedent for the present declarative-constraint approach is the Answer Set Programming (ASP) generator of A. M. Smith and M. Mateas [17], which uses an ASP solver as the acceptance oracle for game content. A complementary line of work surveyed under the Procedural Content Generation via Machine Learning (PCGML) banner [18] uses machine-learned generators trained on existing levels, but typically provides no formal guarantee on the produced output. The difficulty for the present problem is not merely to produce varied levels, but to produce levels that satisfy logically defined properties.

PCG has been increasingly developed for reinforcement learning, where the role of the generated content is not to entertain a human player but to provide training material for a learning agent. S. Risi and J. Togelius [19] survey this line and argue that PCG is a natural lever for moving beyond fixed-benchmark RL toward generalisation across infinite environment families. Within that line, PCGRL [20] inverts the relationship between PCG and RL: rather than using PCG to feed RL agents, it trains an RL agent to **act as** the level designer. The thesis here runs in the opposite direction (a fixed solver-in-the-loop generator produces certified levels that feed RL training), so PCGRL is a useful contrast rather than a comparable system.

What distinguishes this thesis from the lines above is the **verification** step. A constructive or search-based generator may bias generation toward interesting layouts, but without a verifier it cannot certify that a sampled level is solvable or that success genuinely depends on cooperation. The present thesis

therefore adopts a constraint-aware view of PCG: generation is coupled to a formal decision procedure, and the solver acts as an acceptance oracle rather than as a post-hoc descriptive tool.

2.5 Curriculum Learning and generated environments

The general idea of training on a sequence of progressively harder tasks predates the modern deep-learning era; Y. Bengio, J. Louradour, R. Collobert, and J. Weston [21] formalised **curriculum learning** as a meta-learning strategy in which the order of training examples is itself a design choice. In reinforcement learning specifically, the survey by S. Narvekar, B. Peng, M. Leonetti, J. Sinapov, M. E. Taylor, and P. Stone [22] catalogues the design space along three axes (the **task generator**, the **sequencing policy**, and the **transfer mechanism**) and shows that curriculum learning consistently helps on long-horizon and sparse-reward problems, the regime in which LLE Level 6 sits.

A more recent line of work tightens the coupling between curriculum and environment generation. **POET** [23] co-evolves a population of environments and a population of agents in an open-ended loop: environments that are too easy or too hard for the current agent population are eliminated, and surviving environments act as a self-organising curriculum. **PAIRED** [24] extends this idea to **unsupervised environment design**: an adversary parameterises environments to maximise the regret of a protagonist agent against an antagonist baseline, which provably keeps the generated environments solvable while remaining at the frontier of the protagonist's ability.

The present thesis sits adjacent to this line of work. Like POET and PAIRED, we generate environments rather than reuse a fixed test set, and we use those environments as a curriculum toward a hard target. Unlike POET and PAIRED, the generator here is not adversarial and not adaptive to the learner's current policy: it is a **static** solver-in-the-loop generator that emits levels certified to satisfy fixed structural properties (solvability, cooperation, profile) at a chosen difficulty configuration. The curriculum used in Chapter 7 is hand-staged with four manually ordered configurations of growing grid size and cooperation requirement.

2.6 SAT-based planning

A second compilation lineage directly relevant to this thesis is **SAT-based planning** (we recall the propositional satisfiability background in Section 3.2). H. Kautz and B. Selman [25] introduced SATPLAN, encoding bounded-horizon STRIPS planning instances as propositional formulas; subsequent work refined both the encodings and the search strategies [26]. The treatment by J. Rintanen, K. Heljanko, and I. Niemelä [27] covers parallel-plan encodings and modern algorithmic refinements that drove SAT-based planners to competitive performance with dedicated planners. The bounded-horizon LLE encoding developed in this thesis sits in the same conceptual family: a state-transition decision problem reduced to SAT, with the encoding choices materially affecting solver performance.

2.7 Compilation-based multi-agent path finding

The closest methodological precedent is not PCG for MARL, but compilation-based Multi-Agent Path Finding (MAPF) [28]. In standard MAPF, agents move on a discrete graph from start vertices to goal vertices while avoiding collisions. The computational complexity comes from the interaction between multiple agents and the optimality criterion imposed on the solution.

The survey by P. Surynek [29] shows that MAPF has become a major testbed for compilation-based solving: instead of searching directly in the original state space, one reduces a MAPF instance to a target formalism (CSP, SAT, or MILP) and relies on the target solver to handle the combinatorial burden. MAPF research is not exclusively compilation-based; the dominant search-based alternative, Conflict-Based Search [30], achieves optimal solutions through a two-level constraint-satisfaction tree without reducing to a target formalism. We adopt the compilation route rather than CBS-style search because the property of interest in this thesis is the **existence** of a valid joint plan within a fixed horizon, not its makespan optimality: a SAT decision procedure matches the question we ask, whereas CBS would need substantial extension to encode the laser-propagation and strict-counterfactual semantics introduced in Chapter 5.

Within the compilation route, encoding design materially affects solver performance. The work of M. Frommknecht and P. Surynek [31] studies SAT-based MAPF under the makespan objective using an MDD-SAT formulation and compares solver-facing choices such as eager versus lazy encodings and informative initial assignments. We cite it not because it solves the same problem, but for a recurring lesson: performance depends not only on the underlying decision problem but on how it is encoded and how the resulting CNF interacts with the chosen SAT solver. The empirical chapter of this thesis (Chapter 7) compares two alternative uniqueness encodings in the same spirit.

At the same time, the distance of this work to standard MAPF must be stated explicitly. Standard MAPF encodings reason about graph motion and collisions. The present thesis must additionally encode time-dependent laser propagation, same-colour immunity, and a strict-counterfactual semantics used to define cooperation. The MAPF literature therefore supplies a methodological template, not a drop-in solution.

2.8 Formal methods coupled to reinforcement Learning

A separate line of work uses formal methods as an explicit component of the reinforcement learning loop rather than as a post-hoc analysis. The canonical example is **shielded RL** [32], where a formally synthesised reactive shield filters the agent's candidate actions at runtime so that no policy update can ever violate a stated temporal-logic specification. The shield is computed once from the specification and the environment model, then inserted between the agent and the environment as a hard constraint.

The connection to this thesis is methodological rather than direct. Shielded RL inserts a formal acceptance oracle on the **action** side of the loop, at runtime; we insert one on the **training- material** side, at generation time. Both follow the same recipe (encode a target property in a decidable formalism, run a solver, and reject anything that fails), but they apply it to opposite ends of the agent-environment interface. Shielded RL therefore does not provide a drop-in technique, but it does demonstrate that "formal-methods filter on top of an RL system" is a working pattern.

2.9 Positioning of the thesis

Taken together, the MARL benchmarking literature, the PCG and curriculum-learning literature, and the SAT-planning / compilation-based MAPF literature mark the conceptual neighbourhood of this thesis. None of them, however, fully covers the problem we address: each line studies one side of the problem in isolation. The remaining gap can be summarised under four headings.

- **Cooperative MARL benchmarks and inter-agent structure.** The LLE paper [9] establishes the benchmark and explains why its coordination bottlenecks are difficult for MARL algorithms,

but it does not provide a formal generator for certified cooperative levels. The coordination-graph line [13] gives a representation for known inter-agent dependencies but does not classify the **types** of dependency that emerge from a specific cooperation mechanism.

- **Compilation-based multi-agent planning.** The MAPF compilation literature [29], [31] shows that SAT is an effective backend for multi-agent planning and that encoding design matters in practice, but it solves a different problem (path optimality on a collision graph) and does not address either the laser-propagation semantics or cooperation as a semantic property of the instance.
- **Procedural generation, curriculum learning, and environment design.** The PCG-for-RL line [19], [20] frames generation as a tool for RL generalisation, and the environment-design line [23], [24] treats curriculum generation as adaptive co-evolution. Neither embeds a decision procedure that **proves** each generated instance satisfies a target structural property.
- **Formal methods on top of RL.** The shielded-RL line [32] shows that formal methods integrate cleanly with RL when used as a runtime **action** filter, but does not address the upstream question of whether the **training material** itself satisfies the specification.

This thesis sits at the intersection of those four lines. It transfers the compilation-based SAT mindset from MAPF into the LLE setting, formalises bounded-horizon solvability for an LLE-specific model, introduces a strict-semantics counterfactual that turns a benchmark-level intuition about cooperation into a decidable property inside procedural generation, and classifies the resulting dependency structures into a taxonomy that downstream generators can target as a parameter.

Chapter 3

Background and formalisation

3.1 The Laser Learning Environment

The Laser Learning Environment (LLE) [9] is a grid-based cooperative multi-agent benchmark designed to study coordination-critical tasks. A level consists of a rectangular grid containing walls, coloured laser sources, agent start positions, and exit tiles. Each agent is assigned a colour and must reach an exit tile.

The central mechanic is the laser beam. Each source emits a directional beam that propagates cell by cell until it reaches a wall or the grid boundary. An agent whose colour matches the beam is *immune* to it: it may occupy a cell the beam traverses without being harmed, but its physical presence blocks the beam at that cell. Agents of other colours cannot occupy a cell crossed by an active beam.

This blocking is the source of the cooperative structure studied in this thesis. By occupying a position along its own beam, an agent shortens the beam and opens cells beyond it for teammates who are not immune to that colour. Crucially, this action is locally unrewarded: the helper agent receives no direct benefit from blocking its own beam. The reward is issued only when all agents simultaneously occupy their exits, which requires the full team to coordinate.

We measure agent performance throughout this thesis by the **success rate**: the fraction of evaluation episodes in which the whole team succeeds, that is, every agent reaches an exit within the horizon. This is a strict team-level criterion: an episode in which only some agents exit counts as a failure.

An annotated rendering of LLE Level 6 (the canonical hard target used throughout this thesis) is shown in Figure 1.

LLE additionally supports collectible gems and void tiles. Neither is used in this thesis: gems and the incentive-scoring layer they enable are out of scope, and the formal model developed below does not introduce a void-tile type. Levels that contain void tiles can still be handled by treating each void tile as a wall, since both are impassable to agents.

We positioned LLE within the cooperative-MARL benchmark landscape in Chapter 2; the present chapter makes the mechanics precise through a formal model suitable for reasoning about solvability and cooperation within a bounded time horizon.

3.2 Boolean satisfiability

Boolean Satisfiability (SAT) is the problem of deciding whether a propositional formula has a satisfying assignment, i.e. a mapping from Boolean variables to true or false that makes the formula evaluate to true. It is the canonical NP-complete decision problem [33], where NP is the class of decision problems whose yes-instances admit a polynomial-time-checkable certificate.

In practice, SAT solvers operate on formulas in *Conjunctive Normal Form* (CNF). A CNF formula is a conjunction of *clauses*, where each clause is a disjunction of *literals* and a literal is either a variable x or its negation $\neg x$. For example:

$$(x_1 \vee \neg x_2) \wedge (\neg x_1 \vee x_3) \wedge (x_2 \vee x_3)$$

is a CNF formula over three variables. It is satisfied by setting $x_1 = \text{true}$, $x_2 = \text{false}$, $x_3 = \text{true}$, among other assignments.

Modern SAT solvers based on the Davis-Putnam-Logemann-Loveland (DPLL) procedure [34] and Conflict-Driven Clause Learning (CDCL) [35] can handle industrial instances with millions of variables and clauses. Given a CNF formula, a solver either:

- returns a *satisfying assignment* that witnesses the formula is satisfiable (SAT), or
- certifies that *no* satisfying assignment exists (UNSAT).

The UNSAT certificate is as informative as the SAT witness: it proves the non-existence of a solution. This property is central to the cooperation detection mechanism developed in Chapter 5, where UNSAT under a modified encoding is used to certify that no solution exists without the cooperative interaction.

In this thesis, propositional variables are introduced to represent agent positions, laser beam states, and laser activity at each time step. The constraints of the bounded-horizon solvability problem are then encoded as CNF clauses, and a SAT solver is used as a decision oracle. The full encoding (variables, clauses, and the correctness argument) is developed in Chapter 4; the concrete solver used for every empirical run is specified in Section 7.1.

3.3 Problem formalisation

We now define the semantic objects and decision problems used throughout the thesis. Two properties are central: **solvability** (whether there exists a valid joint trajectory bringing all agents to exits within a bounded horizon) and the **cooperation requirement** (whether every such trajectory must rely on the same-colour beam-blocking mechanism). Both are formally decidable and certified by the solver for every accepted level; the SAT encoding itself is developed in Chapter 4. Throughout, the number of agents n_a satisfies $1 \leq n_a \leq 4$ in line with the LLE engine; this bound is an engine artefact, not a limitation of the encoding, which remains correct for any finite n_a.

Definition 3.1 (LLE Level)

An LLE level is a tuple $L = (H, W, C, s, \mathcal{W}, \mathcal{S}, \mathcal{E})$ where:

- $H, W \in \mathbb{N}^+$: grid height and width.
- $C = \{0, 1, ..., n_a - 1\}$: set of agent colours, with $1 \leq n_a \leq 4$.
- $s : C \to P$: injective map assigning an initial position to each agent.
- $\mathcal{W} \subset P$: set of wall positions.
- $\mathcal{S} \subseteq C \times D \times P$: set of laser sources. A source $(c, d, p) \in \mathcal{S}$ emits a laser of colour c in direction d from position p.
- $\mathcal{E} \subset P$: set of exit positions, with $|\mathcal{E}| = |C|$.

Here $P = \{(x, y) \mid 0 \leq x < W, 0 \leq y < H\}$ is the set of grid positions (x is the column index and y the row index, image-coordinates convention) and $D = \{N, S, E, W\}$ is the set of laser directions. We further denote by

$$P_{\text{src}} = \{p \in P \mid \exists c \in C, d \in D : (c, d, p) \in \mathcal{S}\}$$
$$C_{\text{src}} = \{c \in C \mid \exists d \in D, p \in P : (c, d, p) \in \mathcal{S}\}$$

the set of grid positions that hold a laser source and the set of colours that have one, respectively.

We assume the following structural invariants on every level:

1. the sets $s(C)$, $\mathcal{W}$, $\mathcal{E}$, and P_{src} are pairwise disjoint;
2. each colour has at most one laser source. This is a simplifying assumption of the present thesis, used by the SAT encoding of Chapter 4; the LLE engine itself permits multiple sources per colour.

We define two beam-propagation rules. The **standard** rule mirrors the actual LLE behaviour: an agent of matching colour can occupy a cell of its own beam and thereby stop the beam at that cell. The **strict** rule is a counterfactual in which lasers ignore matching agents and behave like unblockable rays, used in Chapter 5 to test whether a standard solution actually relies on the blocking mechanism.

Definition 3.2 (Active Beams)

Fix a time step t and a joint position map $p_t : C \to P$.

For a source $(c, d, p_s) \in \mathcal{S}$, consider the grid ray starting at p_s and extending in direction d.

- Under the **standard beam semantics**, the beam of source (c, d, p_s) is active on every cell of that ray up to, but not including, the first wall or the cell occupied by the unique agent of colour c (if any such cell lies on the ray).
- Under the **strict beam semantics**, the beam of source (c, d, p_s) is active on every cell of that ray up to, but not including, the first wall.

For $c \in C_{\text{src}}$, a laser of colour c is active at a position when the beam of the unique source of colour c is active there.

Definition 3.3 (Valid Joint Trajectory)

A **horizon** $T_{\max} \in \mathbb{N}^+$ is the maximum number of joint moves allowed in a candidate solution.

A **joint trajectory** of length $T_{\max}$ is a sequence of joint positions $\sigma = \left(p_0, p_1, ..., p_{T_{\max}}\right)$ where each $p_t : C \to P$ assigns a position to every agent at time step t.

A joint trajectory is **valid** if it satisfies the following conditions:

- **Initialisation:** $\forall c \in C, p_0(c) = s(c)$.
- **Movement:** $\forall t < T_{\max}, \forall c \in C$, $p_{t+1}(c)$ is reachable from $p_{t(c)}$ in one step: the agent may stay in place or move to a 4-neighbouring cell. Walls and laser-source cells are the only blocked destinations; exit tiles and cells crossed by the agent's own-colour laser are admissible.
- **No collision:** $\forall t, \forall c_1, c_2 \in C, c_1 \neq c_2, p_{t(c_1)} \neq p_{t(c_2)}$.
- **No following conflicts:** $\forall t < T_{\max}, \forall c_1, c_2 \in C, c_1 \neq c_2$, agents may not enter a cell occupied by another agent at the previous time step; that is, $p_{t+1}(c_1) \neq p_{t(c_2)}$ and $p_{t(c_1)} \neq p_{t+1}(c_2)$. This follows the LLE engine convention: agents act simultaneously without coordinating their moves, so an agent cannot enter a cell that another agent only vacates in the same step; it must wait one step. The same rule rules out direct swaps.
- **Laser safety:** No agent c_1 occupies a cell at time t where a laser of colour $c_2 \in C_{\text{src}}$, with $c_2 \neq c_1$, is active under the standard beam semantics of Definition 3.2. The restriction $c_2 \neq c_1$ encodes **immunity**: an agent is unaffected by (and may stand inside) a laser of its own colour.
- **Stay on exit:** If $p_{t(c)} \in \mathcal{E}$ for some $t < T_{\max}$, then $p_{t+1}(c) = p_{t(c)}$. This follows the LLE engine convention: once an agent enters an exit tile, it remains there for the rest of the episode.

Definition 3.4 (Solvability)

A level L is **solvable** with horizon $T_{\max}$ if there exists a valid joint trajectory σ of length $T_{\max}$ such that the set of occupied positions at time $T_{\max}$ is exactly the set of exits:

$$\left\{p_{T_{\max}}(c) \mid c \in C\right\} = \mathcal{E}$$

A level is **solvable** without qualification if it is solvable for some $T_{\max} \in \mathbb{N}^+$.

The restriction to a bounded horizon is natural for the SAT encoding. In the model studied here, beam activity at time t is a deterministic function of the joint position map p_t, so the full game state is determined by the joint agent positions. Any valid trajectory that visits the same joint position map twice can have the intervening segment truncated without affecting reachability of later states; the stay-on-exit rule does not break this argument, since any agent on an exit at the start of the loop must remain on that same exit throughout the loop. Hence, if a level is solvable at all, it admits a valid trajectory whose length is at most the number of collision-free joint position maps, which is bounded by $|P|^{n_a}$. A sufficient $T_{\max}$ therefore always exists. In practice $T_{\max}$ is chosen by the generation process, and all formal guarantees in this thesis are stated relative to that choice.

Definition 3.5 (Bounded-Horizon LLE Solvability Problem)

The **bounded-horizon LLE solvability problem** is the following decision problem.

- **Input:** an LLE level L and a horizon $T_{\max} \in \mathbb{N}^+$.
- **Question:** does L admit a valid joint trajectory of length $T_{\max}$ whose final occupied positions are exactly the exits?

Definition 3.6 (Strict Beam Semantics)

A **strict trajectory** of length $T_{\max}$ is defined exactly as in Definition 3.3, except that beam activity is computed using the strict beam semantics of Definition 3.2: same-colour occupancy no longer blocks the corresponding beam.

In the model studied here, the relevant cooperative action is precisely this same-colour beam-blocking mechanism: an agent occupies a position that would otherwise allow its own beam to continue, thereby opening a path for another agent.

Definition 3.7 (Cooperation Requirement with Horizon $T_{\max}$)

A level L is said to **require cooperation** with horizon $T_{\max}$ if:

- L is solvable under the standard semantics.
- L admits no strict trajectory of length $T_{\max}$ whose final positions occupy all exit tiles.

When the horizon is clear from context, we simply say that L requires cooperation. This bounded form is the one used throughout the rest of the methods chapter, since both the SAT solver and the cooperation detector operate at a fixed finite horizon.

Chapter 4

SAT-based solver

4.1 Notation

We reuse the level objects introduced in Section 3.3: the grid dimensions H and W, the position set P, the colour set C, the direction set D, the wall set $\mathcal{W}$, the source set $\mathcal{S}$, the exit set $\mathcal{E}$, and the start map s.

The SAT reduction introduces a finite horizon $T_{\max} \in \mathbb{N}^+$, which is the maximum number of joint moves allowed in the bounded decision problem, together with the discrete time-step set $T = \{0, 1, ..., T_{\max}\}$.

The sets P_{src} and C_{src} are as defined in Section 3.3. We recall that each colour appears in at most one laser source; under this assumption, when a source of colour c exists, its position and direction are uniquely determined by c. We write $s(c)$ for the initial position of agent $c \in C$, as defined in Definition 3.1.

For the clause-count and complexity statements throughout this chapter, we use the following shorthand:

Shorthand notation

$n_a = \lvert C \rvert$	number of agents (and exits, since $\lvert\mathcal{E}\rvert = \lvert C \rvert$)
$p = \lvert P \rvert = HW$	number of grid positions
$s = \lvert\mathcal{S}\rvert$	number of laser sources
$e = \lvert\mathcal{E}\rvert$	number of exits
$\tau = T_{\max} + 1$	number of time steps in $T = \{0, ..., T_{\max}\}$
$V = P \setminus (\mathcal{W} \cup P_{\text{src}})$	walkable cells (no walls, no laser sources)

4.2 Propositional variables

We introduce three families of propositional variables.

- $a_{c,x,y,t}$: true iff agent $c \in C$ occupies position $(x, y) \in P$ at time step $t \in T$.

- $b_{c,d,x,y,t}$: true iff the beam emitted by the source $(c, d, p_s) \in \mathcal{S}$ is active at position $(x, y) \in P$ at time $t \in T$.
- $l_{c,x,y,t}$: true iff a laser of colour $c \in C_{\text{src}}$ is active at position $(x, y) \in P$ at time $t \in T$.

4.3 Constraints and logical encoding

We now formalise the constraint families used in the reduction. Each constraint groups a family of CNF clauses corresponding to one logical component of the bounded-horizon decision problem. For every constraint we state the clauses in CNF and report two bounds: the maximum number of **clauses** generated as a function of the input parameters, and the maximum number of **literals per clause**. These bounds feed directly into the polynomial-size analysis of Section 4.4 and use the shorthand notation introduced at the end of Section 4.1.

4.3.1 Initialisation

Constraint 4.1 (Agent initialisation)

Each agent $c \in C$ is placed at its designated starting position $s(c)$ at $t = 0$; all other positions are unoccupied by that agent:

$$\bigwedge_{c \in C} \bigwedge_{(x,y) \in P} \begin{cases} a_{c,x,y,0} & \text{if } (x, y) = s(c) \\ \neg a_{c,x,y,0} & \text{otherwise} \end{cases}$$

Bounds. Exactly $n_a p$ unit clauses; 1 literal per clause.

Constraint 4.2 (Laser-source initialisation)

For each laser source $(c, d, (x_s, y_s)) \in \mathcal{S}$, the beam is active at its origin at every time step:

$$\bigwedge_{(c,d,(x_s,y_s)) \in \mathcal{S}} \bigwedge_{t \in T} b_{c,d,x_s,y_s,t}$$

Bounds. Exactly $s\tau$ unit clauses; 1 literal per clause.

4.3.2 Agent movement

We define the set of positions reachable from (x, y) in one step as (x, y) itself together with its four grid neighbours, excluding walls and laser-source positions:

$$\begin{aligned} \text{next}(x, y) = \{(x', y') \in \{(x, y), (x, y-1), (x+1, y), (x, y+1), (x-1, y)\} \mid \\ (x', y') \in P, (x', y') \notin \mathcal{W}, (x', y') \notin P_{\text{src}}\} \end{aligned}$$

The relation next is symmetric: $(x', y') \in \text{next}(x, y) \iff (x, y) \in \text{next}(x', y')$, because every LLE move (including staying in place) is reversible by the opposite action. The same set therefore serves

as both successor and predecessor relation, which is why the backward-consistency constraint below quantifies over $\text{next}(x, y)$ rather than introducing a separate $\text{prev}(x, y)$.

The constraint "each agent occupies at most one position at each time step" can be encoded in two ways, and we present both because they offer different trade-offs in clause count. The CNF passed to the solver contains **exactly one** of them, never both: the choice is made at run-time when the encoding is built. Both formulations admit the same set of satisfying assignments (the same legal joint trajectories) on top of the forward-consistency clauses; they differ only in how many clauses they generate and, consequently, in solver run-time. Section 7.2.1 compares the two empirically. Schematically, the movement-related clauses of the CNF are

$$\text{CNF}_{\text{movement}} = \text{forward consistency} \cup \begin{cases} \text{global uniqueness} & \text{(formulation A)} \\ \text{local uniqueness} \cup \text{backward consistency} & \text{(formulation B)} \end{cases}$$

Constraint 4.3 (Forward consistency)

If agent c is at position (x, y) at time t, it must be at some position in $\text{next}(x, y)$ at time $t + 1$:

$$\bigwedge_{c \in C} \bigwedge_{t=0}^{T_{\max}-1} \bigwedge_{(x,y) \in P} a_{c,x,y,t} \rightarrow \bigvee_{(x',y') \in \text{next}(x,y)} a_{c,x',y',t+1}$$

$$\Updownarrow$$

$$\bigwedge_{c \in C} \bigwedge_{t=0}^{T_{\max}-1} \bigwedge_{(x,y) \in P} \neg a_{c,x,y,t} \vee \bigvee_{(x',y') \in \text{next}(x,y)} a_{c,x',y',t+1}$$

Bounds. At most $n_a(\tau - 1)p$ clauses; at most 6 literals per clause (one head literal plus $|\text{next}(x, y)| \leq 5$ disjuncts).

Constraint 4.4 (Global uniqueness, formulation A only)

No two distinct positions can both be occupied by agent c at time t. The clauses at $t = 0$ are already implied by initialisation, so the family ranges over $t = 1, \ldots, T_{\max}$:

$$\bigwedge_{c \in C} \bigwedge_{t=1}^{T_{\max}} \bigwedge_{(x_1,y_1) \in P} \bigwedge_{\substack{(x_2,y_2) \in P, \\ (x_2,y_2) \neq (x_1,y_1)}} \neg a_{c,x_1,y_1,t} \vee \neg a_{c,x_2,y_2,t}$$

Bounds. Exactly $n_a(\tau - 1)\binom{p}{2}$ clauses; 2 literals per clause.

Constraint 4.5 (Local uniqueness, formulation B only)

No two distinct positions in $\text{next}(x, y)$ can simultaneously be occupied by agent c at time $t + 1$:

$$\bigwedge_{c \in C} \bigwedge_{t=0}^{T_{\max}-1} \bigwedge_{(x,y) \in P} \bigwedge_{(x',y') \in \text{next}(x,y)} \bigwedge_{\substack{(x'',y'') \in \text{next}(x,y), \\ (x'',y'') \neq (x',y')}} \neg a_{c,x',y',t+1} \vee \neg a_{c,x'',y'',t+1}$$

Bounds. At most $10n_a(\tau - 1)p$ clauses (since $\binom{|\text{next}(x,y)|}{2} \leq \binom{5}{2} = 10$); 2 literals per clause.

Constraint 4.6 (Backward consistency, formulation B only)

If agent c is at position (x, y) at time $t + 1$, it must have been at some position in $\text{next}(x, y)$ at time t (recall that next is symmetric, so it also describes the cells from which (x, y) can be reached):

$$\bigwedge_{c \in C} \bigwedge_{t=0}^{T_{\max}-1} \bigwedge_{(x,y) \in P} a_{c,x,y,t+1} \rightarrow \bigvee_{(x',y') \in \text{next}(x,y)} a_{c,x',y',t}$$

$$\Updownarrow$$

$$\bigwedge_{c \in C} \bigwedge_{t=0}^{T_{\max}-1} \bigwedge_{(x,y) \in P} \neg a_{c,x,y,t+1} \vee \bigvee_{(x',y') \in \text{next}(x,y)} a_{c,x',y',t}$$

Bounds. At most $n_a(\tau - 1)p$ clauses; at most 6 literals per clause.

Constraint 4.7 (No simultaneous occupation)

Two distinct agents cannot share the same position at the same time, nor can they swap positions between consecutive time steps:

$$\bigwedge_{c_1 \in C} \bigwedge_{c_2 \in C, c_2 \neq c_1} \bigwedge_{(x,y) \in P} \bigwedge_{t \in T} \neg a_{c_1,x,y,t} \vee \neg a_{c_2,x,y,t}$$

$$\bigwedge_{c_1 \in C} \bigwedge_{c_2 \in C, c_2 \neq c_1} \bigwedge_{(x,y) \in P} \bigwedge_{t=0}^{T_{\max}-1} \neg a_{c_1,x,y,t+1} \vee \neg a_{c_2,x,y,t}$$

$$\bigwedge_{c_1 \in C} \bigwedge_{c_2 \in C, c_2 \neq c_1} \bigwedge_{(x,y) \in P} \bigwedge_{t=0}^{T_{\max}-1} \neg a_{c_1,x,y,t} \vee \neg a_{c_2,x,y,t+1}$$

Bounds. At most $\binom{n_a}{2}(3\tau - 2)p$ clauses; 2 literals per clause.

Constraint 4.8 (Victory condition)

Each exit must be occupied by at least one agent at time $T_{\max}$:

$$\bigwedge_{(x,y)\in\mathcal{E}} \bigvee_{c\in C} a_{c,x,y,T_{\max}}$$

This clause only asks that the exits be **covered**; but since there are exactly n_a agents, exactly n_a exits, and no two agents can share a cell (Constraint 4.7), covering the n_a exits with n_a distinct agents forces every agent onto a distinct exit at $T_{\max}$. Every agent therefore reaches an exit at the final step.

Bounds. Exactly e clauses; at most n_a literals per clause.

Constraint 4.9 (Stay on exit)

Once an agent reaches an exit, it remains there for all subsequent time steps:

$$\bigwedge_{c\in C} \bigwedge_{(x,y)\in\mathcal{E}} \bigwedge_{t=0}^{T_{\max}-1} \neg a_{c,x,y,t} \vee a_{c,x,y,t+1}$$

Bounds. Exactly $n_a e(\tau-1)$ clauses; 2 literals per clause.

4.3.3 Laser activity

We define $\text{next}_{d(x,y)}$ as the position immediately adjacent to (x,y) in direction d:

$$\text{next}_{d(x,y)} = \begin{cases} (x, y-1) \text{ if } d = N \\ (x+1, y) \text{ if } d = E \\ (x, y+1) \text{ if } d = S \\ (x-1, y) \text{ if } d = W \end{cases}$$

Constraint 4.10 (Walls block beams)

A beam cannot be active at a wall position:

$$\bigwedge_{(c,d,p_s)\in\mathcal{S}} \bigwedge_{(x,y)\in\mathcal{W}} \bigwedge_{t\in T} \neg b_{c,d,x,y,t}$$

Bounds. Exactly $s\ |\mathcal{W}|\ \tau$ unit clauses; 1 literal per clause.

Constraint 4.11 (Beam propagation)

Beam propagation clauses are instantiated only when the successor cell $(x', y') = \text{next}_{d(x,y)}$ lies inside the grid, is not a wall, and is not itself a source cell. Source cells are handled separately by Constraint 4.2. Under these conditions, the beam is active at (x', y') iff it is active at (x, y) and no agent of colour c occupies (x', y'):

$$\bigwedge_{(c,d,p_s)\in\mathcal{S}} \bigwedge_{t\in T} \bigwedge_{\substack{(x,y)\in P\setminus\mathcal{W},\\ \text{next}_{d(x,y)}\in P\setminus\mathcal{W},\\ \text{next}_{d(x,y)}\notin P_{\text{src}}}} b_{c,d,x',y',t} \leftrightarrow \left(b_{c,d,x,y,t} \wedge \neg a_{c,x',y',t}\right)$$

$$\Updownarrow$$

$$\bigwedge_{(c,d,p_s)\in\mathcal{S}} \bigwedge_{t\in T} \bigwedge_{\substack{(x,y)\in P\setminus\mathcal{W},\\ \text{next}_{d(x,y)}\in P\setminus\mathcal{W},\\ \text{next}_{d(x,y)}\notin P_{\text{src}}}} \neg b_{c,d,x,y,t} \vee a_{c,x',y',t} \vee b_{c,d,x',y',t}$$

$$\bigwedge_{(c,d,p_s)\in\mathcal{S}} \bigwedge_{t\in T} \bigwedge_{\substack{(x,y)\in P\setminus\mathcal{W},\\ \text{next}_{d(x,y)}\in P\setminus\mathcal{W},\\ \text{next}_{d(x,y)}\notin P_{\text{src}}}} b_{c,d,x,y,t} \vee \neg b_{c,d,x',y',t}$$

$$\bigwedge_{(c,d,p_s)\in\mathcal{S}} \bigwedge_{t\in T} \bigwedge_{\substack{(x,y)\in P\setminus\mathcal{W},\\ \text{next}_{d(x,y)}\in P\setminus\mathcal{W},\\ \text{next}_{d(x,y)}\notin P_{\text{src}}}} \neg a_{c,x',y',t} \vee \neg b_{c,d,x',y',t}$$

Bounds. At most $3s\tau p$ clauses (three CNF clauses per admissible propagation edge); at most 3 literals per clause.

Constraint 4.12 (Link between beam and laser variables)

Since each colour has at most one source in the instances considered here, the laser variable $l_{c,x,y,t}$ is true at a position iff the beam of the unique source of colour c is active there:

$$\bigwedge_{(c,d,p_s)\in\mathcal{S}} \bigwedge_{(x,y)\in P} \bigwedge_{t\in T} b_{c,d,x,y,t} \leftrightarrow l_{c,x,y,t}$$

$$\Updownarrow$$

$$\bigwedge_{(c,d,p_s)\in\mathcal{S}} \bigwedge_{(x,y)\in P} \bigwedge_{t\in T} \left(\neg b_{c,d,x,y,t} \vee l_{c,x,y,t}\right) \wedge \left(\neg l_{c,x,y,t} \vee b_{c,d,x,y,t}\right)$$

Bounds. Exactly $2s\tau p$ clauses; 2 literals per clause.

Constraint 4.13 (Agents cannot step on active lasers)

An agent of colour c_1 cannot occupy a position where an active laser of some source colour $c_2 \in C_{\text{src}}$, with $c_2 \neq c_1$, is present. Agents are immune only to lasers of their own colour:

$$\bigwedge_{c_1 \in C} \bigwedge_{\substack{c_2 \in C_{\text{src}}, \\ c_2 \neq c_1}} \bigwedge_{(x,y) \in P} \bigwedge_{t \in T} l_{c_2,x,y,t} \rightarrow \neg a_{c_1,x,y,t}$$

$$\Updownarrow$$

$$\bigwedge_{c_1 \in C} \bigwedge_{\substack{c_2 \in C_{\text{src}}, \\ c_2 \neq c_1}} \bigwedge_{(x,y) \in P} \bigwedge_{t \in T} \neg l_{c_2,x,y,t} \vee \neg a_{c_1,x,y,t}$$

Bounds. At most $n_a s \tau p$ clauses; 2 literals per clause.

4.4 Clause complexity and polynomial size

To show that the reduction has polynomial size, it is enough to bound the number of clauses generated as a function of the input parameters introduced in Section 4.1.

We count **clauses** rather than **literals**. A CNF formula is a conjunction of clauses, each clause being a disjunction of literals. The total literal count is at most a constant factor larger than the clause count in our encoding, since every clause family generated above contains at most $\max(|\text{next}(u)| + 1, n_a) \leq 6$ literals per clause (the worst case is forward or backward consistency, with one head literal and $|\text{next}(u)| \leq 5$ disjuncts). Counting clauses is therefore sufficient to bound the formula size polynomially.

Summing the per-constraint bounds reported in Chapter 4 (the **Bounds** lines of Constraints 4.1–4.13) gives the total clause count as a polynomial in n_a, p, s, and τ. With the global formulation (formulation A, using Constraint 4.4) the dominant term is

$$O(n_a \tau p^2 + n_a^2 \tau p + n_a s \tau p)$$

while with the local formulation (formulation B, using Constraints 4.5 and 4.6) it is

$$O(n_a \tau p + n_a^2 \tau p + n_a s \tau p)$$

These are **asymptotic upper bounds**. The local formulation dominates the global one asymptotically, since its uniqueness term is linear in p rather than quadratic; the dominance is not pointwise, however. On sufficiently small grids the global formulation actually produces fewer clauses than the local one, because the local formulation pays a constant overhead the global one avoids: a separate backward-consistency family of size $n_a(\tau - 1)p$ (Constraint 4.6) and a per-cell pairs sum $\sum_{u \in V} \binom{|\text{next}(u)|}{2}$ which is large relative to $\binom{p}{2}$ when p itself is small. The grid size at which the global formulation's quadratic uniqueness term overtakes this local overhead is the **crossover**: below it the global formulation is the more compact, above it the local one. We therefore keep both formulations and benchmark their actual clause counts and solver runtimes in Section 7.2.1.

The same loops introduce $O((n_a + s)p\tau)$ propositional variables (agent, beam, and laser indices), also polynomial in the input parameters. The constraints above quantify agent-position predicates over the

full position set P; the implementation may safely restrict these to the walkable subset V, since clauses generated at walls and laser-source cells are vacuous under the initialisation and the `next` filtering. Beam variables at wall cells are forced false by Constraint 4.10 and play no role in any satisfying assignment; the implementation may skip them without affecting correctness.

Since each clause is generated by a simple bounded computation inside these loops, the reduction itself is computable in polynomial time as well. This justifies the claim that bounded-horizon LLE solvability is polynomial-time reducible to SAT.

4.5 Correctness of the reduction

Proposition 4.14 (Correctness of the SAT Reduction)

Let L be an LLE level and let $T_{\max}$ be a horizon. For either movement formulation described above, the CNF formula $\Phi(L, T_{\max})$ is satisfiable if and only if there exists a valid joint trajectory of length $T_{\max}$ for L.

Proof.

For soundness, assume $\Phi(L, T_{\max})$ is satisfiable. Initialisation fixes exactly one start position for each agent at time 0. For the global formulation, forward consistency together with pairwise exclusion ensures by induction on time that each agent occupies exactly one legal position at every later step. For the local formulation, the same conclusion follows from forward consistency, local exclusivity, and backward consistency. We may therefore derive a joint trajectory by setting $p_{t(c)} = (x, y)$, where (x, y) is, for each colour c and time t, the unique position at which the variable $a_{c,x,y,t}$ is true. The movement clauses enforce legal motion between consecutive steps; the collision clauses enforce both simultaneous separation and the no-following-conflict rule; the laser clauses enforce safety with respect to active beams; and the exit clauses enforce the terminal condition. Hence the extracted trajectory is valid.

For completeness, assume a valid joint trajectory of length $T_{\max}$ is given. Set each $a_{c,x,y,t}$ according to whether agent c occupies (x, y) at time t in the trajectory. Set beam variables $b_{c,d,x,y,t}$ and laser variables $l_{c,x,y,t}$ according to the deterministic beam dynamics induced by the same agent positions. Every clause family is then satisfied by construction: initialisation matches the start state; the movement, uniqueness, and collision clauses match the trajectory semantics; the propagation iff and beam-laser link clauses hold because b and l are set exactly to those deterministic values; the agent-laser blocking clauses hold because trajectory validity already forbids any agent from standing on an active laser of a different colour; and the final positions occupy all exits. Therefore $\Phi(L, T_{\max})$ is satisfiable. □

4.6 Complexity-theoretic consequences

We can now state the consequence for the decision problem introduced in Definition 3.5.

The bounded-horizon LLE solvability problem lies in **NP**. A candidate trajectory can be verified in polynomial time by simulating the joint execution and checking that all agents occupy the exit tiles at the end without violating the movement, collision, and laser constraints defined in Section 3.3.

Combined with the polynomial-time construction above and Proposition 4.14, this shows that bounded-horizon LLE solvability is polynomial-time many-one reducible to SAT:

$$\text{LLE-Solvability} \leq_p \text{SAT}$$

Thus bounded-horizon LLE solvability is **at most as hard as SAT**: any SAT algorithm yields an algorithm for this decision problem with only polynomial overhead from the reduction.

Whether bounded-horizon LLE solvability is also **NP-hard** remains open in the present work. Establishing NP-hardness would require a polynomial-time reduction in the opposite direction, from a known NP-hard problem to LLE solvability. This thesis does not claim such a result.

We must also distinguish proved statements from standard complexity-theoretic beliefs. Whether $\text{P} = \text{NP}$ remains open, so statements here about worst-case difficulty should be read only through the formal claims we have established: SAT is NP-complete, bounded-horizon LLE solvability belongs to NP, and the reduction above places bounded-horizon LLE solvability within the polynomial-time many-one image of SAT.

In practice, this positioning explains why a SAT-based approach is attractive: the solver inherits the strong empirical performance of modern CDCL SAT solvers on many structured instances, even though the worst-case complexity remains exponential.

Chapter 5

Cooperation detection

5.1 Strict SAT encoding

Recall from Definition 3.6 that strict beam semantics changes only one aspect of the dynamics: same-colour occupancy no longer blocks the corresponding beam. Same-colour immunity is unchanged.

Accordingly, the strict SAT encoding keeps the standard laser-safety clauses and replaces only the same-colour beam-propagation rule. For every source $(c, d, p_s) \in \mathcal{S}$, every admissible propagation edge from (x, y) to (x', y') (in the sense of Chapter 4, i.e. a successor inside the grid, not a wall, and not a source cell), and every time step $t \in T$, the strict encoding uses

$$b_{c,d,x',y',t} \leftrightarrow b_{c,d,x,y,t}$$

instead of the standard equivalence

$$b_{c,d,x',y',t} \leftrightarrow \big(b_{c,d,x,y,t} \wedge \neg a_{c,x',y',t}\big).$$

Thus the beam continues through agents of the matching colour instead of stopping at them. We denote the resulting CNF formula by $\Phi_{\text{strict}}(L, T_{\max})$ and the corresponding solver by StrictSolver.

5.2 Why this captures cooperation

Under the LLE mechanics studied here, the cooperative action of interest is for an agent to occupy a cell that would otherwise allow its own beam to continue, thereby making another agent's path safe. Standard solvability allows this beam-blocking mechanism; strict solvability removes it.

Therefore, if a level is solvable under the standard semantics but unsatisfiable under the strict semantics, every successful standard solution must rely on at least one same-colour beam-blocking step.

5.3 Formal theorem and proof

Theorem 5.1 (Cooperation Detection Criterion)

Let L be an LLE level and $T_{\max}$ a time horizon. Then L requires cooperation with horizon $T_{\max}$ if and only if $\Phi(L, T_{\max})$ is satisfiable and $\Phi_{\text{strict}}(L, T_{\max})$ is unsatisfiable.

Proof.

($\rightarrow$) Assume that L requires cooperation with horizon $T_{\max}$. By Definition 3.7, L is solvable under the standard semantics, so $\Phi(L, T_{\max})$ is satisfiable. Suppose for contradiction that $\Phi_{\text{strict}}(L, T_{\max})$ is also satisfiable. Then there exists a strict trajectory whose final positions occupy all exit tiles. Since strict beam semantics differs from the standard one only by removing same-colour beam-blocking, such a trajectory is also a valid standard trajectory that succeeds without using that mechanism. This contradicts Definition 3.7. Therefore $\Phi_{\text{strict}}(L, T_{\max})$ is unsatisfiable.

($\leftarrow$) Assume that $\Phi(L, T_{\max})$ is satisfiable and $\Phi_{\text{strict}}(L, T_{\max})$ is unsatisfiable. The first condition implies that L is solvable under the standard semantics. Suppose, for contradiction, that some successful standard trajectory uses no same-colour beam-blocking step: no agent c ever stands on a cell of the unblocked beam path of the source of colour c. Under that hypothesis the two semantics produce identical beam states for this trajectory: the standard rule blocks the beam of colour c only at a cell occupied by agent c, and by assumption no such occupancy occurs, so each beam reaches the same wall or grid boundary as it would under the strict rule. The same variable assignment therefore satisfies $\Phi_{\text{strict}}(L, T_{\max})$, contradicting unsatisfiability. Hence every successful standard trajectory must use at least one same-colour beam-blocking step, so L requires cooperation with horizon $T_{\max}$. □

5.4 Horizon-dependence of the cooperation criterion

Theorem 5.1 binds cooperation to a fixed horizon $T_{\max}$. This is not an artefact of the SAT encoding: it is the only sense in which cooperation is decidable in this framework. Solvability itself is a horizon-indexed property (Definition 3.4), and so is the companion cooperation requirement (Definition 3.7). The horizon is therefore a true parameter of the cooperation label, and the same level can switch classification when $T_{\max}$ changes.

The mechanism behind this sensitivity is straightforward. The strict SAT encoding refuses same-colour beam-blocking but leaves every other movement and timing constraint identical to the standard one. Hence the strict solver can find a satisfying trajectory simply because there exists a path long enough to walk **around** the laser geometry rather than **through** it, even though the short, natural solution would step into the beam and shield another agent. As soon as the horizon $T_{\max}$ admits such a detour, the strict formula becomes satisfiable and the level stops being labelled cooperative, regardless of what the natural solution does.

Example: detour bypass. Figure 2 shows a 4×4 grid that makes this concrete. Two agents start in the top row; their only exits sit in the bottom row, and a single red laser covers two cells of row 1. Running the cooperation analyser on this level at three different horizons reproduces all three failure modes at once (Table 1).

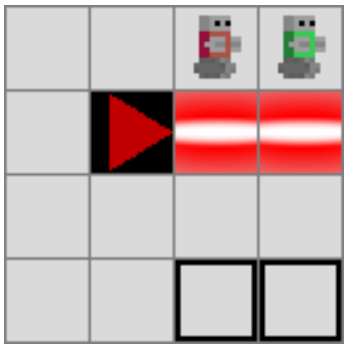

Figure 2: Horizon-dependence demo level. Two agents in the top row; a red laser spans two cells of row 1; two exits in the bottom row.

Horizon range	Solvable	Cooperation required
$0 \leq T_{\max} \leq 2$	no	n/a
$3 \leq T_{\max} \leq 8$	yes	yes
$T_{\max} \geq 9$	yes	no

Table 1: Cooperation classification of the level in Figure 2 across three horizon ranges. For $T_{\max} \leq 2$ no agent has time to reach an exit, so the standard solver returns UNSAT and cooperation is never tested. For $3 \leq T_{\max} \leq 8$ the level is solvable but only via red same-colour blocking, so cooperation is required. For $T_{\max} \geq 9$ a detour around the beam fits: the strict solver finds a trajectory in which the green agent walks around the beam, and cooperation is no longer flagged.

The same level therefore receives three qualitatively different labels depending on the chosen horizon (**unsolvable**, **cooperative**, or **non-cooperative**), without the level itself changing. The phenomenon generalises: any level whose natural short solution uses beam-blocking can be made to look non-cooperative by raising $T_{\max}$ until a geometric detour fits, and can be made to look unsolvable by lowering $T_{\max}$ until even the natural solution does not.

In practice, the criterion correctly identifies cooperation at any **fixed** horizon, but the label it assigns is not an intrinsic property of the level: it depends on the choice of $T_{\max}$. Two opposing failure modes follow. If $T_{\max}$ is set too generously, geometric detours become available and cooperation is under-detected; if it is set too tightly, the standard solver returns UNSAT and the level is rejected as unsolvable before cooperation is tested at all. Neither outcome contradicts Theorem 5.1, which applies strictly relative to the chosen horizon.

The recipe used throughout this thesis is to pick $T_{\max}$ as the smallest horizon at which a representative short solution of the geometry is expected to fit, with a small margin proportional to the grid diameter (concrete values are listed alongside each experimental configuration in Chapter 7). Under that choice, the criterion is tight enough that accepted levels reliably exhibit the intended beam-blocking behaviour at their stated horizon, and loose enough that the underlying standard solver does not spuriously reject solvable instances.

5.5 Practical algorithm

The cooperation detector runs two SAT calls on the same level, in the order shown in Algorithm 1. The first call rejects unsolvable levels; the second decides cooperation by checking the strict counterfactual under the same horizon. Both calls share the same bounded horizon and differ only in the beam-propagation clauses. For benchmark levels, the horizon can be chosen from known solution lengths; for generated levels, it is the user-supplied generation parameter $T_{\max}$.

Algorithm 1: CooperationDetection $(L, T_{\max})$

Input: level L, horizon $T_{\max}$
Output: one of `UNSOLVABLE`, `COOPERATIVE`, `NON_COOPERATIVE`

if $\mathrm{Solver}(L, T_{\max}) = \mathrm{UNSAT}$ **then return** `UNSOLVABLE`
if $\mathrm{StrictSolver}(L, T_{\max}) = \mathrm{UNSAT}$ **then return** `COOPERATIVE`
return `NON_COOPERATIVE`

Algorithm 1: Cooperation detection on level L at horizon $T_{\max}$. The first call rejects unsolvable levels; the second decides cooperation by checking the strict counterfactual under the same horizon.

5.6 Cooperation profiles

Theorem 5.1 yields a **binary** answer: a level either requires cooperation or it does not. Once the binary criterion holds, however, several qualitatively different cooperation structures fall under that single label. This section refines the binary verdict into a **cooperation profile**, computed by a **profile analyser** layered on top of the binary detector. The profile is not an end in itself: it is the **generation target** that lets the generators of Chapter 6 request a specific kind of cooperation, and the quantity whose per-generator distribution we report in Chapter 7.

Two properties shape everything that follows. First, the profile is computed from a **dependency graph** between agents, itself built from a **helper-event set** extracted from **one** satisfying assignment of $\Phi(L, T_{\max})$. It therefore describes the cooperation structure of a single extracted plan and is **not** an invariant of the level: two solver runs can in principle disagree (we return to this in the closing remark of the chapter). It is best read as a sound **filter**: when the analyser returns a profile, the extracted plan provably exhibits it. Second, alongside the label the analyser reports a **necessary-helper set**, identified by colour-wise counterfactual SAT calls; unlike the label, this set **is** a level invariant, and it complements the label without entering the classification decision.

The decision procedure produces one of seven labels, previewed here by their intuition and defined precisely, with geometric examples, in Section 5.6.3:

- **unsolvable**: $\Phi(L, T_{\max})$ is UNSAT.
- **independent**: cooperation is not required.
- **asymmetric**: one-way helping that is never reciprocated.
- **chain**: help relayed along a line of agents.
- **distributed**: one agent helped by several others.
- **mutual**: two agents help each other.
- **fully coupled**: every agent reaches every other.

Every label is constructible within LLE's $1 \leq n_a \leq 4$ regime: `asymmetric` and `fully_coupled` need at least two agents; the remaining labels are reachable from three agents upward. The taxonomy therefore does not require any structural configuration that LLE cannot host.

When several of these patterns occur in the same level, the analyser reports only the one we consider most cooperative, following the priority

$$\text{fully coupled} \succ \text{mutual} \succ \text{distributed} \succ \text{chain} \succ \text{asymmetric}$$

from strongest to weakest, which guarantees every level a unique top-ranked label. This order is total by deliberate choice rather than mathematical necessity: the graph structure alone fixes only a

coarser partial order, and we extend it by ranking all-to-all coupling above a single one-way relation, a judgement another author could reasonably make differently. Section 5.6.4 makes the partial order, its structural basis, and this extension precise.

The remainder of the section is organised as follows. Section 5.6.1 and Section 5.6.2 introduce the two pieces of machinery, helper events and the dependency graph, needed to read the profile catalogue, which is then presented with geometric LLE examples in Section 5.6.3. Section 5.6.4 formalises the priority order used by the classifier when a single graph matches several profiles and clarifies why the order is a labelling rule rather than a hierarchy on the underlying graph categories. The remaining two subsections cover auxiliary analyser outputs that do not affect the label decision: selective-strict semantics (Section 5.6.5) and the necessary-helper set (Section 5.6.6).

5.6.1 Helper events from a SAT model

Given a satisfying assignment of $\Phi(L, T_{\max})$, the analyser recovers the joint trajectory $\sigma = \left(p_0, ..., p_{T_{\max}}\right)$. For each time step t and each ordered pair of distinct agents (c, c'), a **helper event** with helper c, beneficiary c', and time t is recorded if two geometric conditions hold:

1. agent c stands at time t on a cell that lies on the unblocked beam path of a source of colour c, so that source's beam is blocked by c at position $p_{t(c)}$; and
2. agent c' stands at time t on a cell strictly downstream of $p_{t(c)}$ on the same beam path, so c' would lie inside the unblocked beam and is therefore protected by c.

Helper events are a **property of the chosen joint plan**: a different satisfying plan may yield a different set. A helper event records only that c **did** shield c' in this plan, not that c' **needed** c, since in another plan the beneficiary might reach its exit by a different route. Edges therefore assert observed help, not necessity; necessity is certified separately by the necessary-helper set of Section 5.6.6.

5.6.2 Dependency graph

Fix the satisfying assignment σ of $\Phi(L, T_{\max})$ from which the helper events of Section 5.6.1 were read. These events induce a directed graph $G_{L,\sigma} = (V, E)$ on the set of agents,

$$V = C, \quad E = \{(c, c') \mid \exists t : (c, c', t) \text{ is a helper event of } \sigma\},$$

in which an edge $c \to c'$ points from helper to beneficiary. If c helps c' several times, possibly at different time steps or along different beams, the two are still joined by a single edge. The time dimension is otherwise discarded by the graph; to retain one trace of it we expose a separate scalar, the **synchronous width**, the maximum number of distinct helpers active at the same time step. It captures a difficulty axis the time-collapsed graph cannot express, namely whether cooperation must occur **simultaneously** (several agents shielding at once) or can be spread sequentially over the horizon. It is reported alongside the profile label but does not enter the classification decision.

Since the helper events are read from a single model σ, so is $G_{L,\sigma}$; where the model is fixed or immaterial we abbreviate it to G_L.

5.6.3 Profile families

The profile label is the output of a small decision procedure applied to G_L, with the binary cooperation requirement as a hard precondition. When a single graph matches several profiles, a priority order over labels (formalised in Section 5.6.4) selects the strongest one; the two non-cooperative labels (`unsolvable` and `independent`) short-circuit before this order is consulted. We describe each label below with a short geometric example.

Independent. The binary detector returns non-cooperative: $\Phi_{\text{strict}}(L, T_{\max})$ is satisfiable, so no agent ever has to block a beam. The profile analyser short-circuits and returns `independent`. **Example.** A grid with two agents whose direct paths to their respective exits do not cross any beam at all (Figure 3).

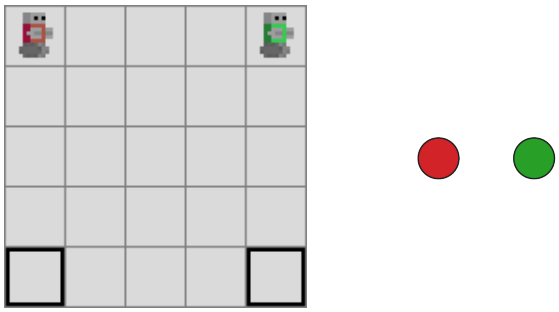

Figure 3: `independent`: two agents with disjoint direct paths to two exits. No laser is present, so the strict-SAT encoding admits the same trajectory as the standard one. The dependency graph (right) has no edges.

Asymmetric. The residual category: cooperation is required, but G_L matches none of the four structural patterns below: no relay (chain), no agent helped by two others (distributed), no reciprocal pair (mutual), and not strongly connected (fully coupled). The label covers both the typical case (a small set of one-way edges) and the degenerate case where the extracted plan contains no observable helper event at all (e.g. when the helper sits at the last cell of its own beam path). **Example.** Two agents of distinct colours; only the red beam blocks the green agent's path to its exit, so the red agent must step into its own beam at some moment to let the green agent pass. The reciprocal situation never arises since there is no green beam. The dependency graph has the single edge red $\rightarrow$ green (Figure 4).

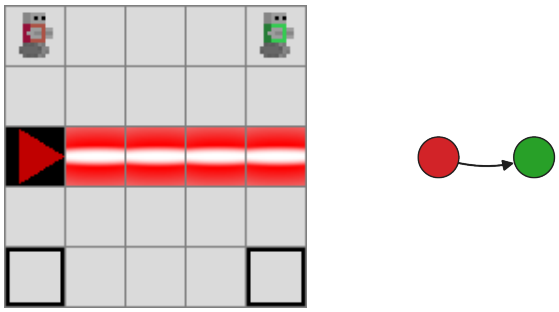

Figure 4: `asymmetric`: one red laser splits the grid horizontally. The red agent (top-left) crosses its own beam unscathed; the green agent (top-right) requires red to block the beam so it can reach the bottom-right exit. Edges: red $\rightarrow$ green.

Chain. Cooperation is required, the dependency graph contains a directed **simple** path of length at least two (a relay through at least three distinct agents: some agent helps one agent and is helped by a different one), and none of the richer profiles below applies. The simplest case is a single linear handoff, as in the example. **Example.** Three agents arranged so that the red agent must shield the green agent across the red beam, the green agent must shield the blue agent across the green beam, and neither the blue agent nor the red agent has any further helping role (Figure 5). The graph is red $\rightarrow$ green $\rightarrow$ blue.

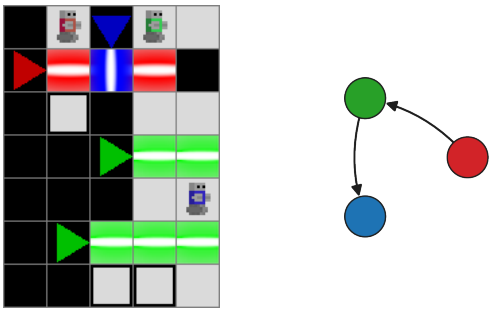

Figure 5: `chain`: three agents and two beams. Walls confine each agent to a separate region so helping flows in one direction only: red helps green across the red beam, green helps blue across the green beam. Edges: $\{\text{red} \to \text{green}, \text{green} \to \text{blue}\}$.

Distributed. At least one agent has in-degree ≥ 2 in the dependency graph. **Example.** Three agents in which two distinct same-colour helpers (the red and green agents) must each block their respective beams to free the blue agent's path to its exit (Figure 6). The edges are $\{\text{red} \to \text{green}, \text{red} \to \text{blue}, \text{green} \to \text{blue}\}$, so the blue agent has in-degree two and the level is `distributed`. (The extra edge red $\to$ green does not promote the level to `mutual` because the reciprocal edge green $\to$ red is absent.) Note that the taxonomy grades cooperation by **in-degree**, how many distinct helpers a single beneficiary depends on, and deliberately gives no separate profile to the mirror case of one helper that shields several beneficiaries (a high **out-degree** hub). We classify the structure of **support received** rather than of **help given**; such a hub therefore contributes only through the in-degrees it induces and, absent any richer pattern, falls under `asymmetric`.

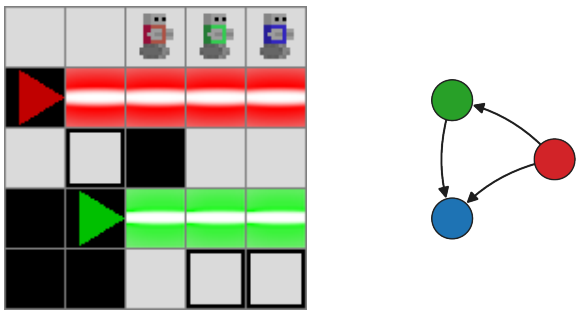

Figure 6: `distributed`: the blue agent must traverse both the red and the green beam to reach its exit, requiring blocking from both other agents. In-degree of the blue agent is two. Edges: $\{\text{red} \to \text{green}, \text{red} \to \text{blue}, \text{green} \to \text{blue}\}$.

Mutual. The dependency graph contains a mutual pair, i.e. both edges $(c, c'), (c', c) \in E$ for some pair $c \neq c'$. **Example.** Two laser sources of distinct colours, each crossing the other agent's path: each agent must block its own beam to shield the other at some point in the plan (Figure 7). Both edges are present, and a level is classified as `mutual` even when additional one-way edges between other pairs also exist, as long as the agents do not all belong to a single strongly connected component. Because **fully coupled** takes priority in the decision order, the `mutual` label is only reachable with $n_a \geq 3$ agents: when $n_a = 2$, a reciprocal pair is itself a strongly connected component spanning the whole agent set (size $2 = n_a > 1$), so the level is labelled `fully_coupled` and the `mutual` branch is never reached. The example in Figure 7 therefore includes a third agent (blue) on an independent path, outside the reciprocal pair, which keeps the largest strongly connected component at size two while $n_a = 3$.

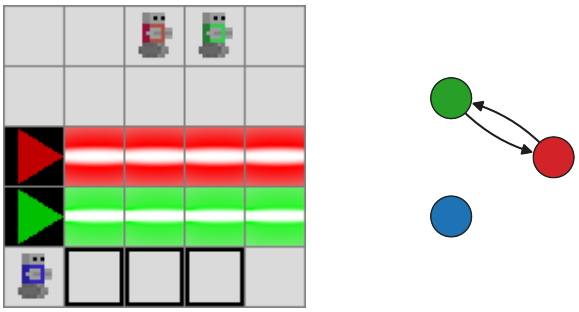

Figure 7: `mutual`: a red and a green beam stacked. Each agent is immune to its own beam but must wait for the other to block the foreign beam before crossing. A third agent (blue) on an independent path is required to keep the profile distinct from `fully_coupled`: with only the reciprocal pair the red and green agents would form a strongly connected component spanning the whole agent set. The blue agent participates in no helper relationship, hence appears as an isolated vertex in the dependency graph. Edges: $\{\text{red} \to \text{green}, \text{green} \to \text{red}\}$.

Fully coupled. The dependency graph is strongly connected: its single strongly connected component (SCC) spans the entire agent set (size $n_a > 1$). **Example.** Three agents in which every pair helps each other, so the dependency graph is the complete digraph $K^*_{n_a}$ (the digraph with all $n_a(n_a - 1)$ edges between distinct agents), which is trivially strongly connected (Figure 8). This is the highest-priority profile and is rare on small grids.

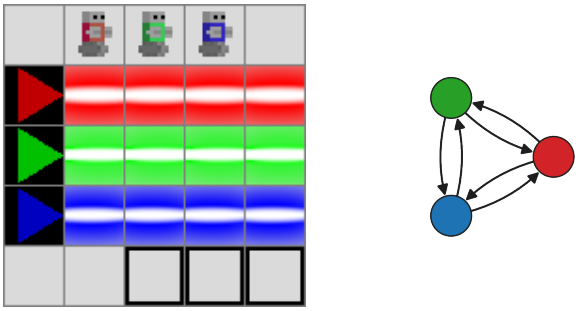

Figure 8: `fully_coupled`: three agents and three beams stacked, so every agent must shield, and be shielded by, both others to reach the bottom row of exits. The complete dependency graph has all six edges between distinct agents (every directed pair among red, green, and blue).

The analyser also exposes the auxiliary scalars used by the decision procedure (longest chain length, largest SCC size, synchronous width) so downstream code can re-classify levels under a different policy without re-running any SAT calls.

5.6.4 Ordering structure of the profile labels

The analyser flags each level with a single label, yet a level can exhibit several cooperation patterns at once: they are not mutually exclusive, and one dependency graph $G_L = (V, E)$, with $V = C$ the agent set and $n_a = |V| > 1$, can satisfy several simultaneously. The analyser must therefore decide which pattern takes precedence, and flagging the single **most important** match is the entire reason for the priority order formalised here. It is cleaner to read the patterns first as predicates on G_L, before deciding which one to report. We use one base predicate and four structural refinements of it:

- $\mathcal{A}$ (asymmetric): cooperation is required, i.e. Theorem 5.1 holds for L. This is the base case; the analyser builds G_L only when cooperation is required, so each structured pattern below is a property of a cooperative level and hence a special case of it: $\mathcal{C}, \mathcal{D}, \mathcal{M}, \mathcal{F} \subseteq \mathcal{A}$.

- $\mathcal{C}$ (chain): G_L contains a directed **simple** path of length at least two, i.e. some agent helps one agent and is helped by a **different** one (equivalently, the relay runs through at least three distinct agents). A bare reciprocal pair $c \leftrightarrow c'$ is therefore not a chain: it revisits c and involves only two agents, so it falls under $\mathcal{M}$ rather than $\mathcal{C}$.
- $\mathcal{D}$ (distributed): some vertex has in-degree at least two.
- $\mathcal{M}$ (mutual): some pair $c \neq c'$ has both $(c, c'), (c', c) \in E$.
- $\mathcal{F}$ (fully coupled): G_L is strongly connected, i.e. its single strongly connected component spans all n_a agents.

Each label coincides with its predicate once the higher-priority predicates are removed (the exact correspondence is the priority below). The `asymmetric` label is therefore the residual $\mathcal{A} \setminus (\mathcal{C} \cup \mathcal{D} \cup \mathcal{M} \cup \mathcal{F})$, assigned only when cooperation is required but none of the four structural patterns holds.

Figure 9 shows these predicates as an Euler diagram, with the base predicate $\mathcal{A}$ enclosing the four structural ones. They overlap rather than partition: chain, distributed, and mutual can co-occur freely, while fully coupled lies inside chain (and inside distributed when a reciprocal pair is present).

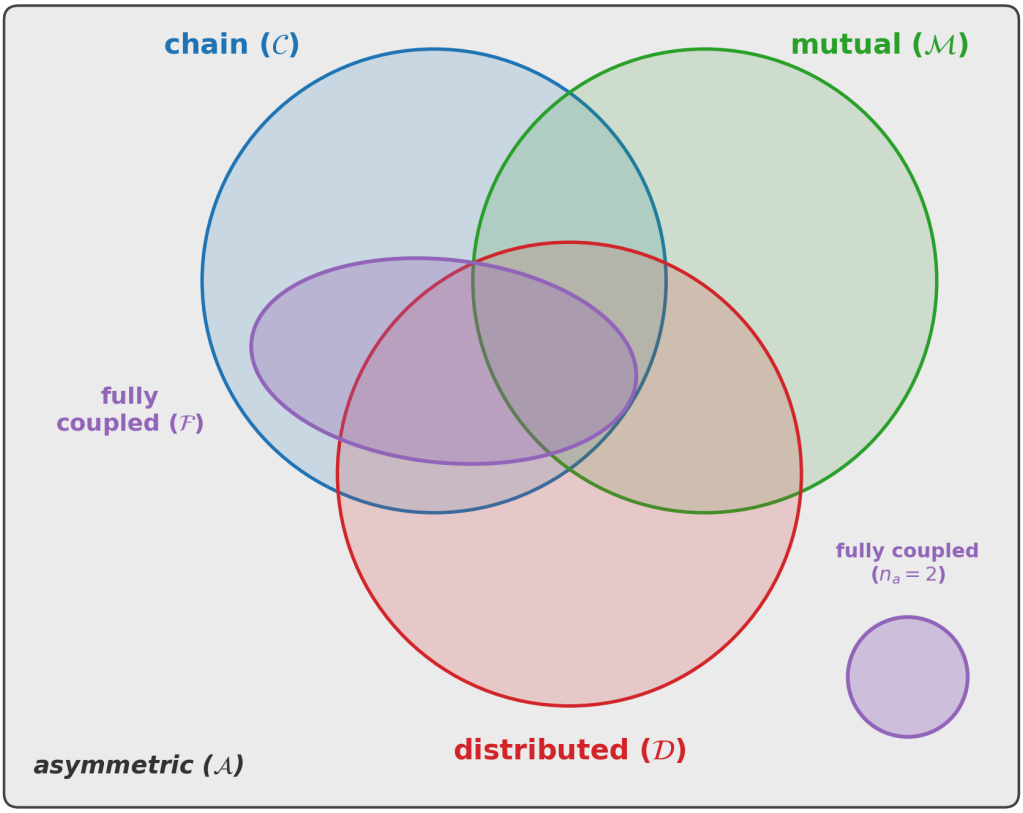

Figure 9: Euler diagram of the cooperation predicates on G_L. The bounding box is the base predicate $\mathcal{A}$; chain ($\mathcal{C}$), distributed ($\mathcal{D}$), and mutual ($\mathcal{M}$) overlap, and fully coupled ($\mathcal{F}$) lies inside $\mathcal{C}$ (the small separate circle is the two-agent case). Each graph is labelled by the priority of Table 2, predicate minus the inner ones, leaving `asymmetric` as the residual area.

The classifier collapses these coexisting predicates into a single label by the priority $\mathcal{F} \succ \mathcal{M} \succ \mathcal{D} \succ \mathcal{C} \succ \mathcal{A}$ and returns the most cooperative pattern a graph exhibits; Table 2 gives each label as its predicate with the higher-priority ones removed. The non-cooperative labels `unsolvable` and `independent` are decided beforehand by the binary detector and do not appear there.

Priority	Profile	Predicate form	Meaning
1	`fully_coupled`	$\mathcal{F}$	every agent reaches every other
2	`mutual`	$\mathcal{M} \setminus \mathcal{F}$	direct reciprocal helping
3	`distributed`	$\mathcal{D} \setminus (\mathcal{M} \cup \mathcal{F})$	one beneficiary, several helpers
4	`chain`	$\mathcal{C} \setminus (\mathcal{M} \cup \mathcal{D} \cup \mathcal{F})$	a relayed dependency (handoff)
5	`asymmetric`	$\mathcal{A} \setminus (\mathcal{C} \cup \mathcal{D} \cup \mathcal{M} \cup \mathcal{F})$	one-way help, no relay

Table 2: Priority-based labelling of cooperative-required levels. The classifier walks the rows from top to bottom and assigns the first label whose predicate matches.

The priority of Table 2 is **total**, but it refines a coarser **partial** order forced by the graph structure, which we read through **subgraph containment**: one profile is at least as cooperative as another exactly when the characteristic subgraph of the second is present in every dependency graph realising the first, equivalently when the first predicate entails the second.

Why some pairs are ordered. `asymmetric` is the base predicate, so it lies below all four structural patterns. Beyond that, the only entailment between two structural patterns is $\mathcal{F} \Rightarrow \mathcal{C}$: a graph strongly connected on $n_a \geq 3$ agents always contains a directed simple path of length two. (If none existed, then for every helping edge $c \to c'$ the beneficiary c' could only help c back, so no agent could have two distinct helpers; strong connectivity would then force every in-degree to be exactly one, making the graph a single cycle through all agents, which itself contains a length-two path.) Hence every fully coupled graph is also a chain, and `fully_coupled` sits above `chain`. (A related fact, the conditional $\mathcal{F} \cap \mathcal{M} \Rightarrow \mathcal{D}$, holds because a strongly connected graph with a reciprocal pair must contain a vertex of in-degree at least two: otherwise every in-degree is one, forcing a single Hamiltonian cycle, which for $n_a \geq 3$ has no reciprocal pair. This containment shapes the Euler diagram, but being conditional it does not on its own order `fully_coupled` against `distributed`.)

Why the rest are incomparable. Every other pair is incomparable, which four **pure** witness graphs on three distinct agents $c_1, c_2, c_3 \in C$ (the agents being identified by their colour) settle at once: each realises the predicates listed in Table 3 and no others. The pure chain, fan-in, and reciprocal pair each isolate a single structural pattern, so any two of `chain`, `distributed`, and `mutual` are separated by a witness that has one but not the other, making the three pairwise incomparable. The three-cycle is fully coupled (and a chain) yet neither distributed nor mutual, while the fan-in and the reciprocal pair are distributed resp. mutual without being strongly connected; hence `fully_coupled` is incomparable to both `distributed` and `mutual`. The intuition is that strong connectivity is a **global** property whereas reciprocity and shared dependence are **local** ones, so neither forces the other.

Witness ($n_a = 3$)	**Edges**	**Realises**	**Excludes**
pure chain	$c_1 \to c_2 \to c_3$	$\mathcal{C}$	$\mathcal{D}, \mathcal{M}, \mathcal{F}$
pure fan-in	$c_1 \to c_3$, $c_2 \to c_3$	$\mathcal{D}$	$\mathcal{C}, \mathcal{M}, \mathcal{F}$
reciprocal pair	$c_1 \leftrightarrow c_2$, c_3 isolated	$\mathcal{M}$	$\mathcal{C}, \mathcal{D}, \mathcal{F}$
three-cycle	$c_1 \to c_2 \to c_3 \to c_1$	$\mathcal{C}, \mathcal{F}$	$\mathcal{D}, \mathcal{M}$

Table 3: Pure witness graphs on $n_a = 3$ agents, each realising the listed predicates and no others. They establish every incomparability in Table 4: for any two patterns, some witness exhibits one without the other.

Since no pattern sits above every other, the relation is a genuine **partial** order, listed in Table 4; the priority of Table 2 is one linear extension that settles the incomparable pairs by convention.

Profile	Structurally more cooperative than	Incomparable (ordered by convention)
`fully_coupled`	`chain`, `asymmetric`	`mutual`, `distributed`
`mutual`	`asymmetric`	`chain`, `distributed`, `fully_coupled`
`distributed`	`asymmetric`	`chain`, `mutual`, `fully_coupled`
`chain`	`asymmetric`	`mutual`, `distributed`
`asymmetric`	none (least cooperative)	none

Table 4: Structural partial order on the cooperation profiles (for $n_a \geq 3$): each profile's strictly more cooperative peers and its incomparable ones, the latter fixed only by the convention of Table 2.

5.6.5 Selective-strict semantics

Some profile decisions require asking whether a single agent is **individually** indispensable as a helper. To support this, the SAT encoding offers a **selective-strict** laser mode, parameterised by a set of colours $K \subseteq C_{\text{src}}$ (chosen distinct from the source-set symbol $\mathcal{S}$ to avoid confusion):

- for every source $(c, d, p_s) \in \mathcal{S}$ with $c \in K$, the beam-propagation clauses use the strict equivalence of Chapter 4;
- for every source $(c, d, p_s) \in \mathcal{S}$ with $c \notin K$, the standard equivalence is used.

When $K = \emptyset$, the encoding coincides with the standard semantics; when $K = C_{\text{src}}$, it coincides with the strict semantics of Definition 3.6. Intermediate choices forbid same-colour blocking for the colours in K while leaving the remaining beams untouched. Selective-strict is the SAT lever that lets us single out one helper at a time without affecting the other beams.

The same argument used in the proof of Theorem 5.1 generalises to this intermediate encoding: the selective-strict clauses for the colours in K are exactly the strict clauses of Chapter 4 restricted to those colours, while every other colour keeps its standard clauses, so the model-transfer construction of the theorem applies colour by colour. Concretely, Φ with the selective-strict encoding parameterised by K is satisfiable if and only if L admits a successful standard trajectory in which no agent of colour $c \in K$ ever stands on the unblocked beam path of the source of colour c. We therefore use the selective-strict encoding as a sound decision oracle for "L remains solvable when each colour in K is individually barred from blocking its own beam", which is exactly what the necessary-helper analysis below requires.

5.6.6 Necessary helpers

The **necessary-helper set** is, by contrast, a property of the level itself. For every colour $c \in C_{\text{src}}$ the analyser runs one selective-strict SAT call with $K = \{c\}$. If the call returns UNSAT, the colour c is added to the set. Operationally, c is necessary when the level cannot be solved as soon as c alone is barred from helping with its own beam, even though every other agent retains that ability.

The necessary-helper set does not depend on which standard model the solver returns: each counterfactual is its own one-shot satisfiability check, independent of the joint plan chosen elsewhere.

Scope note. The cooperation notion defined here is intentionally specific: it captures same-colour beam-blocking as the relevant cooperative act. A level that requires two agents to coordinate their movements for unrelated geometric reasons (without any laser blocking being involved) would not be identified as cooperative by this detector. This definition is not claimed to exhaust every possible

interpretation of cooperation in multi-agent environments; it is the specific mechanism studied in this benchmark, and the formal guarantee is scoped accordingly.

Model-dependence of finer-grained analyses. It is worth restating which outputs are intrinsic to the level and which are not. The binary detector is: the satisfiability of $\Phi(L, T_{\max})$ and $\Phi_{\text{strict}}(L, T_{\max})$ does not depend on which satisfying assignment the solver returns, and the necessary-helper set is likewise an invariant, being a sequence of one-shot counterfactual checks. The profile label is not: it is read from a single model the solver picks arbitrarily, so two runs on the same level can in principle return different labels. As stated at the start of the section, this is acceptable because the label serves as a sound filter on the extracted plan rather than as a claim about the level.

Chapter 6

Procedural generators

6.1 Design principles

All generators follow a common architecture built around three principles:

1. **SAT as oracle**: the solver is not post-hoc; it is embedded in the generation loop. A candidate level is accepted only if the solver confirms the desired property (solvability, cooperation, or a specific cooperation profile).
2. **Separation of concerns**: level construction and property verification are decoupled. Generators build candidate levels using domain-specific heuristics; the solver decides acceptance.
3. **Extensibility**: every generator shares a common base class and registers itself in a name-keyed registry, so a new generator can be added without modifying core code.

Each generator repeatedly performs the same loop: sample or construct a candidate layout, reject it if it violates generator-specific structural constraints, build the corresponding LLE world instance, and run the appropriate SAT-based acceptance test. A single standard SAT call certifies solvability. When the user additionally requires cooperation, the generator also runs the strict-beam counterfactual test from Chapter 5; when a specific cooperation profile is requested, the profile analyser of Section 5.6 acts as a soft filter on top. The cooperation requirement and the profile filter are parameters rather than separate generator classes: **what** the generator constructs and **which extra properties** are checked are independent axes.

6.2 Generation targets

Viewed through the solvability and cooperation definitions of Section 3.3, the generator family targets the three level categories shown in Figure 10. Every accepted level is solvable, i.e. categories (b) or (c); unsolvable levels in category (a) are always rejected. When the user additionally requires cooperation, levels in (b) are rejected as well, so only category (c) survives.

(a) Unsolvable **(b)** Solvable, no cooperation **(c)** Solvable and cooperative

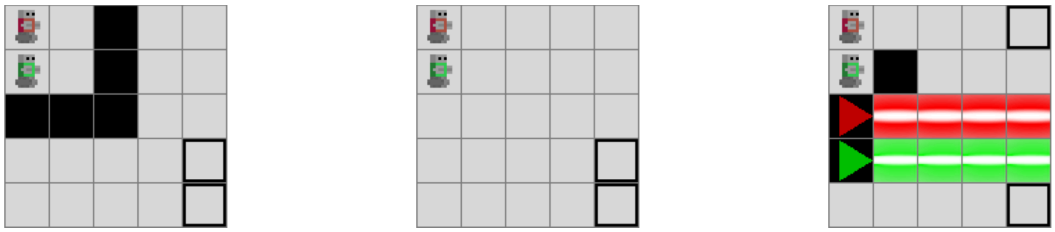

Figure 10: Target level categories for the generator family.

6.3 Cooperation profile targeting

Every generator in this chapter certifies **solvability** for every accepted level. That is the role of the SAT oracle, and it is always on. On top of that always-on guarantee, the user can stack two additional acceptance requirements drawn from Section 5.6:

- **Cooperation** (optional). The candidate must additionally satisfy the strict-UNSAT counterfactual of Theorem 5.1, so every accepted level provably requires the same-colour beam-blocking mechanism. Without this flag the cooperation profile is not inspected.
- **Profile filter** (optional, requires cooperation). The candidate must additionally match one of the requested cooperation-profile labels (e.g. `mutual`, `chain`, `distributed`). The accepted set is the corresponding sub-class of category (c) above.

Solvability is therefore the floor; cooperation and profile filtering are layered on top. All other generator parameters (grid size, number of agents n_a, number of lasers n_l, wall budget, horizon $T_{\max}$) are independent knobs and can be combined freely with any choice on the cooperation axis.

6.4 Random generator

The random generator samples every layout component uniformly: agent start positions, exits, wall positions, and laser source positions are drawn pairwise-distinct from the grid, and each laser source is given a random direction. The resulting candidate is submitted to the SAT oracle under whichever acceptance setting (solvability only, plus cooperation, or plus cooperation with profile filter) was requested. When a lower-bound horizon $T_{\min}$ is provided (a user-supplied difficulty knob, separate from the encoding horizon $T_{\max}$), the generator also requires the candidate to be unsatisfiable for $T_{\min} - 1$, selecting levels that fall inside a difficulty window.

This generator is deliberately simple. Its main value is methodological: it gives an unbiased sampling baseline against which more structured generators can be compared. Its main weakness is rejection rate. As the grid grows and the number of interacting entities increases, purely random layouts quickly become dominated by unsolvable or trivial instances.

6.5 Constrained random generator

A structured variant that biases generation toward well-formed, solvable configurations before any SAT call is made. Relative to the random generator, it adds a purely geometric pre-filter that rejects already-degenerate candidates. Table 5 lists the three rejected configurations in full.

Rejected configuration	Geometric condition	Why it is degenerate
Laser firing off the grid	the cell immediately in front of the source is out of bounds (e.g. a source on the bottom edge facing south)	the beam covers no tile, so the source behaves like an inert wall
Zero-length beam	the cell immediately in front of the source is a wall or another laser source	the beam is blocked at its first step and emits no active beam
Exit on a beam tile	an exit cell coincides with a tile covered by some laser beam	the exit is reachable only by standing on a live beam, so it is unusable unless the beam is blocked

Table 5: Geometric configurations rejected by the constrained random generator before any SAT call. The beam-tile set in the last two rows already accounts for beams stopping at walls and at other laser sources. These filters do not establish solvability; they only discard layouts whose lasers are inert or whose exits are beam-stranded.

These degeneracies are not merely cosmetic. A laser whose beam has zero length, or that points off the grid, emits **no active beam**: its source then behaves like an inert wall tile, and the level presents none of the beam-crossing structure the laser was meant to introduce. Crucially, the base random generator still **accepts** such a level whenever it is solvable (a level reduced to plain navigation usually is), so its solvable output is dominated by these laser-free instances. The geometric filters do not themselves prove solvability or cooperation, but by discarding inert-laser and beam-stranded layouts they ensure that accepted levels actually contain the laser interactions the experiments depend on. In the cooperative setting, where an active beam is a **prerequisite** for the same-colour blocking mechanism, they also avoid many SAT calls that could never have yielded a cooperative level. The generator otherwise offers the same formal guarantees as the random generator (solvable, optionally cooperative with profile filter), and is the default we use when the experiments call for "random sampling" of solvable or cooperative levels. The effect is visible by comparing the unfiltered pools of Section A.14 with the filtered pools of Section A.15 at the same grid, agent, laser, and wall settings.

6.6 Constructive generator

The constructive generator replaces blind sampling with a partial-by-construction layout. On a grid of H rows and W columns with n_a agents, it picks a random orientation, samples a set of n_a distinct **lane indices** without replacement on the orientation axis (rows for the horizontal orientation, columns for the vertical one), places one agent start at one end of each lane and the corresponding exit at the other end, and reserves every cell of every lane as non-buildable. Lane indices are sampled **without** a contiguity constraint, so the lane band can be split anywhere on the orientation axis, which is the main source of within-pool diversity.

When the generator is run in cooperative mode, the laser-placement step plants a deliberate cooperation dependency for every requested laser, rather than relying on the solver to **discover** one after the fact. For each of the n_l lasers, the generator picks

- a **perpendicular column** (or row, in the vertical orientation) from a pool of distinct values in the interior of the perpendicular axis;
- an axis position strictly before or after the lane band, so the beam crosses the lane geometry;
- a direction perpendicular to the lane axis, pointing toward the lane band.

Each laser is given a **distinct colour** in $\{0, \ldots, n_l - 1\}$. We require $n_l \leq n_a$ so that each laser colour corresponds to a unique agent of that colour, in line with the at-most-one-source-per-colour assumption of Definition 3.1. Distinctness of the perpendicular columns guarantees that the resulting beams are parallel straight lines on disjoint columns, so no two laser sources or beam segments coincide.

The full unblocked beam segment of every laser, from source to grid edge, is added to the reserved cell set before walls are placed; without this step a wall could land between two non-adjacent lanes and silently break the cooperation requirement. The remaining free cells are then shuffled uniformly and the first n_w of them are placed as walls, where n_w is the requested wall budget; shuffling rather than taking a row-major prefix is what gives within-pool wall-mask diversity. The finished candidate is verified by the SAT oracle exactly as in the random and constrained random generators.

The distinct-colour multi-laser construction is the key qualitative change relative to a single-structural-laser template. With $n_l = 1$ the geometry yields a single helper-beneficiary pair and the cooperation profile is **asymmetric** by construction. With $n_l = 2$ the two lasers cross each other's lanes and the profile is **mutual**: every helper is also a beneficiary. With $n_l \geq 3$ on grids large enough to accommodate the construction the profile mix shifts toward **distributed** as one agent benefits from multiple distinct helpers. Example pools in solvable and cooperative settings are shown in Section A.16 and Section A.17; pools generated under an explicit profile filter (mutual, distributed) are shown in Section A.18.

6.7 Level-6-style generator

The level-6-style generator is a specialised constructive variant targeting the layout shape of LLE Level 6 (see Section 3.1): clustered agent starts and clustered exits placed on opposing sides of the grid, with a corridor of lasers between them. Agents are placed in a small rectangular cluster (a 2×2 block for four agents) in one third of the grid; exits are placed in a matching cluster in the opposite third. The orientation (vertical with start above exit, or horizontal with start left of exit) is chosen at random per call, and the perpendicular position of each cluster is also randomised. Lasers are then placed inside the corridor between the clusters, oriented perpendicular to the cluster axis so that each beam crosses the corridor; remaining free cells receive walls up to the requested wall budget.

This geometry consistently produces **mutual** cooperation profiles that mirror the structure of Level 6, and it is the generator we recommend for producing training instances intended to transfer to Level 6 (see the curriculum-transfer experiment in Chapter 7). The profile filter from Section 6.3 still applies: the default target is `mutual`, but other profile labels can be requested when the parameter combination supports them. Example pools at three grid sizes (up to the 12×13 Level 6 footprint) are shown in Section A.19.

6.8 Summary

Generator	Pre-SAT construction bias	Profile tendency
Random	Uniform random sampling of every layout component	Mostly `asymmetric`
Constrained random	Uniform sampling + geometric rejection (Table 5)	Mostly `asymmetric`
Constructive	Reserved agent lanes + a deliberately planted laser dependency	`asymmetric` (1 laser), `mutual` (2 lasers), `distributed` (≥ 3 lasers)
Level-6-style	Clustered starts and exits with a corridor of lasers between them	`mutual` by design

Table 6: The four generator families. Every family certifies **solvability** through the SAT oracle for each accepted level; the table summarises the pre-SAT construction bias and the cooperation profile each family tends to produce. The measured per-generator profile distributions are reported in Section 7.3.2.

Chapter 7

Empirical evaluation

The empirical evaluation proceeds in six parts: a solver-engineering baseline, followed by five parts that answer research questions RQ3–RQ6 (RQ1 and RQ2 are settled by the formal results of Chapter 4 and Chapter 5). Each part is developed in its own section.

- **SAT encoding** (baseline, Section 7.2.1): the local versus global agent-uniqueness formulation, compared on CNF size and solve time; a solver-engineering baseline that justifies the encoding used throughout, not one of the research questions.
- **Generator efficiency** (RQ3, Section 7.3.1): per-generator rejection rates under the rejection-sampling strategy.
- **Output diversity** (RQ4, Section 7.3.2): the cooperation-profile distribution of accepted levels.
- **Learnability** (RQ5, Section 7.4.1): whether off-the-shelf value-based MARL agents learn generated cooperative levels on a small grid.
- **Data scaling** (RQ5, Section 7.4.2): whether enlarging the generated training pool restores generalisation.
- **Curriculum learning** (RQ6, Section 7.5.1, Section 7.5.2): whether staged exposure helps, on a reachable target and up to the mutually-cooperative LLE Level 6.

Software versions and seed conventions for every experiment in this chapter are summarised in Section A.2.

7.1 Evaluation protocol

The first benchmark implemented in this project isolates the SAT solver rather than the generators. Its goal is to compare the two movement formulations used in the encoding: the **local** formulation, which combines neighbourhood-based exclusivity with backward consistency, and the **global** formulation, which enforces uniqueness by pairwise exclusion over the whole grid.

7.1.1 Metrics

For each run, the benchmarking code records:

- the total number of clauses in the generated CNF;
- the clause count contributed by each major constraint family;
- the CNF generation time;

- the SAT solving time;
- the total time obtained by summing generation and solving.

The implementation also stores per-constraint method profiles for the movement constraint, which allows us to inspect the part of the final CNF generated inside the movement-constraint module.

7.1.2 Solver and level sets

All benchmark runs use the same SAT backend as the main solver implementation, namely `Minisat22`, a descendant of the MiniSat solver of Eén and Sörensson [35], accessed through the PySAT interface [36]. The benchmarking script can evaluate the six hand-crafted benchmark levels distributed with the environment, each paired with a horizon known to be sufficient for solvability. It can also benchmark custom levels constructed programmatically.

The SAT encoding comparison reported in the following sections uses four representative levels: three synthetic instances of increasing size and one original benchmark level. This combination exposes both scaling behaviour and the behaviour of the solver on a realistic cooperative puzzle.

7.1.3 Protocol

For each level and each movement formulation, the benchmark performs one profiled run to extract the exact clause counts and the full constraint breakdown. It then repeats the same solver invocation for 100 runs, reporting the mean and standard deviation of generation and solve time. Each run starts from a freshly built world in its initial state, so the 100 repetitions are independent and none is affected by state a previous run may have left in the mutable `lle.World` object.

No timeout or parallel speedup is introduced in this protocol. The measurements should therefore be read as direct comparisons between the two SAT formulations on the same machine, rather than as hardware-independent absolute performance claims. The clause counts and timing statistics recorded here are the basis for the results reported in the following sections.

7.2 Solver experiments

This part evaluates the SAT solver that every generator relies on. The operating protocol is the one defined in Section 7.1; here we report only the benchmark levels, the clause counts and solving times of the global and local movement formulations of Chapter 4, and what they imply for the default encoding.

7.2.1 Experimental question

The experiment asks whether the **global** and **local** movement formulations introduced in Chapter 4 differ materially in CNF size and runtime on levels of increasing complexity.

This comparison is not one of the research questions of Chapter 1: it is a solver-engineering baseline for the solvability verifier behind RQ1, reported first because every later experiment relies on the chosen encoding. It matters because the uniqueness constraint appears at every time step for every agent, so a poor formulation affects the full reduction, not only a negligible subpart of it. The protocol itself is

defined in Section 7.1; the present chapter reports only the level set, the observed results, and their interpretation.

7.2.2 Benchmark instances

Four levels of increasing size and complexity were used:

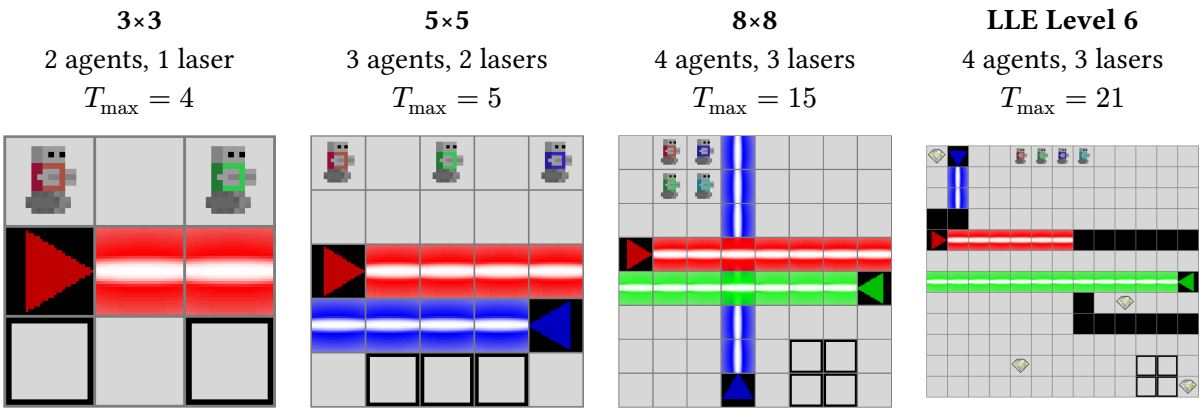

Figure 11: The four test levels used for benchmarking, ordered by increasing complexity. Each level is paired with the horizon $T_{\max}$ used to encode it in the SAT formulation; the horizon is set to the minimum number of joint steps known to be sufficient for solvability of the level.

The first three levels were constructed to expose scaling behaviour as grid size and agent count increase. The fourth is LLE Level 6, one of the hand-crafted benchmark levels distributed with LLE, included to assess solver behaviour on a realistic cooperative instance. Per-level parameters (grid, agent and laser counts, horizon) are tabulated in Section A.1.

7.2.3 Results

Clause count

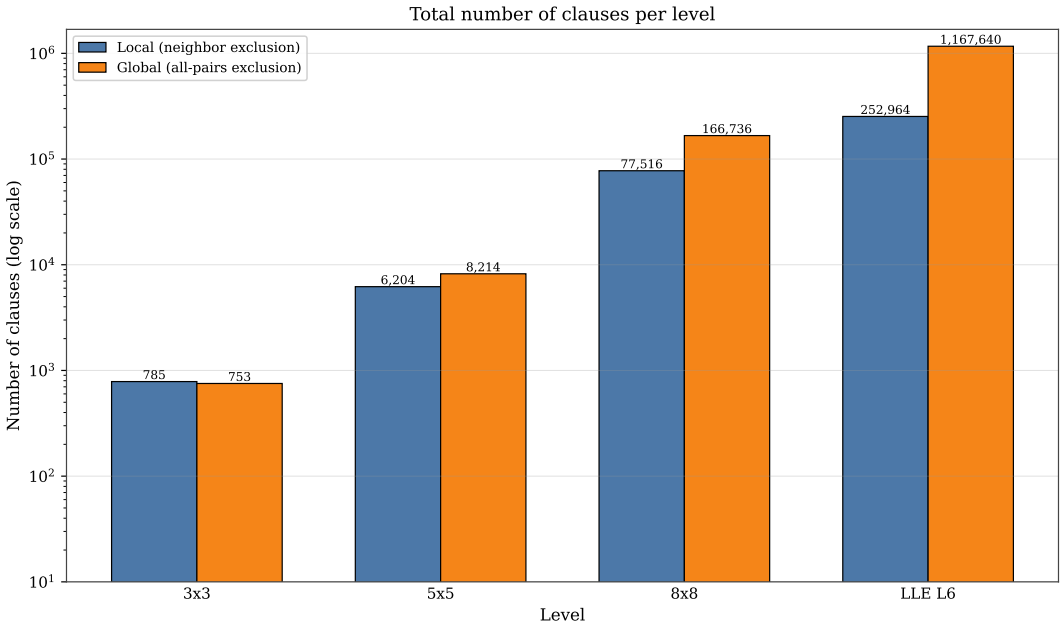

Figure 12: Number of clauses generated by the global and local uniqueness formulations across the four benchmark levels. The vertical axis is logarithmic because clause counts span more than three orders of magnitude.

The clause counts follow the theoretical expectation of Section 4.4: the global formulation's uniqueness clauses grow quadratically in the position count p, the local formulation's only linearly, so the global formulation is **asymptotically** larger. The local formulation pays a fixed overhead the global one avoids (its backward-consistency clauses and a per-cell neighbour-pair sum), which dominates on small grids. Below the **crossover** grid size, where the quadratic growth has yet to overtake this overhead, the global formulation is more compact: at 3×3 it produces 753 clauses versus 785. From 5×5 onward the trend reverses: the local formulation generates 6,204 clauses versus 8,214, then 77,516 versus 166,736 at 8×8, and 252,964 versus 1,167,640 on LLE Level 6.

This widening gap is the main result: the local formulation scales better because its exclusivity clauses are tied to neighbourhood-sized successor sets, whereas the global one introduces pairwise exclusions across the full position set. The per-constraint-family decomposition is reported in Section A.4.

Solving duration

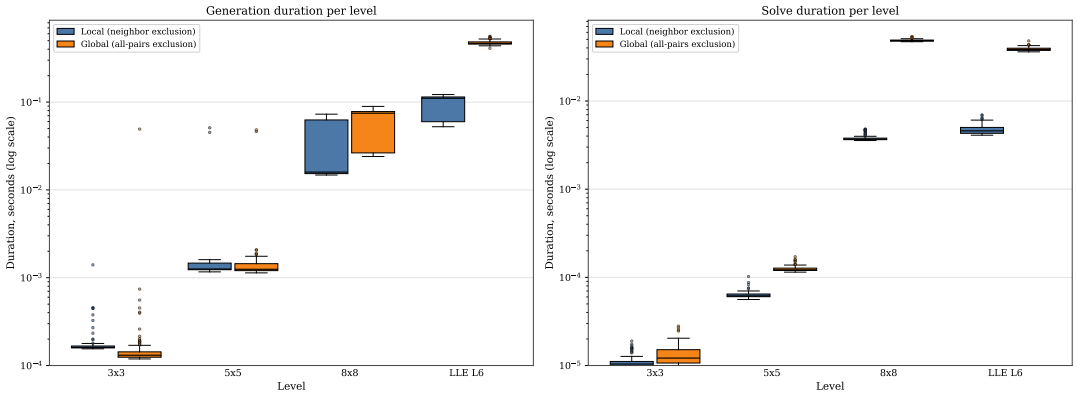

Figure 13: Per-run SAT generation duration (left) and solve duration (right), in seconds on a logarithmic scale, for the local and global uniqueness formulations across the four benchmark levels. Each box spans the interquartile range with the median marked; whiskers reach 1.5 times the interquartile range and individual outliers are plotted as points, over the 100 timing runs.

Timing follows the same pattern. On the smallest instances both methods are essentially instantaneous; once levels grow, the local formulation is consistently faster to generate and to solve, most visibly on 8×8 and LLE Level 6, where the larger global CNF carries a clear runtime penalty.

One subtlety: on LLE Level 6 the global formulation produces about 7 times more clauses than on the synthetic 8×8 instance, yet solves **faster**. Runtime is driven by search structure, not formula size: hand-crafted levels admit propagation-friendly assignments that CDCL solvers find quickly, while randomly structured instances can trigger pathological search at smaller clause counts.

The practical conclusion matches the analytical one: the local formulation is the preferable default for the solver. Raw per-level durations are reported in Section A.5.

7.2.4 Discussion

The experiment establishes one claim: the internal SAT formulation matters. The choice between local and global uniqueness affects both CNF size and runtime, and the effect is already substantial on realistic LLE instances, which fixes the local formulation as the solver's default.

It says nothing about the broader quality of the generator framework: it does not measure acceptance rates, the diversity of accepted levels, or any downstream training effect. The Generator and Learning parts that follow address these questions in turn.

7.3 Generator experiments

This part evaluates the generators of Chapter 6. Both experiments draw candidates from the same generator families and accept or reject each one with the SAT test of Chapter 4. They differ only in what they measure on the accepted stream: its production cost (Section 7.3.1) and its structural diversity (Section 7.3.2). Each protocol below gives the generator settings and grid configurations specific to that experiment, and a joint discussion closes the part.

7.3.1 Generator rejection rates

Acceptance rates determine the practical cost of the rejection-sampling strategy used by the generators. A generator with a 1 % acceptance rate requires approximately one hundred solver calls per usable level, which directly affects generation throughput. This experiment measures, for each generator and grid size, how many attempts are needed to find one accepted level and how much wall-clock time those attempts consume.

Protocol

For each generator and grid-size combination, a fixed number of independent trials is executed: 200 trials for the 3×3 and 5×5 grids, and 20 trials for the 8×8 grid. Each trial searches for one accepted level by repeatedly drawing a fresh single-attempt candidate until one is accepted. A trial ends when one level is accepted or a per-trial budget is exhausted (500 attempts for the small grids; 100 attempts or 30 seconds for the 8×8 grid). Failed trials are excluded from the mean attempt count and reported separately.

Each attempt samples a candidate layout, validates geometric constraints (where applicable), constructs the corresponding LLE world, and runs the SAT-based acceptance test of the generator. The mean number of attempts per accepted level is the direct measurement of the rejection cost: it is approximately the reciprocal of the acceptance rate.

Five generator settings are evaluated, drawn from the four families of Chapter 6: the Constrained Random generator in solvable and cooperative settings, the Constructive generator in solvable and cooperative settings, and the Level-6-Style generator. Three grid configurations are used: 3×3 with 2 agents and 1 laser, 5×5 with 3 agents and 2 lasers, and 8×8 with 4 agents and 3 lasers.

Results

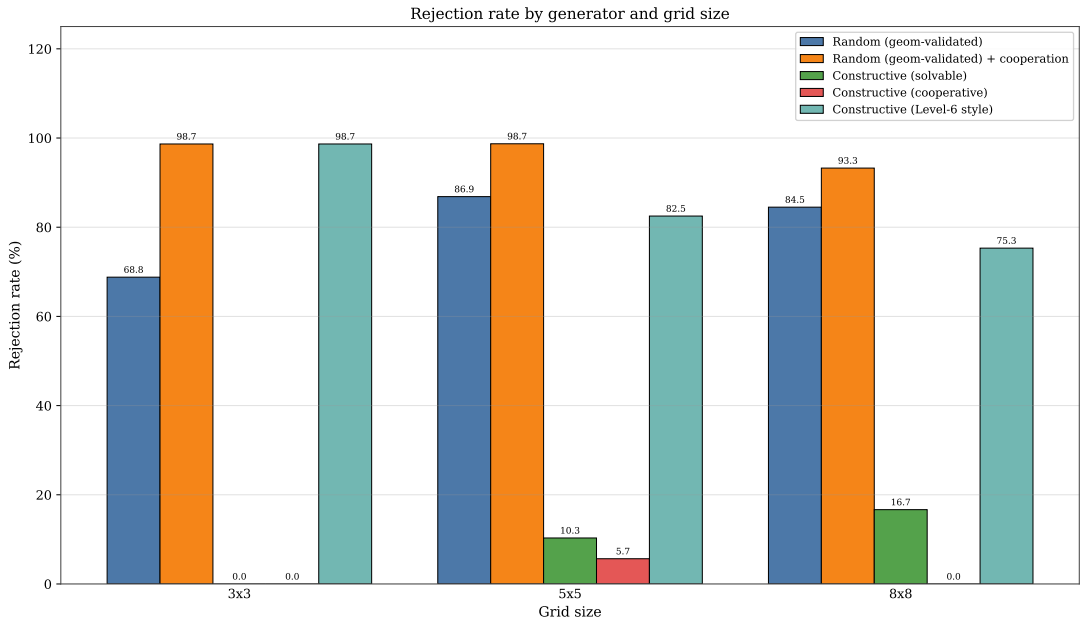

Figure 14: Rejection rate (%) per generator and grid size, derived from the mean number of attempts per accepted level.

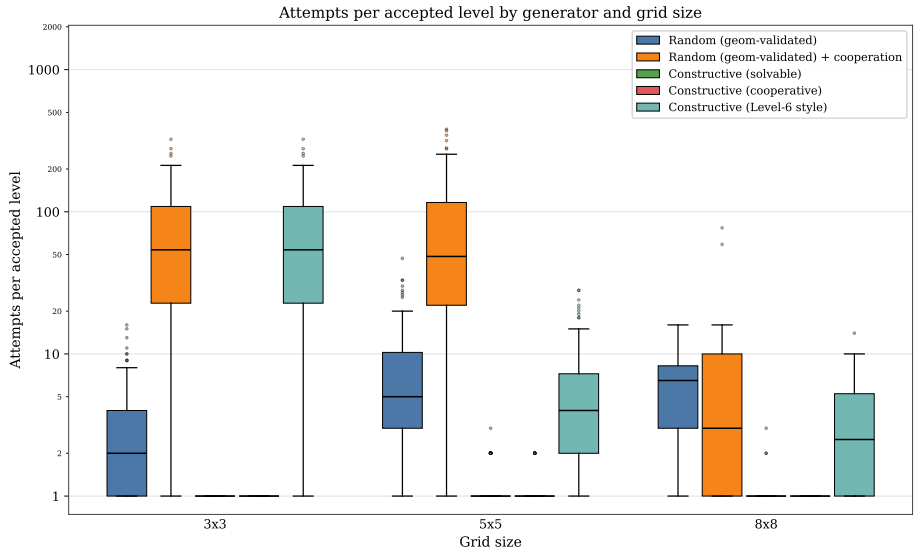

Figure 15: Distribution of the number of attempts needed to find one accepted level, per generator and grid size, on a logarithmic scale. Each box spans the interquartile range with the median marked; whiskers reach 1.5 times the interquartile range and outliers are plotted as points, over the successful trials.

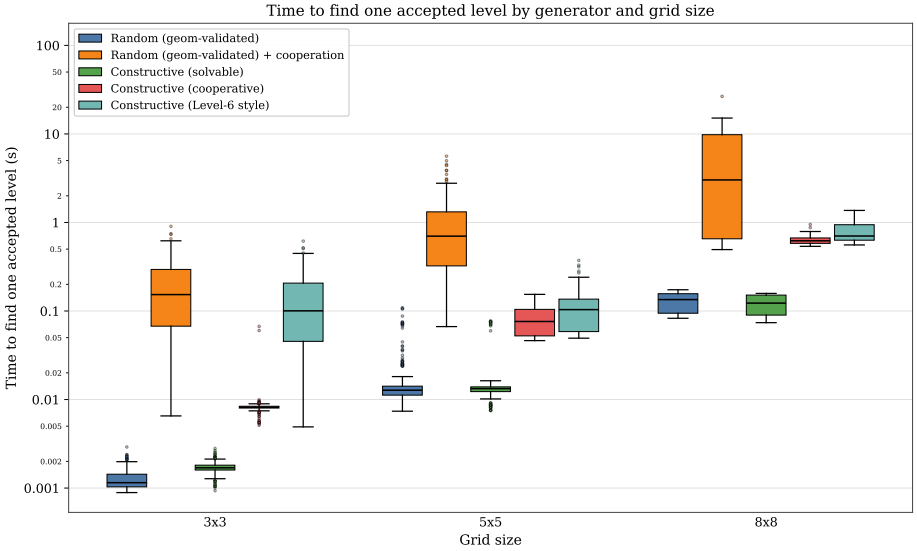

Figure 16: Distribution of the wall-clock time to find one accepted level, per generator and grid size, on a logarithmic scale; the time includes all rejected attempts that precede each accepted level. Each box spans the interquartile range with the median marked; whiskers reach 1.5 times the interquartile range and outliers are plotted as points, over the successful trials.

The results split the five generators into three regimes.

The Constrained Random generator in **solvable** mode has rejection rates of 68.8 %, 86.9 %, and 84.5 % across the three grid sizes, with mean attempts of 3.2, 7.6, and 6.5. Rejection here is driven by the fact that random layouts rarely admit a valid joint trajectory within the requested horizon.

The two Constructive variants are the cheapest in the family. The Constructive generator in **solvable** mode runs at 0.0 %, 10.3 %, and 16.7 % rejection across the three grid sizes (1.00, 1.11, and 1.20 mean attempts), and in **cooperative** mode at 0.0 %, 5.7 %, and 0.0 % (1.00, 1.06, and 1.00 mean attempts). Both settings stay cheap, at or near a single attempt per accepted level: the multi-colour structural-laser construction of Chapter 6 guarantees that every accepted candidate satisfies the binary cooperation criterion by construction, so the SAT acceptance step rejects only the small fraction of candidates whose random wall sample happens to produce an unsolvable or trivial layout. The residual differences between the two settings on the 5×5 and 8×8 grids fall within the sampling noise of the 20-trial large-grid budget.

The Constrained Random generator in **cooperative** mode has 98.7 % rejection at 3×3 (74.8 mean attempts), 98.7 % at 5×5 (77.5 mean attempts), and 93.3 % at 8×8 (14.8 mean attempts over the 12 of 20 trials that completed within budget). The high cost reflects the strict subset structure: cooperation-requiring layouts are a strict subset of solvable layouts, and a uniformly sampled candidate is unlikely to require beam-blocking.[2]

The Level-6-Style generator sits between the two regimes. On the 3×3 grid its rejection is 98.7 % (74.8 mean attempts), because the cluster template requires more grid cells than the small grid can offer without geometric conflicts. From 5×5 upward the cluster geometry becomes feasible: rejection drops to 82.5 % and 75.3 % at 5×5 and 8×8 (5.7 and 4.1 mean attempts respectively). The Level-6-Style template trades a moderate efficiency cost for the richer cooperation profile reported in Section 7.3.2. Detailed per-(generator, grid) numbers are tabulated in Section A.6.

7.3.2 Cooperation profile distribution

The binary cooperation detector of Chapter 5 certifies that every accepted cooperative level requires at least one same-colour beam-blocking step, but it does not distinguish **which** structural pattern of inter-agent dependency the level exhibits. This experiment measures the distribution of cooperation profiles among accepted levels, using the cooperation profile analyser also introduced in Chapter 5.

Protocol

For each of the three cooperative generators (Constrained Random, Constructive, and Level-6-Style, all in cooperative mode) and two grid configurations (5×5 with 2 agents and 1 laser, 8×8 with 3 agents and 2 lasers), the script accepts 100 cooperative levels on the small grid and 50 on the large grid, then classifies each into one of the analyser's profile families: `asymmetric`, `mutual`, `chain`, `distributed`, or `fully_coupled`.

Results

Figure 17 reports the profile breakdown for each generator and grid configuration. The two regimes behave very differently, and we discuss each in turn.

[2] The 8×8 configuration of the Constrained Random generator in cooperative mode is the most expensive setting measured: 8 of its 20 trials exhausted the 30-second per-trial budget without finding an accepted level and are excluded from the reported mean, so that figure is an optimistic estimate over the trials that did complete. Obtaining this row reliably also requires running each generation attempt in an isolated worker subprocess, because attempts on this configuration intermittently crash the underlying SAT solver's native extension; the isolation discards only the offending candidate and resamples, rather than aborting the whole run.

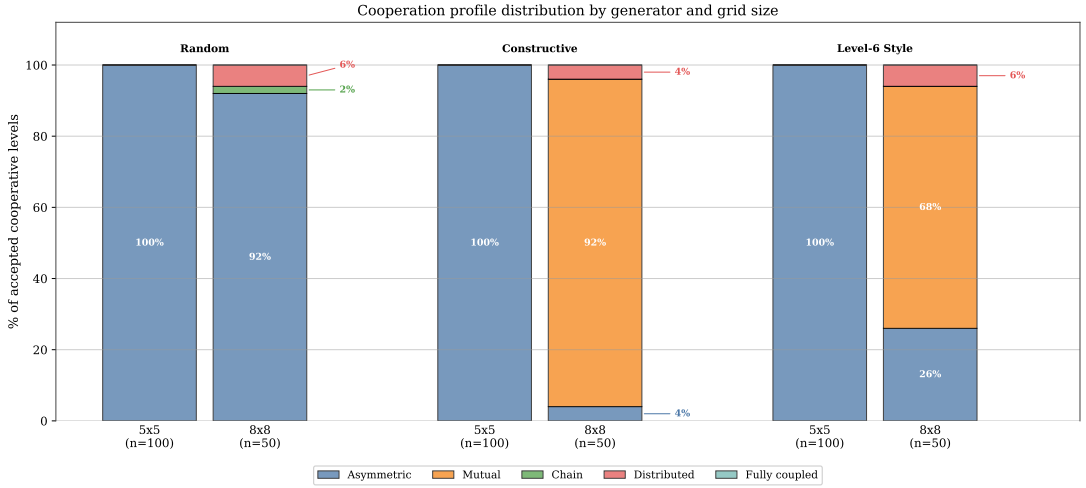

Figure 17: Distribution of cooperation profiles among accepted cooperative levels. Each bar is one (generator, grid) configuration; stacked segments sum to 100 %. `Fully coupled` is listed in the legend for completeness but was never observed (0 % everywhere).

On the 5×5 grid every accepted level from all three generators is classified as `asymmetric` (100 out of 100 in each case). With only 2 agents, the families `chain`, `distributed`, and `fully_coupled` are **structurally impossible** by construction (they require at least 3 agents), and the `mutual` family, though reachable in principle, does not appear: with a single laser the cooperation pattern is restricted to a one-way blocking action by the laser's colour-matched agent.

The 8×8 configuration with 3 agents and 2 lasers exposes the contrast between the three generators. The Constructive generator produces 46 `mutual`, 2 `asymmetric`, and 2 `distributed` levels out of 50, i.e. a 92 % majority of mutual cooperation. This reflects the multi-colour structural-laser construction: with two distinct-colour lasers each crossing the entire lane band, the red agent must block the red laser for the other agents to pass, and symmetrically the green agent must block the green laser, so two helpers act on their own beams in turn, the canonical mutual pattern.

The Constrained Random generator produces 46 `asymmetric`, 3 `distributed`, and 1 `chain` level out of 50 on the same 8×8 configuration. Random layouts that happen to require beam-blocking do so most often through a one-way dependency; richer profiles arise only incidentally and the generator does not reach the `mutual` family here.

The Level-6-Style generator produces 34 `mutual`, 13 `asymmetric`, and 3 `distributed` levels out of 50 on the same configuration. The cluster-and-corridor geometry forces all agents through a shared corridor and tends to produce multiple agents helping each other simultaneously, much like the canonical LLE Level 6 itself (which the analyser also classifies as `mutual`). The two Constructive families therefore split their cooperative output as follows: the Constructive generator favours the mutual pattern by design (92 % mutual), and the Level-6-Style generator favours it by geometric pressure (68 % mutual).

Across the three generators, no level was classified as `fully_coupled` in this benchmark. That family requires a strongly connected dependency graph of size at least three, which neither the lane template nor the cluster template targets by construction; reliably generating fully-coupled instances would require either a larger laser count or the profile-targeted generation mode, in which the generator's acceptance filter explicitly rejects non-matching profiles. The raw per-(generator, grid) profile counts are tabulated in Section A.7.

7.3.3 Discussion

The two benchmarks above characterise complementary aspects of the generation framework.

The rejection-rate experiment measures **generation efficiency**: how many candidate layouts a generator must propose before one is accepted. The headline finding is that the Constructive generator in cooperative mode runs at near-zero rejection (0–6 % across the three grid sizes, comparable to the solvable setting), thanks to the multi-colour structural-laser construction that guarantees cooperation by construction rather than by rejection sampling. This near-zero rejection rate makes the Constructive generator practical for producing large pools at scale.

The profile-distribution experiment measures **output diversity**. The headline finding is that the same Constructive cooperative generator produces predominantly `mutual` levels (92 % mutual on 8×8 with two lasers). Together with the Level-6-Style generator (68 % mutual on the same grid), the family spans the asymmetric / mutual / distributed range of the analyser's classification.

What these experiments do not establish is whether certified levels are useful for training. The generator framework guarantees formal properties of accepted levels, but whether those properties translate into better or faster learning for MARL agents compared to uncertified baselines remains an open empirical question, addressed in Section 7.4.1 and Section 7.5.2.

7.4 Learning generated levels with MARL

This part asks whether the certified levels are useful for training MARL agents. The two experiments share one operating protocol, stated in full in Section 7.4.1 and reused by Section 7.4.2: the same 5×5 / 2-agent / 1-laser cooperative task and constructive cooperative generator, the same three value-based algorithms (IQL, VDN, QMIX) trained over 20 seeds, the same hyperparameters, and the success-rate metric of Section 3.1. Each section reports only the parameters it changes, the training-pool size and the step budget. A joint discussion closes the part.

7.4.1 Learnability of generated cooperative levels

We now ask whether off-the-shelf MARL agents can learn to solve levels produced by the Constructive generator in cooperative mode, and whether the choice of MARL algorithm matters as the cooperation requirement scales up in grid size and agent count. The cooperative generator certifies every accepted level under the binary criterion of Chapter 5: standard SAT and strict UNSAT, so every training and evaluation level requires at least one same-colour beam-blocking step. The learnability question is therefore not whether the levels are solvable in principle (they are, by construction), but whether the joint policy that solves them is reachable by common value-based methods within a modest training budget.

We restrict ourselves to three baseline cooperative-MARL algorithms: independent Q-learning (IQL), in which each agent runs a separate deep Q-network [37]; value-decomposition networks (VDN) [4]; and QMIX [5]. They form a natural progression in the strength of the credit-assignment assumption: from none (IQL), to additive team-value (VDN), to a monotonic mixing network (QMIX).

Protocol

The experiment fixes a single small-grid configuration that exercises the cooperation pressure end-to-end while remaining within reach of a 200,000-step budget: a 5×5 grid with two agents and one laser, horizon $T_{\max} = 10$, and the Constructive cooperative generator. The configuration is summarised in Table 7.

Grid	Agents	Lasers	$T_{\max}$	Generator	Steps
5×5	2	1	10	Constructive (cooperative)	200,000

Table 7: Learnability-experiment configuration.

The pre-flight script generates two disjoint level pools: a training pool of $|\mathcal{D}_{\text{train}}| = 20$ levels and a held-out test pool of $|\mathcal{D}_{\text{test}}| = 20$ levels. The same pools are reused for all algorithms and all training seeds, so any cross-algorithm difference is attributable to optimisation rather than to a pool resample. Per-level renderings of both pools are reproduced in the appendix (Section A.9, Section A.10).

We train $|\mathcal{A}| \times |\mathcal{S}| = 3 \times 20 = 60$ independent agent instances:

$$\mathcal{A} = \{\text{IQL}, \text{VDN}, \text{QMIX}\}, \quad \mathcal{S} = \{0, 1, \ldots, 19\}$$

The seed $s \in \mathcal{S}$ controls the Python, NumPy, and PyTorch random streams as well as the pool-sampling RNG. Hyperparameters are identical across algorithms (Section A.3): an Adam optimiser at learning rate 5×10^{-4}, batch size 64, discount factor $\gamma = 0.95$, a gradient update every 5 environment steps, gradient-norm clipping at 10, an ε-greedy training policy decaying linearly from 1.0 to 0.05 over the first 100,000 environment steps, and a fully greedy evaluation policy.

Within each training run, every episode samples one level $L \in \mathcal{D}_{\text{train}}$ uniformly with replacement; the episode horizon equals the generator's $T_{\max}$. Every 10,000 environment steps we run a greedy evaluation of 50 episodes on $\mathcal{D}_{\text{train}}$ and 50 episodes on $\mathcal{D}_{\text{test}}$, sampling levels uniformly from each pool. After the final training step we run a longer evaluation of 200 episodes per pool. We use the success-rate metric defined in Section 3.1: the success rate $\hat{s}$ reported below is the fraction of these 200 final-evaluation episodes in which the whole team reaches its exits, computed separately on the train and test pools, with one estimate per (algorithm, seed) cell.

The headline aggregate per algorithm $a \in \mathcal{A}$ is the mean and standard deviation over the 20 seeds. We also report a 95 % confidence interval $\pm t^* \frac{\sigma}{\sqrt{20}}$ in the per-seed appendix tables, where σ is the across-seed standard deviation and $t^* \approx 2.093$ is the 0.975 quantile of Student's t distribution with $20 - 1 = 19$ degrees of freedom.

Results

Figure 18 reports the train- and test-pool success rates as a function of environment steps, averaged over the 20 seeds with a 95 % confidence band. All three algorithms reach a non-trivial success rate on the training pool well before the 200,000-step budget is exhausted, confirming that the 5×5 cooperative task is learnable in principle. The held-out test pool is markedly harder for every algorithm: the test curves plateau below the corresponding training curves, with a train–test gap of 0.37 for IQL, 0.39 for QMIX, and 0.52 for VDN at the end of training (Table 8, and the per-seed numbers in Table 19).

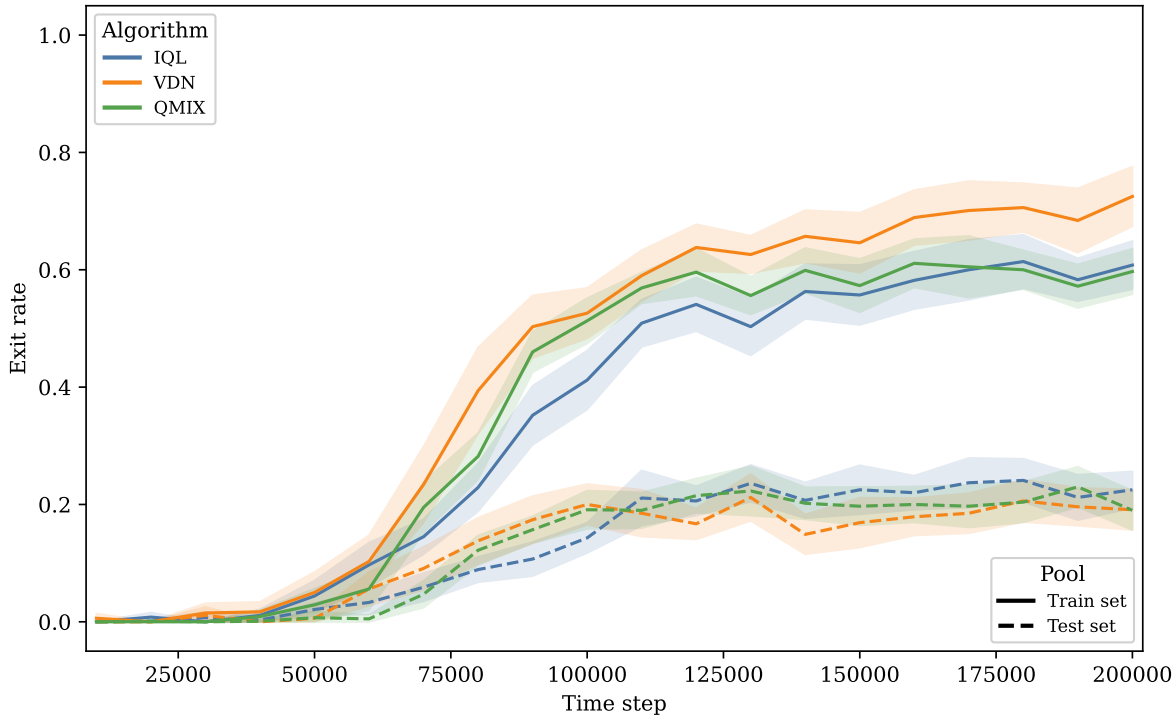

Figure 18: Mean success rate (greedy exit rate) on the training and held-out test pools as a function of environment steps. Colour encodes the algorithm (IQL, VDN, QMIX) and line style the pool (solid: train; dashed: test). Shaded bands are 95 % confidence intervals over the 20 training seeds, clipped to the per-step minimum and maximum.

Figure 19 and Table 8 summarise the final 200-episode evaluation. On the training pool VDN is strongest (0.70 ± 0.05), ahead of IQL (0.60 ± 0.03) and QMIX (0.59 ± 0.03); on the held-out pool the order reverses, with IQL at 0.23 ± 0.03, QMIX at 0.20 ± 0.03, and VDN at 0.18 ± 0.03. The three held-out means lie within each other's confidence intervals, so no algorithm generalises reliably better than another in this configuration.

The contrast between the two pools is the substantive result, and VDN illustrates it most sharply. It fits the training pool best (on individual seeds its training success reaches 0.96) yet ends with the lowest held-out success, so the extra capacity that additive value decomposition buys is spent memorising the 20 training levels rather than learning a transferable cooperation policy. The three algorithms span a deliberate range of credit-assignment strength (none for IQL, additive for VDN, monotonic mixing for QMIX) and nonetheless collapse to the same held-out band; the bottleneck here is therefore the number of distinct training levels, not the sophistication of the learning rule. This is what motivates the training-pool scaling experiment of Section 7.4.2, which holds the algorithms fixed and varies $|\mathcal{D}_{\text{train}}|$.

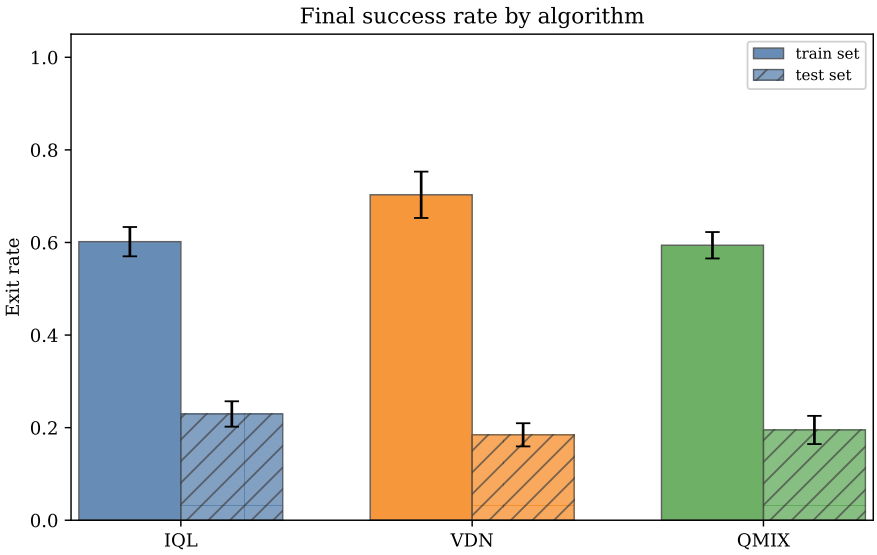

Figure 19: Final-evaluation success rate per algorithm (200 episodes per pool), train versus test. Error bars are 95 % confidence intervals over the 20 training seeds.

Algorithm	n	Train mean $\pm$ CI95	Test mean $\pm$ CI95	Train $-$ test gap
IQL	20	0.60 ± 0.03	0.23 ± 0.03	0.37
VDN	20	0.70 ± 0.05	0.18 ± 0.03	0.52
QMIX	20	0.59 ± 0.03	0.20 ± 0.03	0.39

Table 8: Per-algorithm final success rates on the 5×5 cooperative pools, aggregated over 20 training seeds. The 200-episode greedy evaluation is summarised as a mean and a 95 % confidence interval $\pm t^* \frac{\sigma}{\sqrt{20}}$ over the 20 seeds ($t^* \approx 2.093$, the 0.975 quantile of Student's t with 19 degrees of freedom).

7.4.2 Scaling the training pool closes the generalisation gap

The learnability experiment of Section 7.4.1 isolated **generalisation**, rather than credit assignment, as the dominant failure mode: all three algorithms reached a non-trivial training-pool success rate but overfit the twenty training levels, with a train–test gap of up to 0.52. Because the constructive cooperative generator produces certified levels at near-zero marginal cost (Section 7.3.1), the natural remedy is simply more **training data**. We therefore ask whether enlarging the training pool, holding the task, the budget regime, and the held-out test pool fixed, closes the generalisation gap.

Protocol

The task, generator, horizon, and hyperparameters are exactly those of Section 7.4.1 (a 5×5 grid, two agents, one laser, $T_{\max} = 10$, constructive cooperative generator). The only manipulated variable is the training-pool size $|\mathcal{D}_{\text{train}}| \in \{20, 100, 500\}$. The three training pools are **nested**: they are drawn as prefixes of a single seeded generation stream, so the 100-level pool contains the 20-level pool and the 500-level pool contains both. A single held-out test pool of $|\mathcal{D}_{\text{test}}| = 50$ levels, drawn from an independent seed, is **identical** across all three conditions, so any change in held-out success is attributable to the training-pool size alone.

For each pool size we train $|\mathcal{A}| \times |\mathcal{S}| = 3 \times 20 = 60$ independent agents ($\mathcal{A} = \{\text{IQL}, \text{VDN}, \text{QMIX}\}$, $\mathcal{S} = \{0, \ldots, 19\}$) for 300,000 environment steps, and report the final 200-episode greedy success rate on each pool. Aggregates are taken over all sixty runs per pool size, summarised as a mean and a 95% confidence interval.

Results

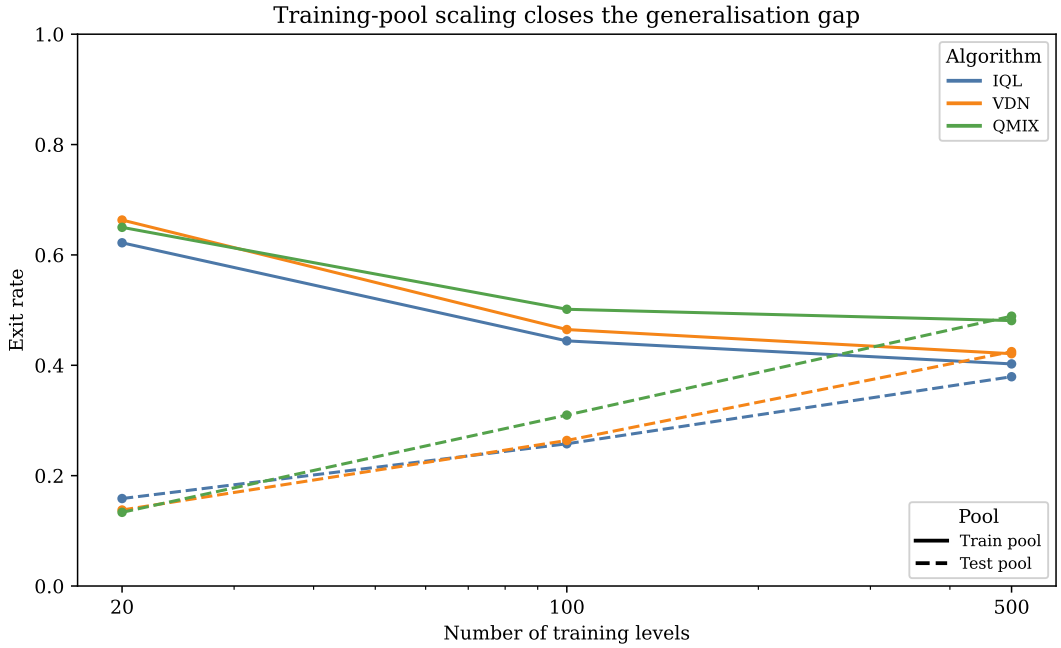

Figure 20: Final greedy success rate on the training and held-out test pools as a function of the number of training levels (log scale), for the 5×5 / 2-agent / 1-laser cooperative task. Colour encodes the algorithm (IQL, VDN, QMIX) and line style the pool (solid: train; dashed: test). As the pool grows, training success falls while held-out success rises, and the two meet.

Figure 20 and Table 9 show a clear monotone effect. Held-out test success rises from 0.14 at twenty levels to 0.28 at one hundred and 0.43 at five hundred, roughly a threefold improvement, while training success **falls** from 0.65 to 0.43 over the same range. The train–test gap therefore collapses from 0.50 to 0.19 to approximately zero: at five hundred levels the agent performs essentially identically on training and held-out levels, i.e. it has stopped memorising the training pool and learned a policy that transfers across the level distribution. The monotone increase in held-out success holds for each algorithm individually (IQL 0.16 → 0.26 → 0.38; VDN 0.14 → 0.26 → 0.42; QMIX 0.13 → 0.31 → 0.49), so the effect is a property of the data regime rather than of any single learning algorithm.

$\lvert\mathcal{D}_{\text{train}}\rvert$	n	Train mean ± CI95	Test mean ± CI95	Train — test gap
20	60	0.65 ± 0.02	0.14 ± 0.01	0.50
100	60	0.47 ± 0.02	0.28 ± 0.02	0.19
500	60	0.43 ± 0.02	0.43 ± 0.03	0.00

Table 9: Final success rates on the 5×5 cooperative task as the training-pool size grows, with a fixed 50-level held-out test pool. Each row is the mean over all three algorithms (IQL, VDN, QMIX) and 20 seeds ($n = 60$ runs).

7.4.3 Discussion

Together the two experiments isolate the practical payoff of the generator. The learnability ceiling reported in Section 7.4.1 is not an intrinsic limit of the value-based algorithms on cooperative levels; it is a consequence of training on too few distinct instances. Because the constructive cooperative generator emits an effectively unlimited supply of certified levels (solvable and cooperation-requiring, Chapter 5) at near-zero rejection cost, it directly supplies the quantity of training data needed to close the gap, something a fixed library of hand-crafted levels cannot. The generator's formal guarantees thus translate into a concrete downstream benefit: scaling generated training data converts overfitting into generalisation. This also delimits the role of the curriculum studied next: curricula target either the **ordering** of staged exposure on a target that is already reachable (Section 7.5.1) or levels too **hard** to learn directly (Section 7.5.2), whereas data scaling targets levels that are learnable but not yet **generalised**.

7.5 Curriculum learning

Data scaling closes the gap on targets that are already learnable. This part turns to curriculum learning, which stages training across a ladder of growing difficulty. We ask two questions: whether the ordering of stages matters on a target that direct training can already reach (Section 7.5.1), and whether a curriculum lets agents reach a mutual-cooperation target they cannot learn directly (Section 7.5.2). A joint discussion closes the part.

7.5.1 Curriculum ordering on a reachable 6×6 cooperative target

Section 7.4.2 showed that the learnability ceiling on **reachable** generated levels is a data-quantity effect rather than an intrinsic algorithmic limit. RQ6 (Section 7.5.2) asks the harder question of whether a staged curriculum transfers to the hand-crafted LLE Level 6, a target on which direct training is expected to fail outright. As a controlled precursor we first isolate the curriculum **mechanism itself** on a target that direct training can already reach. Holding the task, the difficulty ladder, the total training budget, and the held-out evaluation fixed, we vary only **how** the budget is allocated across a ladder of growing geometric and cooperative complexity, and ask whether the **ordering** of staged exposure changes final performance or sample efficiency on the target. Because the target is reachable by the direct baseline, any difference between conditions is attributable to scheduling alone rather than to the target being unreachable.

Protocol

The difficulty ladder has three rungs, each supplied by the generators of Chapter 6: a 4×4 / 2-agent / 0-laser navigation warm-up (random generator, $T_{\max} = 8$), a 5×5 / 2-agent / 1-laser cooperative rung ($T_{\max} = 10$), and the 6×6 / 2-agent / 1-laser cooperative **target** rung ($T_{\max} = 12$). The agent count is fixed at two across all rungs so the Q-network input shape is constant; smaller observations are zero-padded to the target geometry. Each rung draws 100 distinct certified levels for training, and the target rung additionally holds out an independent pool of 50 levels for the generalisation metric.

We compare four budget-matched scheduling conditions, each trained for a total of 400,000 environment steps:

- **direct** spends the entire budget on the 6×6 target rung;
- **forward** walks the ladder easy-to-hard, allocating $(50, 150, 200) \times 10^3$ steps to the three rungs in order;
- **reverse** uses the same per-rung budgets but visits the rungs hard-to-easy, so it reaches the target early and ends on the navigation rung, arriving at the held-out evaluation cold;
- **mixed** draws a rung uniformly at random each episode (domain randomisation) for the whole budget.

Each condition is run with all three algorithms ($\mathcal{A} = \{\text{IQL}, \text{VDN}, \text{QMIX}\}$) and eight seeds, for 96 runs in total. The exploration rate is reset at every stage boundary and decayed independently within each rung's budget, so that early rungs are not starved of exploitation nor the target of exploration. Evaluation is always performed on the 6×6 target: every 10,000 steps on 50 episodes during training, and a final 200-episode greedy success rate on both the training and held-out test pools at the end. Aggregates are reported as a mean and a 95% confidence interval over seeds.

Results

Table 10 summarises the final greedy success rate of each scheduling condition, pooled over the three algorithms and the eight seeds per cell ($n = 24$ runs per condition). On the held-out 6×6 target the ranking is **mixed** (0.22 ± 0.04), then **direct** (0.17 ± 0.03), then **forward** (0.15 ± 0.03), and last **reverse** (0.07 ± 0.02). The ordered easy-to-hard curriculum (**forward**) does not beat the **direct** baseline; the only condition that improves on direct is **mixed**, which exposes all three rungs in random order throughout training.

Condition	n	Train mean $\pm$ CI95	Test mean $\pm$ CI95	Train $-$ test gap
direct	24	0.41 ± 0.05	0.17 ± 0.03	0.24
forward	24	0.39 ± 0.06	0.15 ± 0.03	0.24
mixed	24	0.39 ± 0.07	0.22 ± 0.04	0.17
reverse	24	0.10 ± 0.02	0.07 ± 0.02	0.03

Table 10: Final greedy success rate on the 6×6 / 2-agent / 1-laser target per scheduling condition, aggregated over three algorithms and eight seeds ($n = 24$ runs per condition). The 200-episode greedy evaluation is summarised as a mean and a 95% confidence interval ($\pm t_{23,0.025} \frac{\sigma}{\sqrt{24}}$, $t_{23,0.025} \approx 2.07$). Per-(condition, algorithm) numbers are in Section A.12.

The learning curves and the per-algorithm final success rates are shown in Figure 21, Figure 22, and Figure 23; they confirm that the ordering above holds within each algorithm, not only in the pooled aggregate.

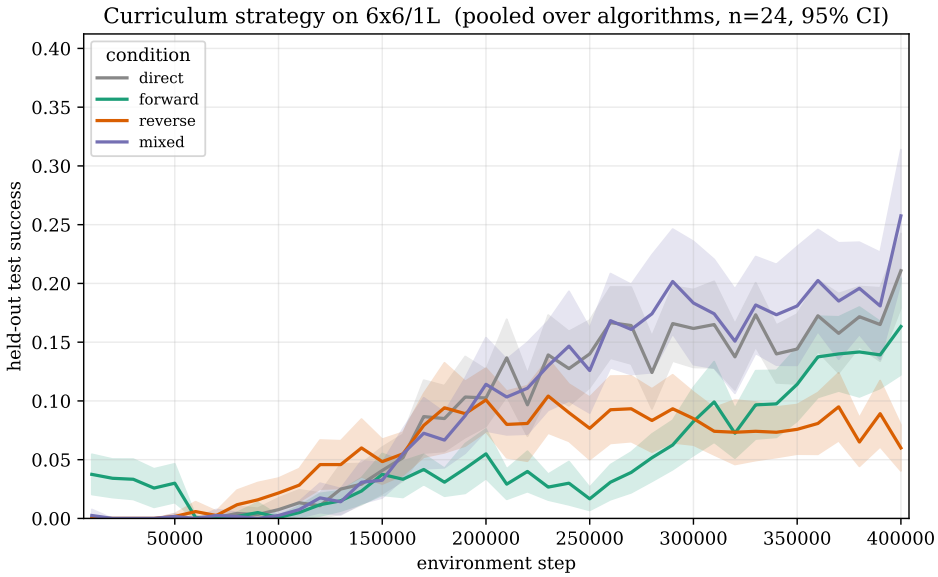

Figure 21: Held-out test success on the 6×6 / 2-agent / 1-laser target as a function of environment step, one line per scheduling condition (direct, forward, reverse, mixed) pooled over the three algorithms and the eight seeds. Shaded bands are 95% confidence intervals.

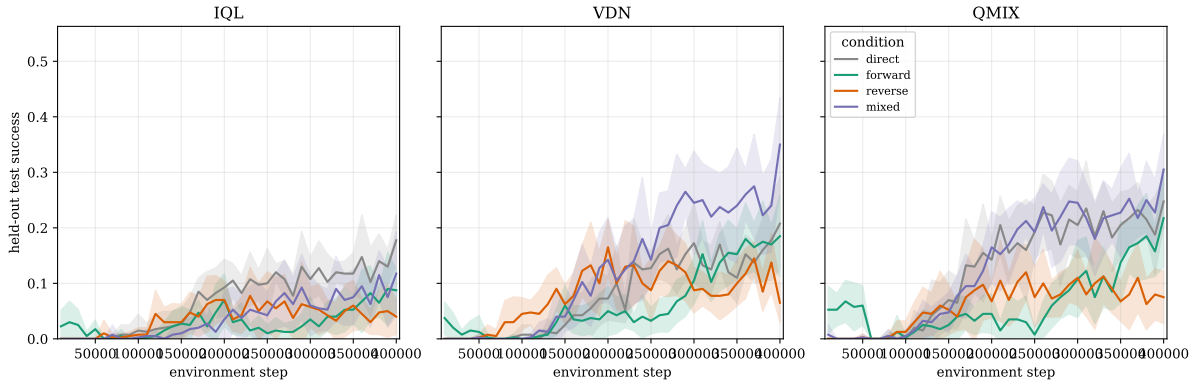

Figure 22: The same held-out test learning curves as Figure 21, shown per algorithm (IQL, VDN, QMIX) rather than pooled.

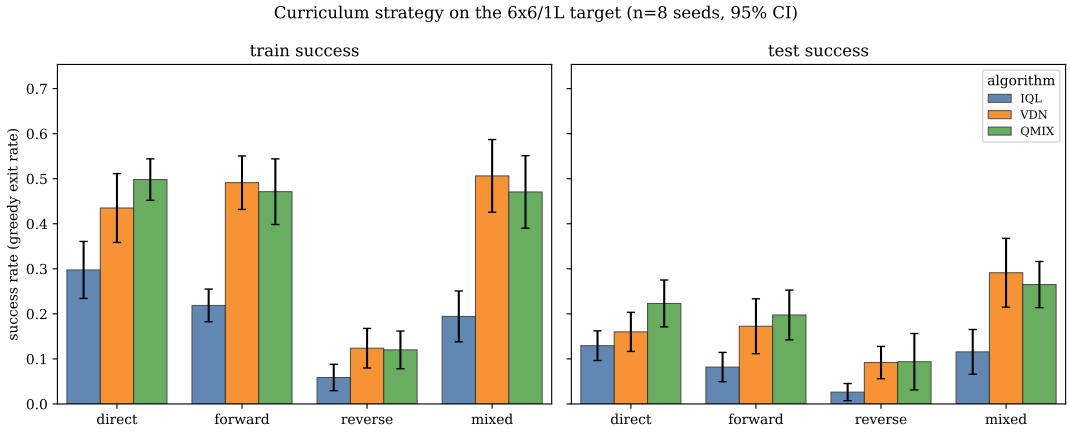

Figure 23: Final greedy success rate on the training and held-out test pools at the end of the 400,000-step budget, grouped by scheduling condition, with one bar per algorithm and 95% confidence-interval error bars over the eight seeds.

Two mechanisms explain why ordering does not help on a reachable target. First, under a fixed budget every step **forward** spends on the easier rungs is a step it does not spend on the target; when the target is directly learnable, that target experience dominates any transfer benefit, so moving budget to warm-up rungs can only match or cost final performance. Second, **reverse**, which reaches the target early and then trains on the easier rungs, collapses to near the floor: its training-pool success (0.10) is far below the others, the signature of catastrophic forgetting, since the network overwrites the target competence acquired early while fine-tuning on the navigation rung. The one condition that helps, **mixed**, is domain randomisation rather than a curriculum: its benefit comes from the **diversity** of exposure, not from any easy-to-hard ordering, and it is the same data-variety effect already isolated in Section 7.4.2.

The practical conclusion is that on a target the agent can already reach, scheduling the **order** of staged exposure offers no advantage over training directly on the target, and a poorly chosen order (reverse) is actively harmful. This leaves open the case curriculum learning is actually designed for: a target the agent **cannot** reach directly, addressed in Section 7.5.2.

7.5.2 The limits of curriculum learning on mutually-cooperative targets

This section addresses RQ6 of Chapter 1.

Section 7.5.1 isolated the curriculum **mechanism** on a target that direct training can already reach, and found that ordering confers no advantage. The motivating promise of curriculum learning is different, however: that staged exposure can make **reachable** a target that direct training cannot solve at all. The canonical such target in this thesis is the hand-crafted LLE Level 6, a 12×13 map with four agents and three lasers whose solution requires **mutual** cooperation: the dependency pattern in which several agents must each block a beam for the others before anyone can exit (Chapter 5). We therefore ask whether a curriculum of generated levels of growing cooperative complexity enables value-based agents to solve Level 6, and, more generally, whether such agents can cross the mutual-cooperation boundary at all.

Protocol

We approached the question in three stages of increasing ambition, each reusing the trainer, hyperparameters (Section A.3), and greedy 200-episode evaluation of the preceding experiments.

1. **Frontier probe.** Direct training from scratch on a fixed 6×6 grid with two agents and two lasers (the smallest instance whose solution requires **mutual** cooperation), for IQL, VDN, and QMIX. This isolates whether the mutual target carries any learnable signal at all before a curriculum is layered on top, holding the grid fixed so that the laser count is the only change from the reachable asymmetric target of Section 7.5.1.
2. **Two-laser curriculum.** The four scheduling conditions of Section 7.5.1 (direct, forward, reverse, mixed) on a fixed 6×6 grid, with a three-rung ladder that ramps only the cooperation requirement (zero, one, then two lasers) toward the mutual two-laser target, under a budget-matched 600,000-step budget.
3. **Level-6 transfer.** A four-stage curriculum of generated levels growing in both geometry and cooperation (6×6/1 laser $\rightarrow$ 8×8/2 lasers $\rightarrow$ 10×10/3 lasers $\rightarrow$ a 12×13 Level-6-style stage), with four agents throughout, evaluated on the hand-crafted Level 6. The curriculum condition (CURR) is compared against three baselines that isolate the contribution of staging: training only

on the hardest stage (B1), an anti-curriculum hard-to-easy ordering (B2), and direct training on Level 6 (B3). Runs use up to 2,000,000 environment steps, an order of magnitude beyond the reachable-target budget.

Because every preliminary condition returned zero success (below), and because a single run at the Level-6 budget is computationally expensive, we deliberately limited each sweep to a small number of seeds rather than expanding to the eight or twenty seeds used for the reachable targets. As the results show, the conclusion rests on the **uniformity** of the zero across grids, pool sizes, algorithms, budgets, and schedules, not on seed-level precision. Full per-run numbers and the exact stage configurations are tabulated in Section A.13.

Results

Across all three stages, no configuration learned to solve a mutually-cooperative target. Table 11 places these outcomes next to the **reachable** results of the preceding sections on a single axis of increasing cooperation difficulty. The contrast is the result: success is non-trivial as long as cooperation is **asymmetric** (a one-way blocking action) and collapses to zero the moment the target requires **mutual** blocking, and it stays at zero even when the training budget is increased tenfold.

Task (grid / agents / lasers)	Cooperation	Budget	Train	Test
5×5 / 2 / 1	asymmetric	200k	0.63	0.20
6×6 / 2 / 1	asymmetric	400k	0.41	0.17
6×6 / 2 / 2	**mutual**	600k	0.08	0.00
5×5 / 2 / 2	**mutual**	200k	0.00	0.00
12×13 / 4 / 3 (Level 6)	**mutual**	2M	0.00	0.00

Table 11: Greedy success rate as the cooperation requirement increases, pooled over algorithms and seeds. "Train" is success on the levels seen during training (the held-out **generated** pool for the Level-6 row); "Test" is success on the held-out target (Level 6 for the last row). The first two rows are the reachable asymmetric targets of Section 7.4.1 and Section 7.5.1; the last three are the mutual targets of this section. For the mutual rows the value is the mean over the available algorithm and seed runs (the 6×6 / 2-laser row is the direct-training frontier probe); for the Level-6 row it is the best achieved over the four conditions (B1, B2, B3, CURR). In every case held-out success is zero. Per-run numbers are in Section A.13.

On the **frontier probe**, all three algorithms occasionally solved a **training** level (a peak greedy success of 0.06–0.09) but never generalised: held-out success was exactly zero for every algorithm and seed. Shrinking the grid to 5×5 did not help: two lasers suffice to drive success to zero regardless of grid size. Two diagnostic checks confirm that this is a learnability wall rather than a budget shortfall: training on a **single** fixed mutual level (removing the need to generalise entirely) still plateaued below 0.12 success, and extending the budget to 1,500,000 steps left success flat, with the success curve peaking early and then **decaying** rather than climbing.

On the **two-laser curriculum**, every scheduling condition, including forward staging, finished at essentially zero held-out success (the best condition reached 0.01). Ordering a curriculum toward a target that carries no learnable signal does not create one.

On the **Level-6 transfer**, every condition reached 0.0 success on Level 6 and, tellingly, 0.0 success on the in-distribution held-out **generated** pool as well, even after 2,000,000 steps. The full four-stage curriculum (CURR) was no better than direct training on Level 6 (B3), and the agents' mean episode return stayed negative throughout: they accumulated step penalties without ever assembling the coordinated double-blocking sequence the level requires.

7.5.3 Discussion

The two regimes of Section 7.5.1 and Section 7.5.2 fail for complementary reasons. On a **reachable** target, a curriculum is unnecessary: ordering trades budget away from the target and, hard-to-easy, induces catastrophic forgetting, so it can only match or underperform direct training. On an **unreachable** mutual target, a curriculum is powerless, for three reasons that the experiments above separate.

- **There is no signal to amplify.** A curriculum works by transporting a policy with non-zero value from an easy stage to a harder one, where it stumbles onto reward more often and bootstraps. The mutual target returns reward only after a long, jointly-gated sequence (both lasers must be blocked before any agent can exit), so a from-scratch or warm-started policy receives identically zero reward (the negative returns above). A curriculum can make a faint signal louder; it cannot manufacture one from zero.
- **The missing skill is absent from every easier stage.** The easier rungs teach navigation and **asymmetric** cooperation, in which one agent blocks and the other passes. That pattern has a partial-credit path, since the passing agent can be rewarded before coordination is perfect. The mutual target requires a **deadlock-style** interdependency in which neither agent can make progress first. This is a discontinuity, not a ramp: no easy-to-hard ordering of asymmetric stages gradually constructs the mutual behaviour, because that behaviour is qualitatively absent below the final rung.
- **The algorithm cannot represent the solution.** Value-decomposition methods (VDN, QMIX) assume the joint action-value factorises monotonically into per-agent utilities. Mutual cooperation requires both agents to take individually-costly actions simultaneously, such as standing in a beam so the other may pass. This is the canonical case that monotonic factorisation cannot represent. The nonzero **training** but zero **held-out** success on the frontier probe is the fingerprint: the rare successes are memorised joint trajectories, not a generalisable coordination policy. Reordering the training data does not change the function class, so the representational bottleneck is left untouched.

The decisive observation is the zero held-out **generated**-pool success in the Level-6 transfer: the failure is not one of **transfer** or distribution shift between generated levels and the hand-crafted target: the agents never learned the base mutually-cooperative task at all. The failure of curriculum learning here is therefore not a failure of curriculum design but a consequence of the target lying beyond the reach of the underlying MARL method.

Scope and threats to validity. We are careful not to overclaim. These experiments establish that the value-based algorithms evaluated here (IQL, VDN, QMIX), under a fixed set of hyperparameters and a sparse joint-exit reward, do not solve mutually-cooperative LLE targets, and that no curriculum over generated levels changes this. We do **not** claim that curriculum learning can never succeed on such targets. Three caveats bound the claim. First, the algorithm family is narrow; centralised-critic actor-critic methods and approaches with explicit coordinated exploration were not tested and might cross the wall (Chapter 8). Second, the reward is sparse and unshaped; a dense or intrinsic signal that

credits partial coordination would change the very signal whose absence we identify as the bottleneck. Third, the seed counts on the mutual targets are small, a deliberate choice given the cost of each run and the uniformity of the zero outcome. They suffice to establish that success is zero, but not to measure small effects between equally-failing conditions. Directions that target the actual bottleneck, base learnability rather than curriculum ordering, are discussed in Chapter 8.

Conclusion

8.1 Summary

This thesis developed a SAT-based framework for reasoning about solvability and cooperation in a modelled subset of the Laser Learning Environment. We first formalised bounded-horizon solvability as a decision problem and reduced it to satisfiability of a CNF formula. On top of that reduction, we introduced a strict beam-semantics counterfactual that turns the blocking-based cooperation mechanism of LLE into a second decision problem on the same level, and refined the binary outcome into a profile taxonomy that classifies the dependency structure of a cooperative level.

These decision procedures were then embedded inside a family of procedural generators. The resulting framework does not guarantee that generated levels are pedagogically optimal for MARL, but it does guarantee that accepted levels satisfy the formal properties checked by the solver. This is the central contribution of the thesis: procedural generation coupled to **explicit certification** rather than procedural generation guided only by heuristic plausibility.

8.2 Answering the research questions

We restate the six research questions of Chapter 1 and assess what each chapter delivers.

- **RQ1 — formal verification of solvability.** Answered by the bounded-horizon SAT reduction of Chapter 4, whose correctness is established by Proposition 4.14. The encoding is polynomial-time constructible, places bounded-horizon LLE solvability in NP, and proves the problem is at most as hard as SAT.
- **RQ2 — formal verification of the cooperation requirement.** Answered by the strict-counterfactual encoding of Chapter 5, whose correctness is established by Theorem 5.1: a level requires cooperation if and only if the standard formula is SAT and the strict formula is UNSAT. The criterion is decidable on the same finite horizon used for solvability.
- **RQ3 — embedding verification inside a generator.** Answered by the generator family of Chapter 6. Each generator uses the SAT solver as an acceptance oracle, so every emitted level carries a solver certificate of the advertised property.
- **RQ4 — controlling the cooperation profile.** Answered constructively. The cooperation profile analyser of Section 5.6 labels every certified cooperative level, and the generators expose a profile filter that restricts acceptance to a chosen subset of the taxonomy. The profile-distribution experiment of Section 7.3.2 shows that the constructive generator reliably produces **mutual** profiles when $n_l \geq 2$ and admits **distributed** profiles when $n_l \geq 3$ on grids large enough. The **fully**

coupled label did not arise in the tested parameter range and remains a target for the future-work parameter-sensitivity study below.

- **RQ5 — learnability of certified cooperative levels.** Answered positively on a small grid. The learnability experiment of Section 7.4.1 shows that IQL, VDN, and QMIX all reach non-trivial greedy success rates (0.59–0.70 on the training pool, 0.18–0.23 on the held-out pool) on the 5×5 cooperative pool within a 200,000-step budget. The dominant difficulty observed is generalisation to the held-out pool rather than credit assignment. The data-scaling experiment of Section 7.4.2 shows this gap is a data-quantity effect rather than an algorithmic limit: enlarging the certified training pool to five hundred levels closes the train–test gap from 0.50 to approximately zero. The certification framework thus delivers a concrete downstream payoff — more certified data converts overfitting into generalisation.
- **RQ6 — curriculum transfer to LLE Level 6.** Answered negatively, within scope. The experiments of Section 7.5.1 and Section 7.5.2 show that, for the value-based algorithms evaluated here, a curriculum of generated levels confers no advantage. On a reachable asymmetric target, ordering the stages does not beat direct training, and only data **diversity** helps; on the mutually-cooperative LLE Level 6, no condition — including a four-stage curriculum trained for two million steps — exceeds zero success, and neither does direct training. Because the agents also fail on the in-distribution generated pool, the obstacle is the base task being unlearnable by these methods rather than a failure of curriculum **transfer**. We therefore read RQ6 as a scoped negative result and identify the learnability of mutual coordination — not curriculum design — as the lever for future work.

8.3 Future work

Several directions extend the present work naturally.

- **Complexity classification of bounded-horizon LLE solvability.** This thesis shows that bounded-horizon LLE solvability is in NP and is polynomial-time reducible to SAT, but we do not settle whether the problem is NP-hard. The exact placement has both theoretical and practical consequences: if the problem is NP-hard, the SAT-based oracle is optimal up to constant factors under the standard $\mathrm{P} \neq \mathrm{NP}$ assumption; if it lies in P, then in principle the SAT solver could be replaced by a polynomial-time algorithm tailored to the LLE structure, with a corresponding speed-up for every generator in the family. Establishing the lower bound is therefore both a theoretical complement to the present work and an engineering lever for the generator pipeline.
- **Model extension to gems and void tiles.** The formal model deliberately omits gems and treats void tiles as walls. Incorporating gem tiles would add an additional reachability condition — every agent must visit a designated gem cell at some point before reaching its exit — and would extend the property certified by the SAT oracle accordingly. Distinguishing void tiles from walls would require a revised movement semantics (agents cannot enter void cells under any condition, but the cell still propagates beams differently). Both extensions preserve the SAT-based architecture; only the encoding and the formal definitions need to be updated.
- **Parameter sensitivity and output diversity.** The acceptance-rate experiments fix the agent and laser counts at each grid size. A fuller characterisation would vary these parameters along independent axes to locate the practical frontier at which rejection rates become prohibitive, would measure within-pool diversity of accepted levels at a fixed parameter setting beyond the wall-Hamming statistics reported here, and would attempt to surface the **fully coupled** profile by raising n_l and the grid size.

- **Richer cooperation metrics.** The cooperation-profile taxonomy of Section 5.6 captures five dependency-graph patterns. Extensions could promote the synchronous-width scalar already exposed by the analyser into a first-class profile axis, and could add a chain-length difficulty axis on top of the qualitative chain label. These richer targets would let the generator family target **graded** cooperation difficulty rather than only qualitative profile families.
- **Algorithm-space breadth.** The learnability and curriculum-transfer experiments evaluate three value-based algorithms (IQL, VDN, QMIX). Comparison against centralised-critic actor-critic methods such as MADDPG [6] and COMA [7], and against latent-variable variants such as MAVEN [8], would give a fuller picture of which algorithm families benefit most from certified training material on coordination-critical tasks.
- **Making mutual coordination learnable.** The curriculum experiments of Section 7.5.2 locate the obstacle to solving LLE Level 6 in the base learnability of **mutual** cooperation under a sparse joint-exit reward, not in the ordering of training levels: no curriculum can amplify a learning signal that is identically zero. The natural levers therefore act on the reward and the algorithm rather than on the curriculum. A dense or intrinsic reward that credits partial coordination — for instance, an agent successfully blocking a beam for another — would supply the signal whose absence we identify as the bottleneck, and the centralised-critic or coordinated-exploration methods listed above could supply the representational capacity that monotonic value decomposition lacks. With a base learner able to acquire mutual coordination in isolation, the curriculum question of RQ6 could be revisited on a foundation that the present value-based methods do not provide.

The main open question is therefore not whether solver-based certification is possible in LLE; the present thesis answers that positively. The open question is how far that certification framework can be extended — in terms of richer mechanics, richer cooperation structures, downstream learning effects, and the exact complexity status of the underlying decision problem — before additional formal layers are required.

Bibliography

[1] L. S. Shapley, "Stochastic Games," *Proceedings of the National Academy of Sciences*, vol. 39, no. 10, pp. 1095–1100, 1953, doi: 10.1073/pnas.39.10.1095.

[2] M. L. Littman, "Markov Games as a Framework for Multi-Agent Reinforcement Learning," in *Machine Learning Proceedings 1994*, Morgan Kaufmann, 1994, pp. 157–163. doi: 10.1016/B978-1-55860-335-6.50027-1.

[3] R. S. Sutton and A. G. Barto, *Reinforcement Learning: An Introduction*, 2nd ed. Cambridge, MA: MIT Press, 2018. [Online]. Available: http://incompleteideas.net/book/the-book-2nd.html

[4] P. Sunehag *et al.*, "Value-Decomposition Networks For Cooperative Multi-Agent Learning Based On Team Reward," in *Proceedings of the 17th International Conference on Autonomous Agents and MultiAgent Systems*, in AAMAS 2018. International Foundation for Autonomous Agents, Multiagent Systems, 2018, pp. 2085–2087.

[5] T. Rashid, M. Samvelyan, C. Schroeder de Witt, G. Farquhar, J. Foerster, and S. Whiteson, "QMIX: Monotonic Value Function Factorisation for Deep Multi-Agent Reinforcement Learning," in *Proceedings of the 35th International Conference on Machine Learning*, in ICML 2018, vol. 80. PMLR, 2018, pp. 4295–4304.

[6] R. Lowe, Y. Wu, A. Tamar, J. Harb, P. Abbeel, and I. Mordatch, "Multi-Agent Actor-Critic for Mixed Cooperative-Competitive Environments," in *Advances in Neural Information Processing Systems 30*, in NeurIPS 2017. 2017, pp. 6379–6390.

[7] J. N. Foerster, G. Farquhar, T. Afouras, N. Nardelli, and S. Whiteson, "Counterfactual Multi-Agent Policy Gradients," in *Proceedings of the Thirty-Second AAAI Conference on Artificial Intelligence*, in AAAI-18. AAAI Press, 2018, pp. 2974–2982.

[8] A. Mahajan, T. Rashid, M. Samvelyan, and S. Whiteson, "MAVEN: Multi-Agent Variational Exploration," in *Advances in Neural Information Processing Systems 32*, in NeurIPS 2019. 2019, pp. 7613–7624.

[9] Y. Molinghen, R. Avalos, M. Van Achter, A. Nowé, and T. Lenaerts, "Laser Learning Environment: A New Environment for Coordination-Critical Multi-Agent Tasks," in *Artificial Intelligence and Machine Learning*, F. A. Oliehoek, M. Kok, and S. Verwer, Eds., Cham: Springer Nature Switzerland, 2025, pp. 135–154.

[10] M. Samvelyan *et al.*, "The StarCraft Multi-Agent Challenge," in *Proceedings of the 18th International Conference on Autonomous Agents and MultiAgent Systems (AAMAS)*, 2019, pp. 2186–2188.

[11] M. Carroll *et al.*, "On the Utility of Learning about Humans for Human-AI Coordination," in *Advances in Neural Information Processing Systems*, 2019.

[12] Y. Molinghen and T. Lenaerts, "Zero-Incentive Dynamics: A Look at Reward Sparsity Through the Lens of Unrewarded Subgoals." [Online]. Available: https://arxiv.org/abs/2507.01470

[13] C. Guestrin, M. G. Lagoudakis, and R. Parr, "Coordinated Reinforcement Learning," in *Proceedings of the Nineteenth International Conference on Machine Learning (ICML)*, 2002, pp. 227–234.

[14] U. Biswas, V. Palod, S. Bhambri, and S. Kambhampati, "Who is Helping Whom? Analyzing Interdependencies to Evaluate Cooperation in Human-AI Teaming." [Online]. Available: https://arxiv.org/abs/2502.06976

[15] N. Shaker, J. Togelius, and M. J. Nelson, *Procedural Content Generation in Games*. Springer, 2016. doi: 10.1007/978-3-319-42716-4.

[16] J. Togelius, G. N. Yannakakis, K. O. Stanley, and C. Browne, "Search-Based Procedural Content Generation: A Taxonomy and Survey," *IEEE Transactions on Computational Intelligence and AI in Games*, vol. 3, no. 3, pp. 172–186, 2011, doi: 10.1109/TCIAIG.2011.2148116.

[17] A. M. Smith and M. Mateas, "Answer Set Programming for Procedural Content Generation: A Design Space Approach," *IEEE Transactions on Computational Intelligence and AI in Games*, vol. 3, no. 3, pp. 187–200, 2011, doi: 10.1109/TCIAIG.2011.2158545.

[18] A. Summerville *et al.*, "Procedural Content Generation via Machine Learning (PCGML)," *IEEE Transactions on Games*, vol. 10, no. 3, pp. 257–270, 2018, doi: 10.1109/TG.2018.2846639.

[19] S. Risi and J. Togelius, "Increasing Generality in Machine Learning through Procedural Content Generation," *Nature Machine Intelligence*, vol. 2, no. 8, pp. 428–436, 2020, doi: 10.1038/s42256-020-0208-z.

[20] A. Khalifa, P. Bontrager, S. Earle, and J. Togelius, "PCGRL: Procedural Content Generation via Reinforcement Learning," in *Proceedings of the AAAI Conference on Artificial Intelligence and Interactive Digital Entertainment (AIIDE)*, 2020, pp. 95–101. doi: 10.1609/aiide.v16i1.7416.

[21] Y. Bengio, J. Louradour, R. Collobert, and J. Weston, "Curriculum Learning," in *Proceedings of the 26th Annual International Conference on Machine Learning*, in ICML 2009. ACM, 2009, pp. 41–48. doi: 10.1145/1553374.1553380.

[22] S. Narvekar, B. Peng, M. Leonetti, J. Sinapov, M. E. Taylor, and P. Stone, "Curriculum Learning for Reinforcement Learning Domains: A Framework and Survey," *Journal of Machine Learning Research*, vol. 21, no. 181, pp. 1–50, 2020.

[23] R. Wang, J. Lehman, J. Clune, and K. O. Stanley, "Paired Open-Ended Trailblazer (POET): Endlessly Generating Increasingly Complex and Diverse Learning Environments and Their Solutions," in *Proceedings of the Genetic and Evolutionary Computation Conference (GECCO '19)*, ACM, 2019, pp. 142–151. doi: 10.1145/3321707.3321799.

[24] M. Dennis *et al.*, "Emergent Complexity and Zero-shot Transfer via Unsupervised Environment Design," in *Advances in Neural Information Processing Systems*, 2020, pp. 13049–13061.

[25] H. Kautz and B. Selman, "Planning as Satisfiability," in *Proceedings of the 10th European Conference on Artificial Intelligence*, in ECAI '92. John Wiley & Sons, 1992, pp. 359–363.

[26] H. Kautz and B. Selman, "Pushing the Envelope: Planning, Propositional Logic, and Stochastic Search," in *Proceedings of the Thirteenth National Conference on Artificial Intelligence*, in AAAI-96. AAAI Press, 1996, pp. 1194–1201.

[27] J. Rintanen, K. Heljanko, and I. Niemelä, "Planning as satisfiability: parallel plans and algorithms for plan search," *Artificial Intelligence*, vol. 170, no. 12–13, pp. 1031–1080, 2006, doi: 10.1016/j.artint.2006.08.002.

[28] R. Stern *et al.*, "Multi-Agent Pathfinding: Definitions, Variants, and Benchmarks," in *Proceedings of the Twelfth International Symposium on Combinatorial Search (SoCS)*, 2019, pp. 151–158. [Online]. Available: https://arxiv.org/abs/1906.08291

[29] P. Surynek, "Problem Compilation for Multi-Agent Path Finding: A Survey," in *Proceedings of the Thirty-First International Joint Conference on Artificial Intelligence*, in IJCAI-22. 2022, pp. 5615–5620.

[30] G. Sharon, R. Stern, A. Felner, and N. R. Sturtevant, "Conflict-based search for optimal multi-agent pathfinding," *Artificial Intelligence*, vol. 219, pp. 40–66, 2015, doi: 10.1016/j.artint.2014.11.006.

[31] M. Frommknecht and P. Surynek, "Solving Multi-Agent Pathfinding with Stochastic Local Search SAT Algorithms," in *Proceedings of the 21st International Conference on Informatics in Control, Automation and Robotics*, in ICINCO 2024, vol. 1. 2024, pp. 67–78. doi: 10.5220/0012944800003822.

[32] M. Alshiekh, R. Bloem, R. Ehlers, B. Könighofer, S. Niekum, and U. Topcu, "Safe Reinforcement Learning via Shielding," in *Proceedings of the Thirty-Second AAAI Conference on Artificial Intelligence (AAAI)*, 2018, pp. 2669–2678. doi: 10.1609/aaai.v32i1.11797.

[33] S. A. Cook, "The Complexity of Theorem-Proving Procedures," in *Proceedings of the Third Annual ACM Symposium on Theory of Computing*, 1971, pp. 151–158.

[34] M. Davis, G. Logemann, and D. Loveland, "A Machine Program for Theorem-Proving," *Communications of the ACM*, vol. 5, no. 7, pp. 394–397, 1962, doi: 10.1145/368273.368557.

[35] N. Eén and N. Sörensson, "An Extensible SAT-solver," in *Theory and Applications of Satisfiability Testing – SAT 2003*, in Lecture Notes in Computer Science, vol. 2919. Springer, 2004, pp. 502–518. doi: 10.1007/978-3-540-24605-3_37.

[36] A. Ignatiev, A. Morgado, and J. Marques-Silva, "PySAT: A Python Toolkit for Prototyping with SAT Oracles," in *Theory and Applications of Satisfiability Testing – SAT 2018*, in Lecture Notes in Computer Science, vol. 10929. Springer, 2018, pp. 428–437. doi: 10.1007/978-3-319-94144-8_26.

[37] V. Mnih *et al.*, "Human-level control through deep reinforcement learning," *Nature*, vol. 518, no. 7540, pp. 529–533, 2015, doi: 10.1038/nature14236.

Appendix

A.1 Benchmark levels for SAT encoding comparison

The four levels used in the SAT encoding comparison (Chapter 7) are shown below with their exact parameters.

Level	Grid	Agents	Lasers	Horizon $T_{\max}$
Synthetic 3×3	3×3	2	1	4
Synthetic 5×5	5×5	3	2	5
Synthetic 8×8	8×8	4	3	15
Benchmark Level 6	12×13	4	3	21

Table 12: Parameters of the four benchmark levels used in the SAT encoding comparison.

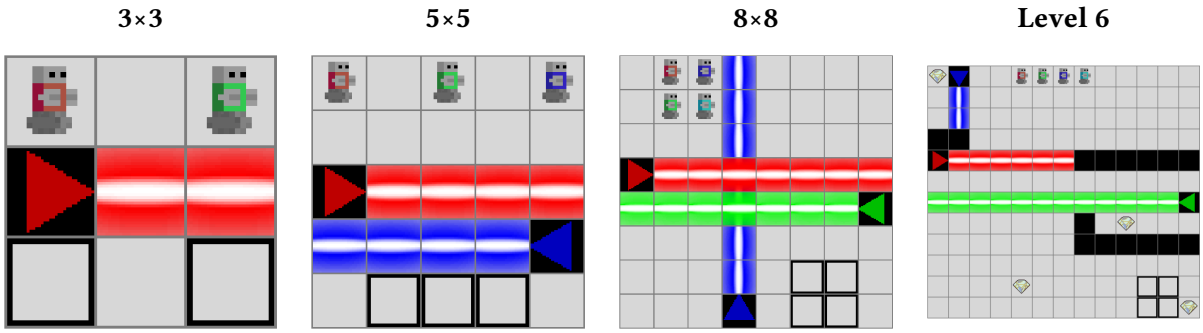

Figure 24: Visual representation of the four benchmark levels.

A.2 Reproducibility — software and seed conventions

All experiments are reproducible from a single Python source tree. The relevant versions and seed conventions are summarised below.

Component	Version / Value
Python	3.12 or later (3.13 used for the runs reported here)
SAT solver	Minisat22 via the PySAT interface [36]
LLE engine	`laser-learning-environment` (Python + Rust bindings)
MARL trainer	`marl` framework, called from `src/experiments/learnability/run_experiment.py`
Plotting backend	`matplotlib`
Solver-comparison seeds	None (deterministic CNF; only timings vary across runs)
Rejection benchmark	Seeded (`seed=20260530`, env `REJECTION_SEED`); 200 trials (3×3, 5×5) or 20 trials (8×8); per-trial attempt distributions shown as boxplots
Profile benchmark	Unseeded (`seed=None`); reported as counts over 100 (5×5) or 50 (8×8) accepted levels
Learnability training-pool seed	20260618
Learnability training seeds	$\mathcal{S} = \{0, 1, \ldots, 19\}$ (twenty seeds per algorithm)
Data-scaling pool seeds	20260618 (train), 20260619 (test); pools nested, test size 50
Data-scaling training seeds	$\mathcal{S} = \{0, 1, \ldots, 19\}$ per algorithm and pool size
Curriculum-strategy seeds	$\mathcal{S} = \{0, 1, \ldots, 7\}$ (eight seeds per condition and algorithm)
Curriculum two-laser / Level-6 seeds	Small per-condition sets (1–6 seeds); see Section A.13
Appendix-gallery pool seeds	Distinct per pool, listed in each gallery parameter table

Table 13: Software components and seed conventions. Hardware specifications are deferred; the SAT benchmarks run on commodity laptops and the MARL runs on a CUDA workstation with sm_61 GPUs.

A.3 Learnability hyperparameters

The learnability experiment (Section 7.4.1), the data-scaling experiment (Section 7.4.2), and the curriculum experiments (Section 7.5.1, Section 7.5.2) share the same trainer construction in `src/experiments/learnability/run_experiment.py` and the curriculum runners under `src/experiments/`. The hyperparameters listed in Table 14 are identical across the three algorithms (IQL, VDN, QMIX) and are reused unchanged for the data-scaling and curriculum runs; the only addition is the per-stage exploration reset used by the curriculum runs, described in Section 7.5.1.

Hyperparameter	Value	Source
Optimiser	Adam	`marl.algos.{DQN,VDN,QMix}` default
Learning rate	5×10^{-4}	`run_experiment.py` (`lr=5e-4`)
Batch size	64	`run_experiment.py` (`batch_size=64`)
Discount factor γ	0.95	`run_experiment.py` (`gamma=0.95`)
Train interval	1 update / 5 env steps	`run_experiment.py` (`train_interval=(5,"step`
Gradient-norm clipping	10.0	`run_experiment.py` (`grad_norm_clipping=10`)
ε schedule (train)	linear $1.0 \rightarrow 0.05$ over 100,000 steps	`EpsilonGreedy.linear(1.0, 0.05, 100_000)`
Evaluation policy	greedy ($\varepsilon = 0$)	`ArgMax()`
Q-network architecture	marl `qnetworks.from_env` default	marl framework default
Mixer (QMIX)	`mixers.QMix.from_env`	`run_experiment.py`
Mixer (VDN)	sum of agent Q-values	`marl.algos.VDN` implicit mixer
Mixer (IQL)	none	`run_experiment.py` (`mixer=None`)
Independent Q-network heads (IQL, VDN)	yes	`qnetworks.from_env(..., independent=True)`
Independent Q-network heads (QMIX)	no (shared)	`qnetworks.from_env(...)` default

Table 14: Hyperparameters used in the learnability experiment (Section 7.4.1) and reused unchanged for the data-scaling (Section 7.4.2) and curriculum (Section 7.5.1, Section 7.5.2) experiments.

A.4 SAT encoding — per-family clause counts

The figures of Chapter 7 report total CNF size for the four benchmark levels under the two movement formulations. Table 15 gives the per-constraint-family decomposition behind those totals.

Level	Method	Initialisation	Movement	Laser	Total
3×3	local	23	537	225	785
3×3	global	23	505	225	753
5×5	local	87	4 233	1 884	6 204
5×5	global	87	6 243	1 884	8 214
8×8	local	304	54 076	23 136	77 516
8×8	global	304	143 296	23 136	166 736
Level 6	local	690	174 592	77 682	252 964
Level 6	global	690	1 089 268	77 682	1 167 640

Table 15: Clause counts per constraint family for the two SAT movement formulations on the four benchmark levels. Source: `results/sat_encoding/benchmark_results.json`.

A.5 SAT encoding — generation and solve durations

Table 16 reports the median CNF generation and SAT solve durations for each (level, method) pair over the 100 timing runs of the benchmark protocol described in Section 7.1; the interquartile range follows in parentheses. We report medians rather than means because the per-run generation duration is heavy-tailed (occasional warm-up and garbage-collection pauses inflate the mean), as the boxplots of Chapter 7 make visible.

Level	Method	Generation (ms)	Solve (ms)	Total (ms)
3×3	local	0.16 (0.16–0.17)	0.010 (0.010–0.011)	0.17
3×3	global	0.13 (0.12–0.14)	0.012 (0.011–0.015)	0.14
5×5	local	1.26 (1.23–1.47)	0.062 (0.060–0.065)	1.32
5×5	global	1.25 (1.20–1.45)	0.122 (0.120–0.127)	1.38
8×8	local	16 (15–63)	3.69 (3.64–3.79)	20.0
8×8	global	75 (26–78)	48 (48–49)	123
Level 6	local	111 (60–114)	4.6 (4.3–5.0)	115
Level 6	global	466 (457–485)	38 (37–40)	505

Table 16: Median CNF generation, SAT solve, and total durations (milliseconds) per level and movement formulation over 100 timing runs, with the interquartile range in parentheses. Source: `results/sat_encoding/benchmark_results.json`.

A.6 Generator rejection — detailed numbers

The figures of Section 7.3.1 report per-generator rejection rates and mean attempts. Table 17 gives the full numbers behind those plots, including the number of successful trials, the number of failed trials (per-trial budget exhausted), and the mean number of attempts per accepted level.

Generator	Grid	Success	Fail	Mean attempts	Rejection (%)
Constrained Random (solvable)	3×3	200	0	3.2	68.8
Constrained Random (solvable)	5×5	200	0	7.6	86.9
Constrained Random (solvable)	8×8	20	0	6.5	84.5
Constrained Random (cooperative)	3×3	200	0	74.8	98.7
Constrained Random (cooperative)	5×5	200	0	77.5	98.7
Constrained Random (cooperative)	8×8	12	8	14.8	93.3
Constructive (solvable)	3×3	200	0	1.00	0.0
Constructive (solvable)	5×5	200	0	1.11	10.3
Constructive (solvable)	8×8	20	0	1.20	16.7
Constructive (cooperative)	3×3	200	0	1.00	0.0
Constructive (cooperative)	5×5	200	0	1.06	5.7
Constructive (cooperative)	8×8	19	1	1.00	0.0
Level-6-Style	3×3	200	0	74.8	98.7
Level-6-Style	5×5	200	0	5.7	82.5
Level-6-Style	8×8	20	0	4.1	75.3

Table 17: Detailed rejection-benchmark numbers per generator setting and grid size. "Success" is the number of successful trials and "Fail" the number of trials that exhausted their per-trial budget (500 attempts for the small grids; 100 attempts or 30 seconds for the 8×8 grid). Mean attempts and rejection rate are computed over the successful trials only. The 8×8 Constrained Random cooperative row is the most expensive setting, with 8 of 20 trials exhausting the time budget; its mean attempts is therefore an optimistic estimate over the trials that completed. That configuration also intermittently crashes the SAT solver's native extension, so each attempt is run in an isolated worker subprocess that discards and resamples the offending candidate. Source: `results/rejection_benchmark/benchmark_results.json`.

A.7 Cooperation profile distribution — detailed counts

The figure of Section 7.3.2 shows the cooperation-profile breakdown of accepted cooperative levels for three generator settings and two grid sizes. Table 18 gives the raw counts.

Generator	Grid	n	asym.	mutual	chain	distr.	full
Constrained Random (cooperative)	5×5	100	100	0	0	0	0
Constrained Random (cooperative)	8×8	50	46	0	1	3	0
Constructive (cooperative)	5×5	100	100	0	0	0	0
Constructive (cooperative)	8×8	50	2	46	0	2	0
Level-6-Style	5×5	100	100	0	0	0	0
Level-6-Style	8×8	50	13	34	0	3	0

Table 18: Raw profile-count breakdown per generator and grid size. The columns **asym.**, **mutual**, **chain**, **distr.**, **full** correspond to the five cooperation-profile labels classified by the analyser of Section 5.6. The 5×5 configuration uses 2 agents and 1 laser; the 8×8 configuration uses 3 agents and 2 lasers. Source: `results/profile_benchmark/benchmark_results.json`.

A.8 Learnability — per-seed final success rates

Table 19 lists the final greedy success rate of every (algorithm, seed) cell in the learnability experiment of Section 7.4.1. Each row is one trained agent; "Train" is the success rate on the 20-level training pool and "Test" the success rate on the 20-level held-out pool, each estimated from 200 greedy evaluation episodes.

Algorithm	Seed	Train	Test
IQL	0	0.56	0.17
IQL	1	0.62	0.38
IQL	2	0.55	0.31
IQL	3	0.67	0.14
IQL	4	0.67	0.25
IQL	5	0.59	0.2
IQL	6	0.73	0.17
IQL	7	0.58	0.19
IQL	8	0.51	0.28
IQL	9	0.5	0.33
IQL	10	0.65	0.19
IQL	11	0.62	0.25
IQL	12	0.59	0.3
IQL	13	0.61	0.25
IQL	14	0.66	0.18
IQL	15	0.5	0.25
IQL	16	0.49	0.24
IQL	17	0.61	0.18
IQL	18	0.76	0.18
IQL	19	0.6	0.17
QMIX	0	0.54	0.19
QMIX	1	0.62	0.18
QMIX	2	0.55	0.23
QMIX	3	0.61	0.19
QMIX	4	0.64	0.08
QMIX	5	0.53	0.23
QMIX	6	0.59	0.32
QMIX	7	0.57	0.28
QMIX	8	0.56	0.24
QMIX	9	0.6	0.28
QMIX	10	0.48	0.26
QMIX	11	0.69	0.19
QMIX	12	0.73	0.13
QMIX	13	0.76	0.21
QMIX	14	0.56	0.16
QMIX	15	0.55	0.05
QMIX	16	0.6	0.07
QMIX	17	0.56	0.2
QMIX	18	0.57	0.23
QMIX	19	0.6	0.23
VDN	0	0.54	0.08
VDN	1	0.78	0.14
VDN	2	0.73	0.11
VDN	3	0.61	0.14
VDN	4	0.77	0.17
VDN	[illegible]	[illegible]	[illegible]

Table 19: Per-(algorithm, seed) final greedy success rates from the learnability experiment of Section 7.4.1. 60 rows total (3 algorithms × 20 seeds). Source: `results/learnability_5x5/aggregated.json`, produced by `src/scripts/aggregate_learnability_results.py`.

A.9 Learnability — training pool

Cooperative pool used as $\mathcal{D}_{\text{train}}$ for Section 7.4.1.

Field	Value
Pool path	`results/learnability_5x5/levels/5x5_2a_1L_cooperative/train`
Grid	5 × 5
Agents	2
Lasers	1
$T_{\max}$	10
Generator	Constructive (cooperative mode)
Pool seed	20260618
Number of levels	20

Table 20: Parameters of the learnability training pool.

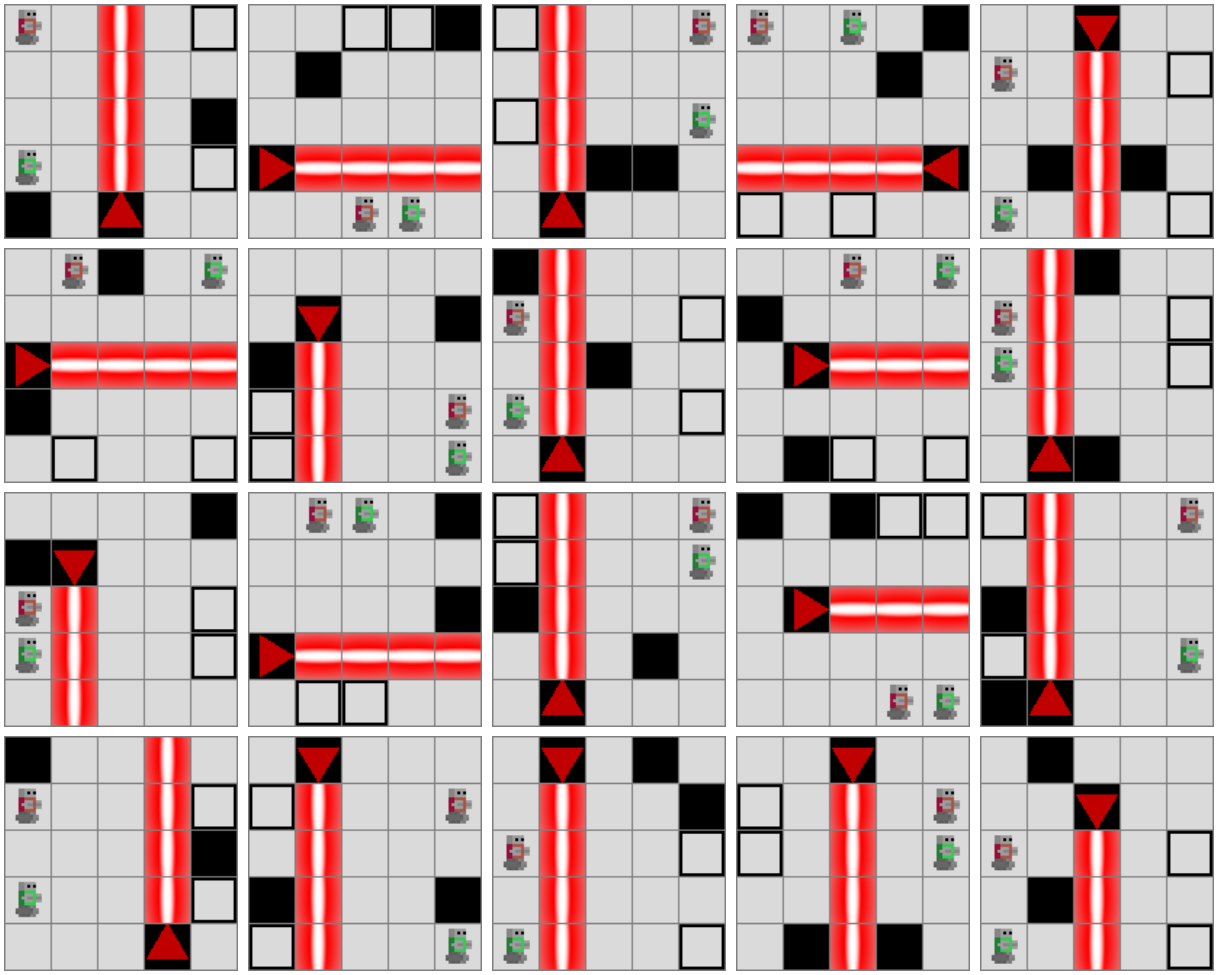

Figure 25: All 20 levels of the learnability training pool, in pool order.

A.10 Learnability — test pool

Held-out cooperative pool used as $\mathcal{D}_{\text{test}}$ for Section 7.4.1.

Field	Value
Pool path	`results/learnability_5x5/levels/5x5_2a_1L_cooperative/test`
Grid	5 × 5
Agents	2
Lasers	1
$T_{\max}$	10
Generator	Constructive (cooperative mode)
Pool seed	20260619
Number of levels	20

Table 21: Parameters of the learnability test pool.

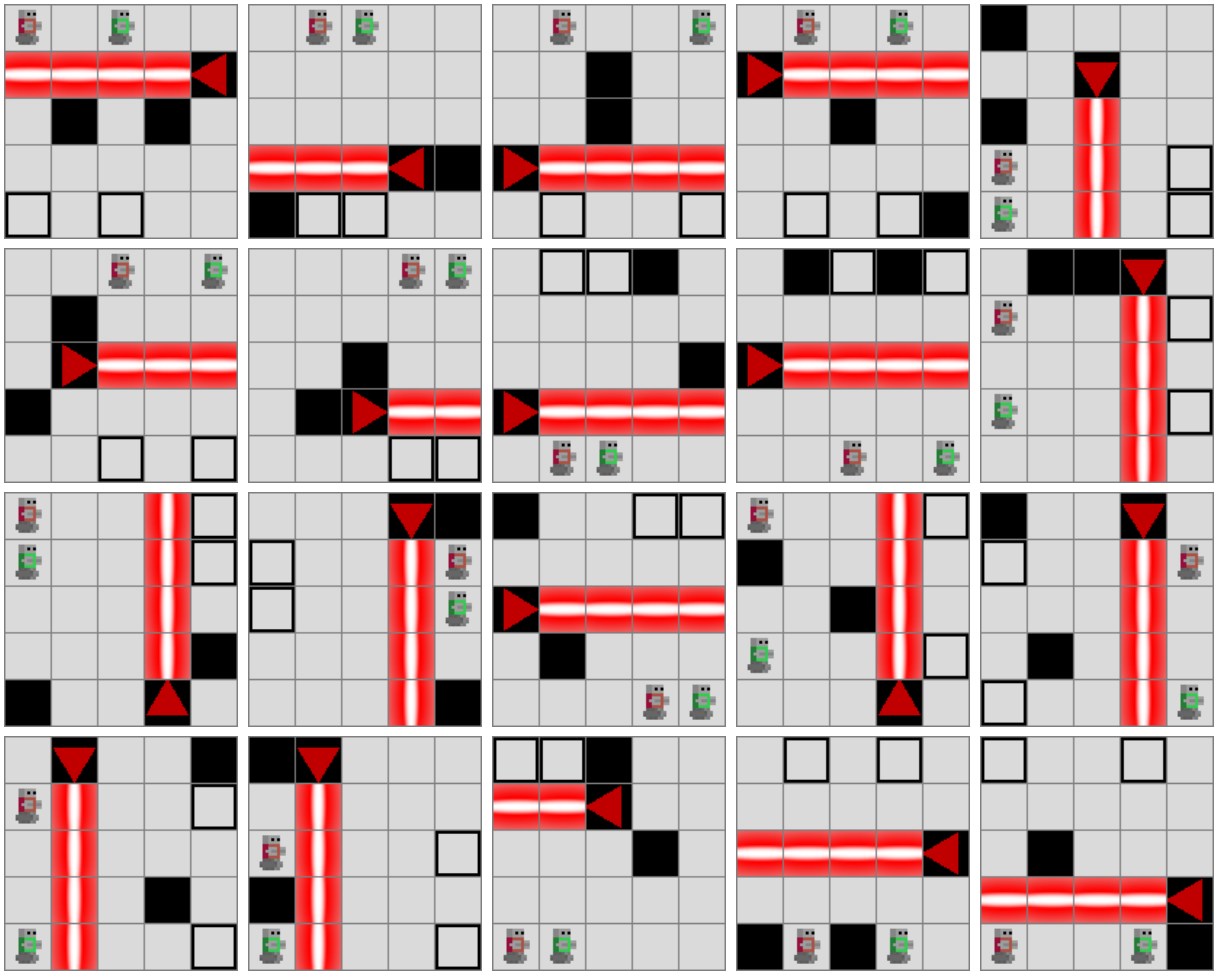

Figure 26: All 20 levels of the learnability test pool, in pool order.

A.11 Data-scaling experiment — configuration

The data-scaling experiment (Section 7.4.2) reuses the learnability task, generator, and hyperparameters (Section A.3) unchanged, and varies only the training-pool size. Table 22 lists the full configuration.

Field	Value
Grid / agents / lasers	5 × 5 / 2 / 1
Horizon $T_{\max}$	10
Generator	Constructive (cooperative mode)
Training-pool sizes $\lvert\mathcal{D}_{\text{train}}\rvert$	20, 100, 500 (nested)
Held-out test-pool size $\lvert\mathcal{D}_{\text{test}}\rvert$	50 (fixed across all conditions)
Training-pool seed	20260618
Test-pool seed	20260619
Environment steps per run	300,000
Algorithms	IQL, VDN, QMIX
Training seeds	$\mathcal{S} = \{0, 1, \ldots, 19\}$
Runs per pool size	60 (3 algorithms × 20 seeds)

Table 22: Configuration of the data-scaling experiment (Section 7.4.2).

The three training pools are nested prefixes of a single seeded stream (seed 20260618): the 20-level pool is exactly the learnability training pool of Section A.9, the 100-level pool contains it, and the 500-level pool contains both. The 50-level held-out test pool (seed 20260619) is a superset of the 20-level learnability test pool of Section A.10. Per-level renderings of the 20-level pool therefore already appear in Figure 25 and Figure 26; the additional levels in the 100- and 500-level pools are further independent draws from the same constructive cooperative generator, representative samples of which are shown in Section A.17. They are not reproduced individually here.

The per-pool-size learning curves below mirror Figure 18: each panel pair shows the train- and test-pool success rate against environment steps for IQL, VDN, and QMIX, with 95% confidence bands over the seeds. Read together, they make the effect of Section 7.4.2 visible step by step — as the training pool grows from 20 to 100 to 500 levels, the test curves (right panel of each figure) climb toward the training curves (left panel), i.e. the generalisation gap closes.

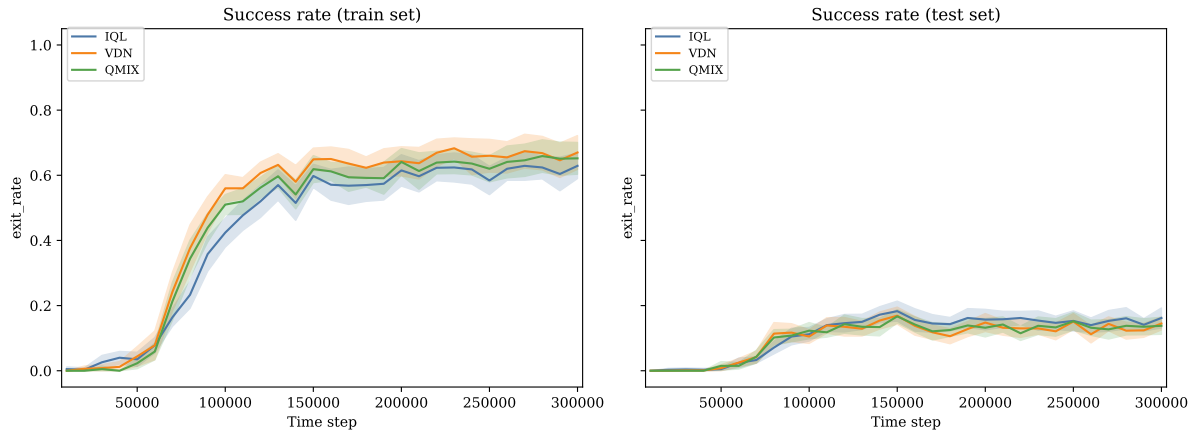

Figure 27: Data scaling with $|\mathcal{D}_{\text{train}}| = 20$ levels: mean train- and test-pool success rate versus environment steps, per algorithm, with 95% confidence bands over the seeds. The test curves plateau far below the training curves (large gap).

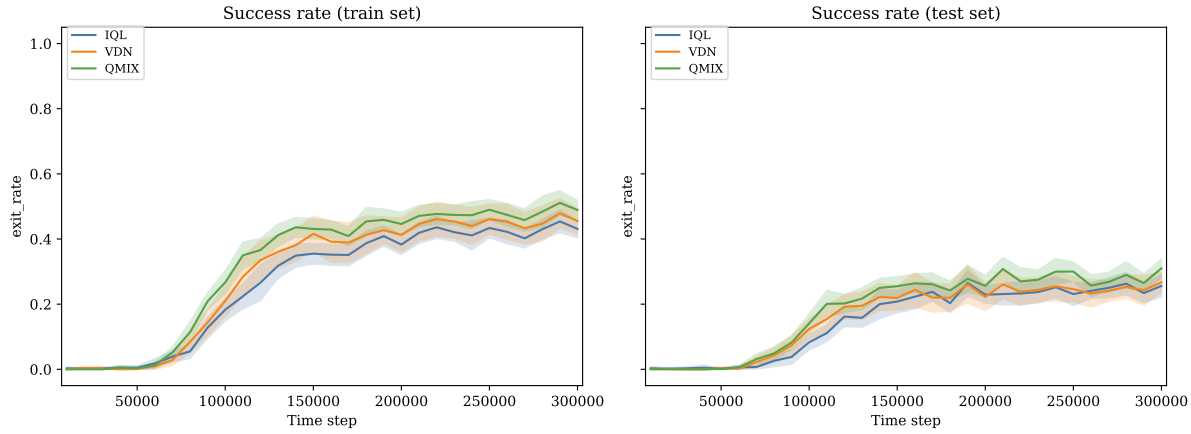

Figure 28: Data scaling with $|\mathcal{D}_{\text{train}}| = 100$ levels: the training curves drop and the test curves rise relative to the 20-level case, narrowing the gap.

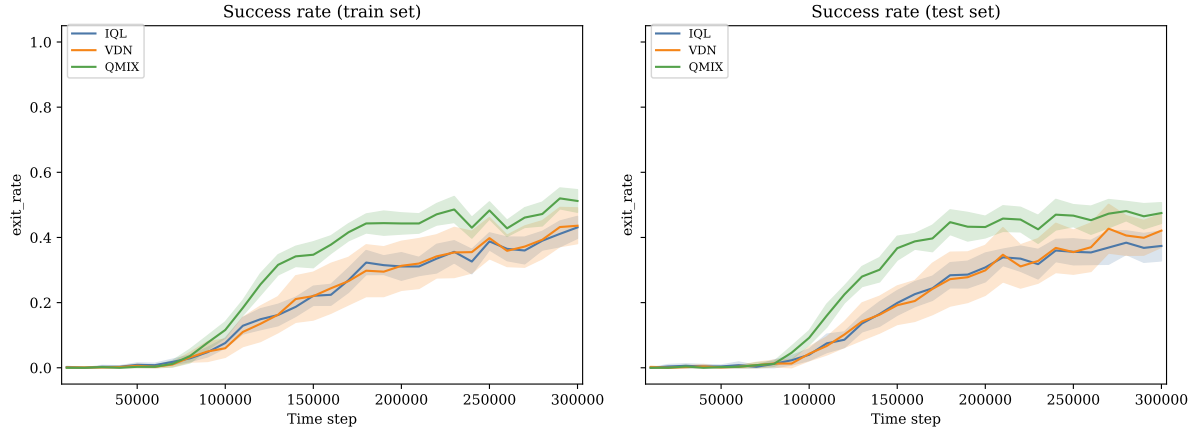

Figure 29: Data scaling with $|\mathcal{D}_{\text{train}}| = 500$ levels: the train and test curves nearly coincide — the agent generalises to held-out levels about as well as it fits the training pool.

A.12 Curriculum-strategy experiment — configuration and per-cell results

The curriculum-strategy experiment (Section 7.5.1) compares four budget-matched scheduling conditions on a 6×6 / 2-agent / 1-laser cooperative target reachable by the direct baseline. Table 23 gives the full configuration and Table 24 the per-(condition, algorithm) final success rates behind the pooled summary of Table 10.

Field	**Value**
Difficulty ladder	4×4/0L (random, $T_{\max} = 8$) $\rightarrow$ 5×5/1L (cooperative, $T_{\max} = 10$) $\rightarrow$ 6×6/1L target (cooperative, $T_{\max} = 12$)
Agents	2 (fixed across rungs; smaller observations zero-padded)
Train pool per rung	100 certified levels
Held-out target pool	50 levels
Conditions	direct, forward $(50, 150, 200) \times 10^3$, reverse (same per-rung budgets, reversed), mixed (uniform rung per episode)
Total budget per run	400,000 environment steps
Algorithms	IQL, VDN, QMIX
Seeds	$\mathcal{S} = \{0, 1, \ldots, 7\}$ (eight per condition and algorithm)
Exploration	ε reset to 1.0 at each stage boundary, decayed to 0.05 over 30% of that rung's budget
Evaluation	greedy, on the 6×6 target; 50 episodes every 10,000 steps and 200 episodes at the end

Table 23: Configuration of the curriculum-strategy experiment (Section 7.5.1).

Condition	Algorithm	n	Train mean $\pm$ CI95	Test mean $\pm$ CI95
direct	IQL	8	0.30 ± 0.06	0.13 ± 0.03
direct	VDN	8	0.43 ± 0.08	0.16 ± 0.04
direct	QMIX	8	0.50 ± 0.05	0.22 ± 0.05
forward	IQL	8	0.22 ± 0.04	0.08 ± 0.03
forward	VDN	8	0.49 ± 0.06	0.17 ± 0.06
forward	QMIX	8	0.47 ± 0.07	0.20 ± 0.06
mixed	IQL	8	0.19 ± 0.06	0.12 ± 0.05
mixed	VDN	8	0.51 ± 0.08	0.29 ± 0.08
mixed	QMIX	8	0.47 ± 0.08	0.27 ± 0.05
reverse	IQL	8	0.06 ± 0.03	0.03 ± 0.02
reverse	VDN	8	0.12 ± 0.04	0.09 ± 0.04
reverse	QMIX	8	0.12 ± 0.04	0.09 ± 0.06

Table 24: Final greedy success rate per (condition, algorithm) on the 6×6 / 2-agent / 1-laser target, each aggregated over eight seeds. Means with 95% confidence intervals ($\pm t_{7,0.025}\frac{\sigma}{\sqrt{8}}$, $t_{7,0.025} \approx 2.365$). Source: `results/curriculum_strategy/runs/`.

A.12.1 Generated training pools

Each rung is SAT-generated once from the master seed `RNG_SEED = 20260521`; the per-rung training pool uses the derived seed $20260521 + 100 \times \text{stage}$ and the held-out target pool the same expression $+1$, so every draw is reproducible and the target's train and eval pools never coincide. Unspecified wall budgets fall back to the generator default $\lfloor \text{grid area} / 10 \rfloor$. Table 25 lists the per-rung parameters; Figure 30, Figure 31, and Figure 32 show sixteen representative training levels from each rung.

Rung	Grid	Agents	Lasers	$T_{\max}$	Walls	Generator	Train	Eval
S1	4×4	2	0	8	1	Random (solvable)	100	0
S2	5×5	2	1	10	2	Constructive (cooperative)	100	0
S3	6×6	2	1	12	3	Constructive (cooperative)	100	50

Table 25: Per-rung generation parameters for the curriculum-strategy pools (Section 7.5.1). Source: `src/experiments/curriculum_strategy/configs.py`; pools rendered by `src/scripts/render_curriculum_strategy_pools.py`.

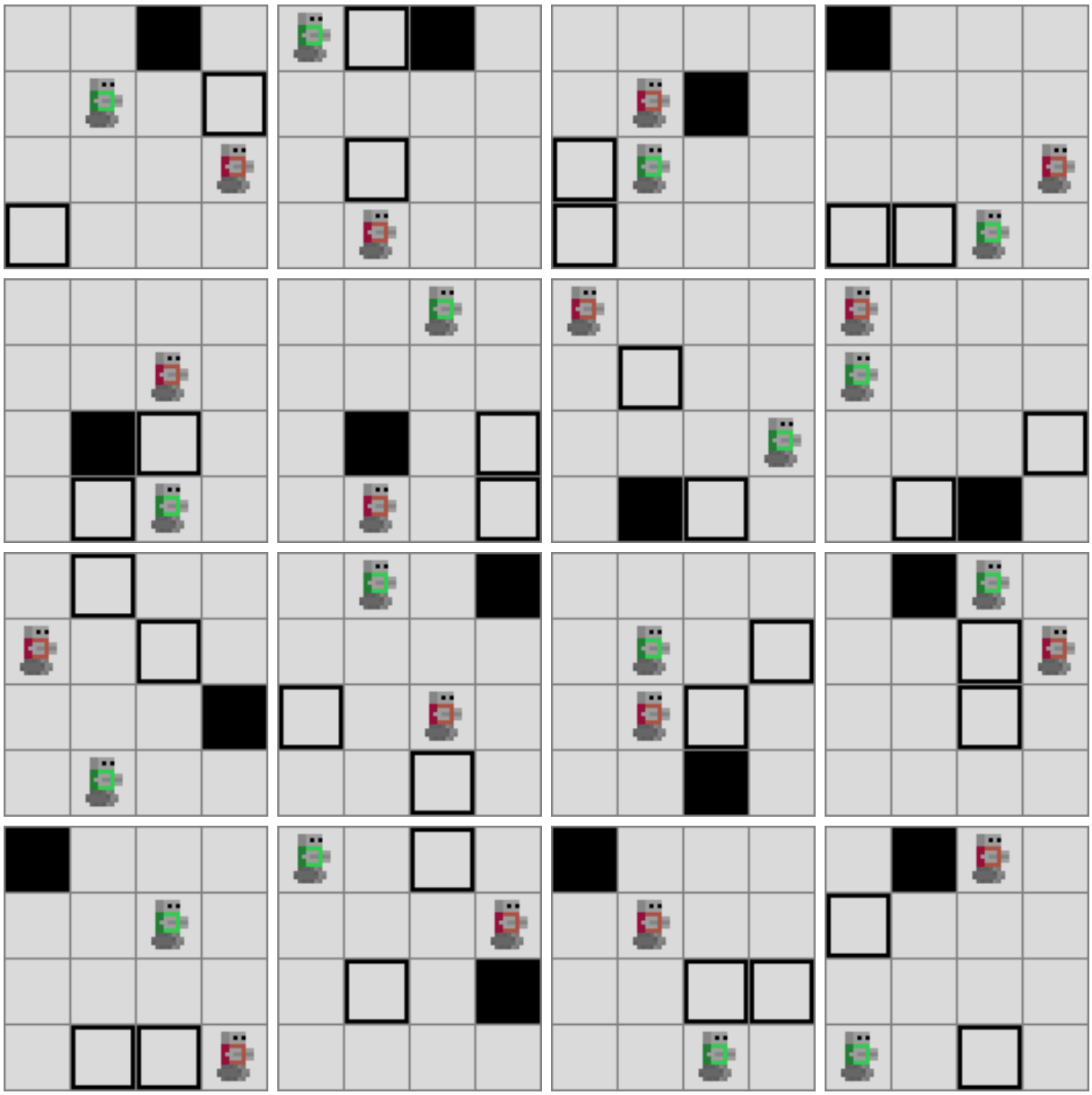

Figure 30: Sixteen S1 training levels: 4×4, 2 agents, 0 lasers (navigation warm-up).

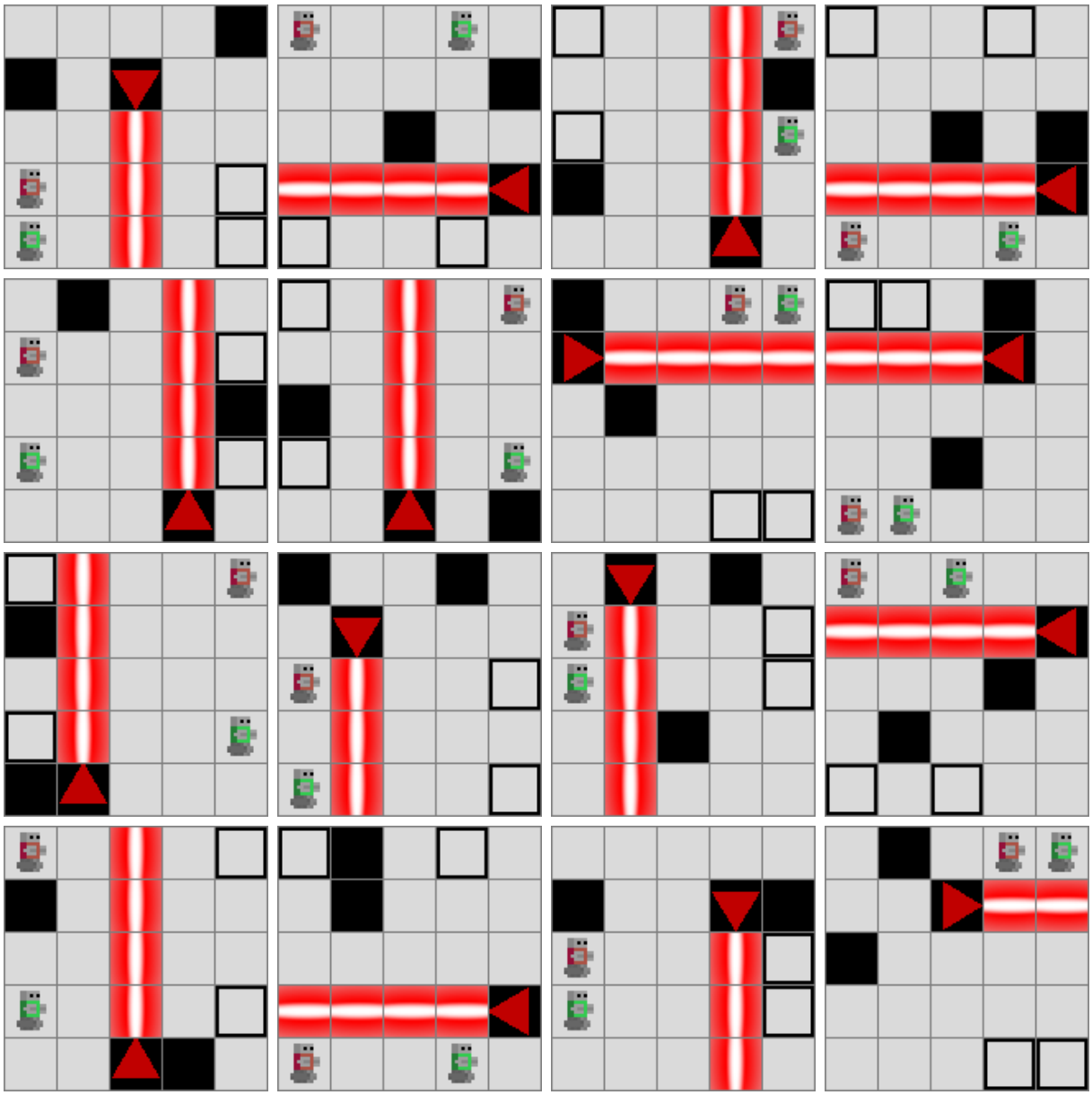

Figure 31: Sixteen S2 training levels: 5×5, 2 agents, 1 laser (asymmetric cooperation).

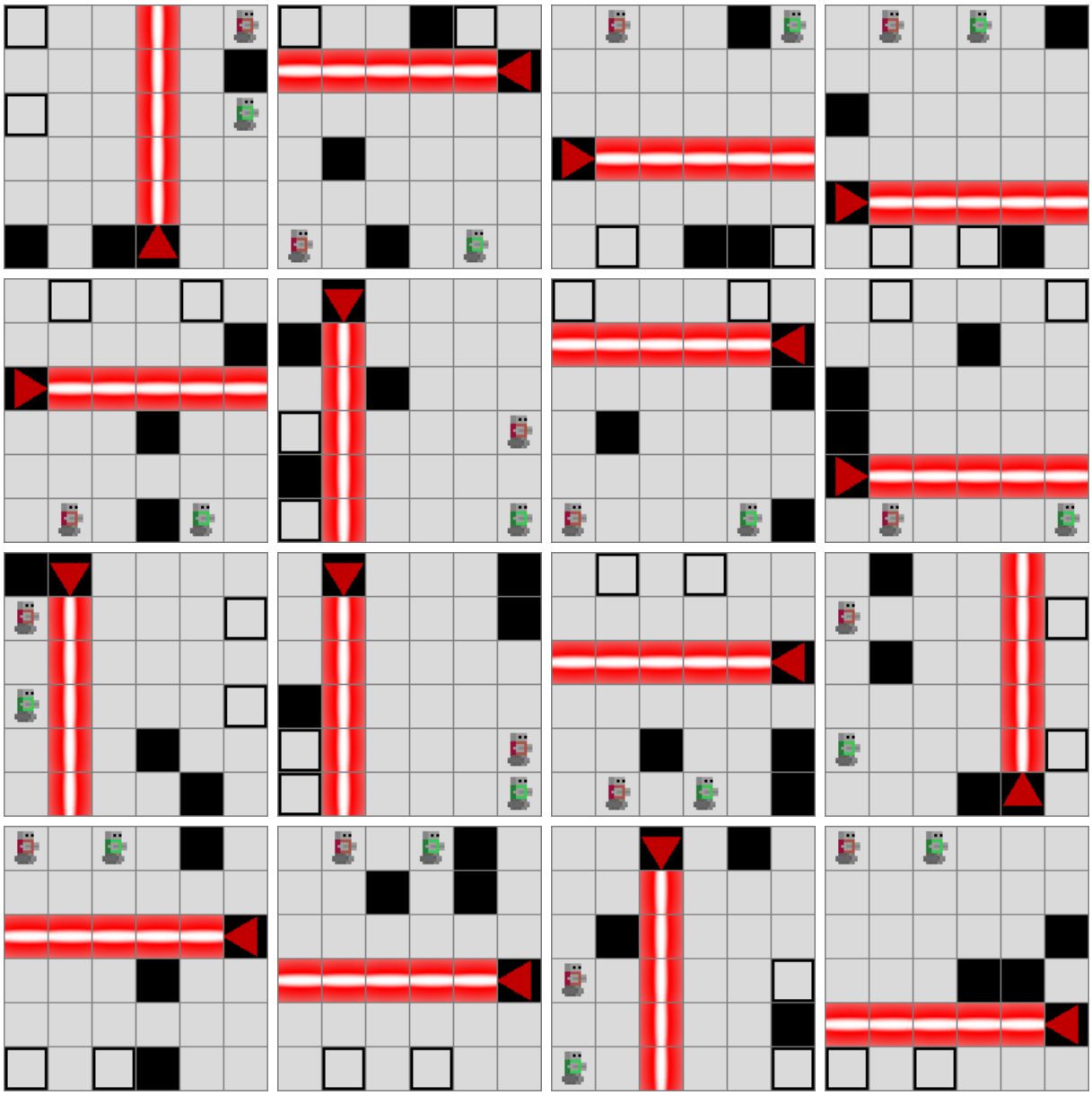

Figure 32: Sixteen S3 (target) training levels: 6×6, 2 agents, 1 laser (asymmetric cooperation).

A.13 Curriculum-transfer experiments — configurations and results

This appendix collects the configurations and per-run results behind the three stages of Section 7.5.2: the frontier probe, the two-laser curriculum, and the Level-6 transfer. Across all three, held-out success on the mutually-cooperative target is uniformly zero; the tables below give the supporting numbers and the exact stage geometries.

A.13.1 Frontier probe and diagnostic checks

Direct training from scratch on a fixed 6×6 / 2-agent / 2-laser **mutual** target ($T_{\max} = 18$, constructive cooperative generator constrained to the `fully_coupled` profile, a 20-level training pool and a 20-level held-out pool), 600,000 steps, three seeds per algorithm. Table 26 reports the final greedy success rates:

every algorithm reaches a small nonzero **training** success and exactly zero **held-out** success. Sixteen of the training levels are shown in Figure 33.

Algorithm	n	Train (mean)	Test (mean)
IQL	3	0.09	0.00
VDN	3	0.06	0.00
QMIX	3	0.08	0.00

Table 26: Frontier probe: final greedy success on the 6 × 6 / 2-agent / 2-laser mutual target, direct training from scratch. Source: `results/learnability_6x6_2L/runs/`.

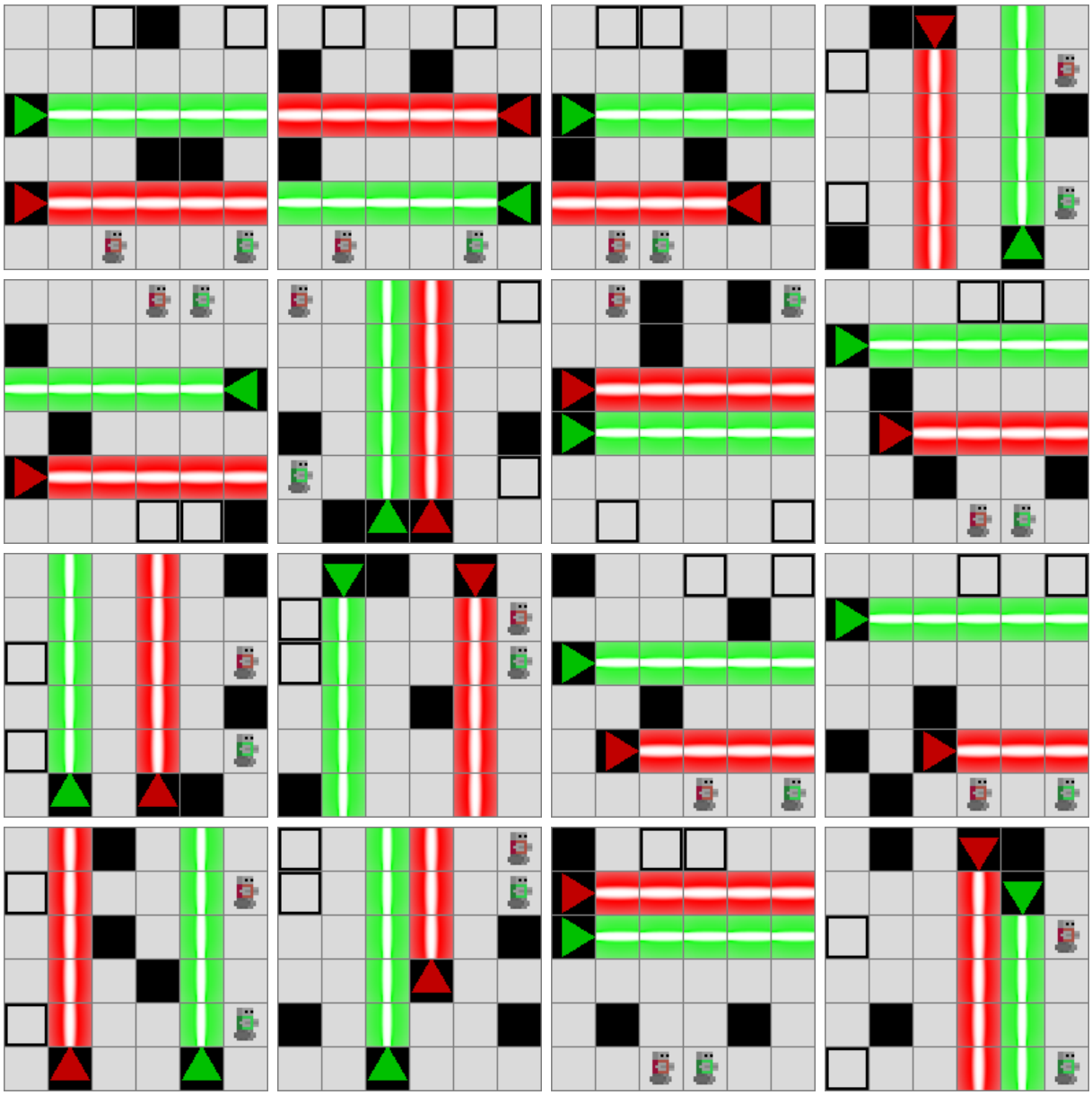

Figure 33: Sixteen of the twenty frontier-probe training levels: 6 × 6, 2 agents, 2 lasers, `fully_coupled` cooperation profile. Source: `results/learnability_6x6_2L/levels_png/train`.

Two diagnostic checks rule out a budget shortfall rather than a learnability wall. An **overfit gate** trains a single algorithm (QMIX) on one fixed mutual level for 600,000 steps — removing the need to generalise entirely — and still plateaus below 0.12 training success with zero held-out success. A

budget-bump gate extends the budget to $1\{,\}500\{,\}000$ steps (VDN, 100-level pool): success stays flat, peaking early (around 0.10) and then decaying to zero rather than climbing. Finally, shrinking the grid to 5×5 / 2-laser (VDN, 200,000 steps) yields 0.00 on both train and test pools — two lasers drive success to zero independently of grid size.

A.13.2 Two-laser curriculum

The four scheduling conditions of Section 7.5.1, on a fixed 6×6 grid with a zero-, one-, then two-laser ladder to the mutual two-laser target, budget-matched at 600,000 steps. Owing to the cost of the runs and the uniformly zero outcome, coverage is partial: **direct** was run with VDN (six seeds) and QMIX (three seeds); **forward**, **reverse**, and **mixed** with VDN (three seeds each). Table 27 reports the final held-out success.

Condition	Algorithms (seeds)	Held-out test (mean)	Train (mean)
direct	VDN (6), QMIX (3)	0.00	0.08
forward	VDN (3)	0.00	0.05
mixed	VDN (3)	0.01	0.01
reverse	VDN (3)	0.00	0.00

Table 27: Two-laser curriculum: final greedy success on the 6×6 / 2-agent / 2-laser mutual target per scheduling condition. Source: `results/curriculum_strategy_2L/runs/`.

The fixed-grid ladder is SAT-generated once from `RNG_SEED = 20260523` (per-rung seed $20260523 + 100 \times$ stage, $+1$ for the held-out target pool). Only the two-laser target rung (S3) is constrained to the `fully_coupled` cooperation profile; the lower rungs accept any solvable (S1) or cooperative (S2) level. Table 28 lists the per-rung parameters and Figure 34, Figure 35, and Figure 36 show sixteen representative training levels from each rung.

Rung	Grid	Agents	Lasers	$T_{\max}$	Walls	Generator	Profile	Train	Eval
S1	6×6	2	0	12	3	Random (solvable)	—	100	0
S2	6×6	2	1	14	3	Constructive (cooperative)	any	100	0
S3	6×6	2	2	18	3	Constructive (cooperative)	`fully_coupled`	100	50

Table 28: Per-rung generation parameters for the two-laser curriculum pools (Section 7.5.2). Source: `src/experiments/curriculum_strategy_2L/configs.py`; pools rendered by `src/scripts/render_curriculum_strategy_2L_pools.py`.

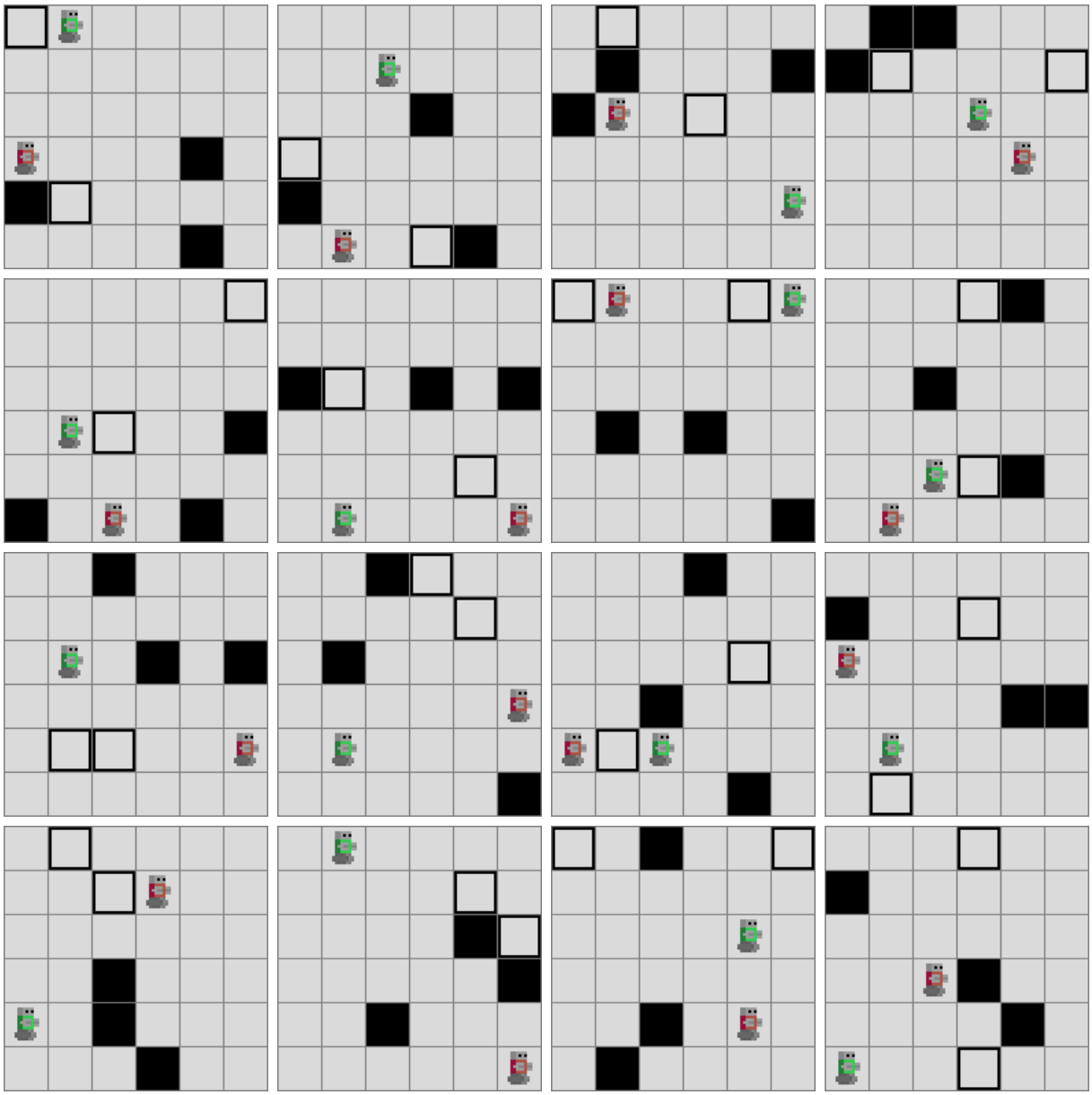

Figure 34: Sixteen S1 training levels: 6×6, 2 agents, 0 lasers (navigation warm-up).

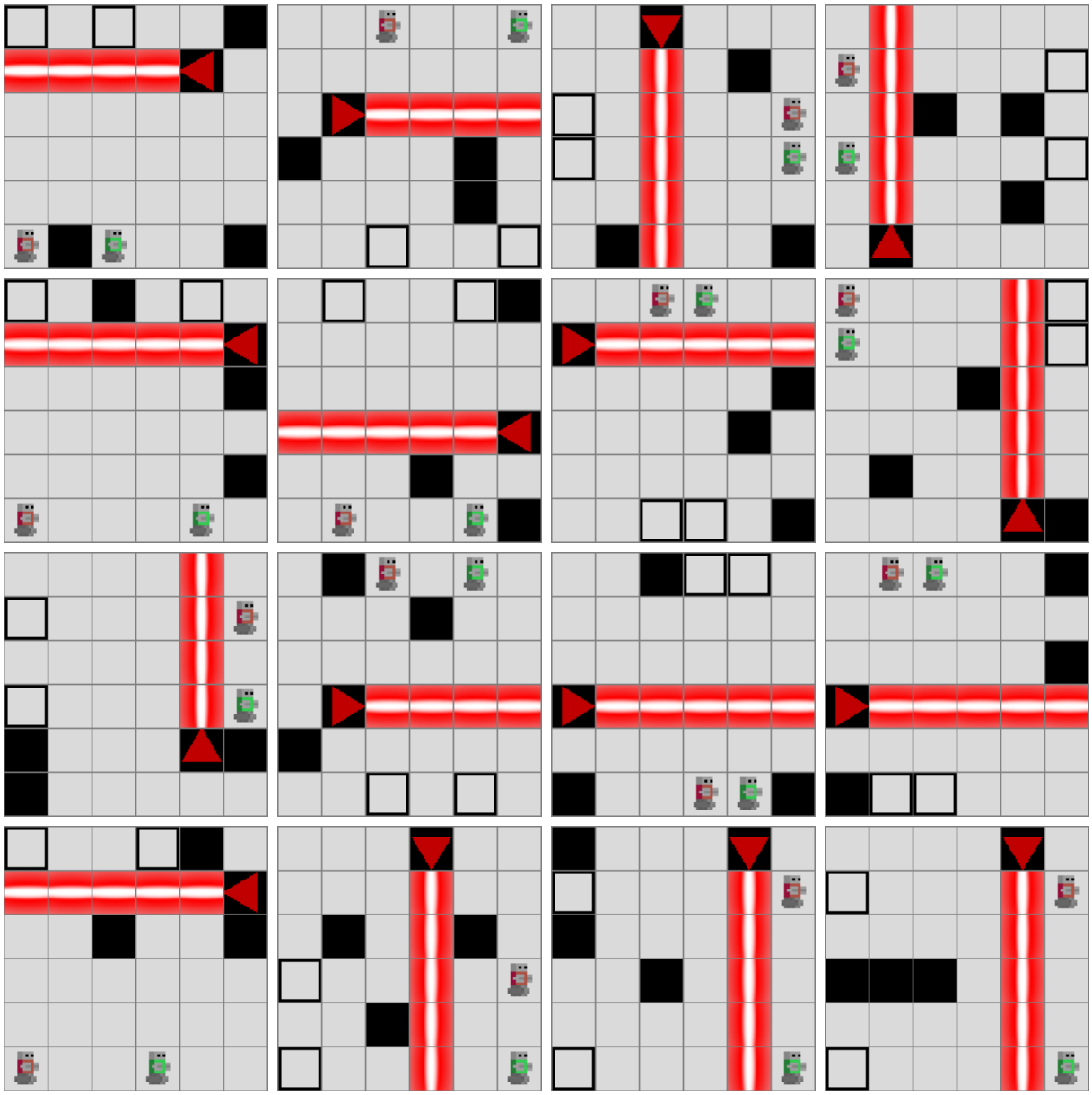

Figure 35: Sixteen S2 training levels: 6 × 6, 2 agents, 1 laser (asymmetric cooperation).

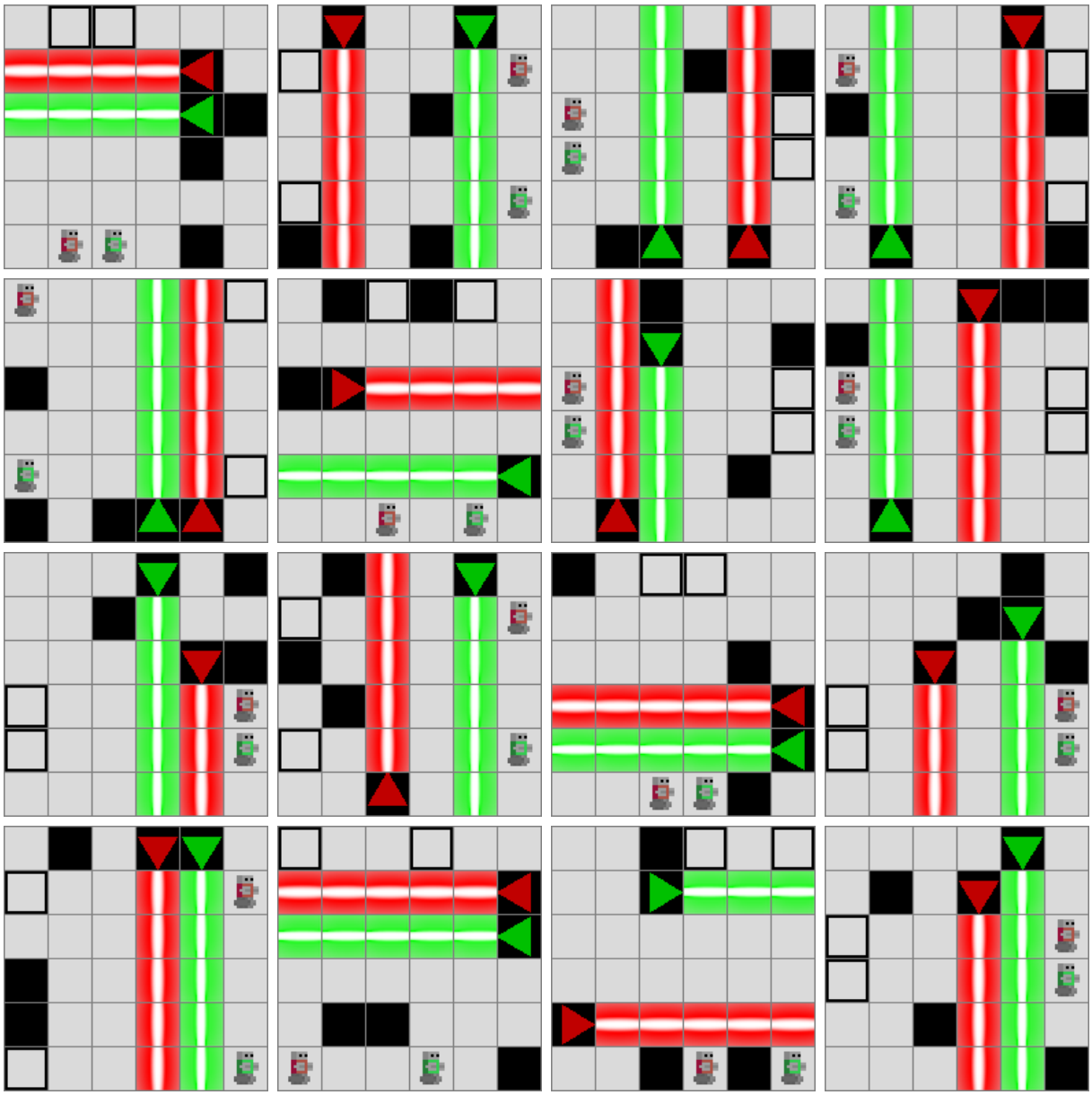

Figure 36: Sixteen S3 (target) training levels: 6 × 6, 2 agents, 2 lasers, `fully_coupled` profile (the mutual two-laser target).

A.13.3 Level-6 transfer

A four-stage curriculum of generated levels growing in geometry and cooperation, four agents throughout, with the gem reward set to zero in every environment so that the reward structure is consistent between the gem-free generated levels and the gem-bearing Level 6. Table 29 gives the stage ladder and Table 30 the per-condition outcomes. Every condition — including the full curriculum at two million steps — scores zero on Level 6 **and** on the in-distribution held-out generated pool; mean Level-6 return stays negative throughout. The stage pools are SAT-generated from master seed `RNG_SEED = 20260514`, and the runs were produced against `marl` commit `23c4d233`. The stage pools themselves were not retained on disk; representative levels for the four stage generators at comparable grid sizes appear in Section A.15 (stage 1), Section A.17 (stages 2–3), and Section A.19 (stage 4, the 12 × 13 Level-6 footprint).

Stage	Grid	Lasers	Generator
1	6×6	1	Random (solvable)
2	8×8	2	Constructive (cooperative)
3	10×10	3	Constructive (cooperative)
4	12×13	3	Level-6-Style

Table 29: Level-6 transfer curriculum stages (condition CURR). Four agents throughout. Source: `src/experiments/curriculum/configs.py`.

Condition	Description	Steps	SR Level 6	SR generated pool
B1	Hardest stage only	750k	0.00	0.00
B2	Anti-curriculum (hard→easy)	750k–2M	0.00	0.00
B3	Direct on Level 6	750k	0.00	0.00
CURR	Four-stage curriculum	up to 2M	0.00	0.00

Table 30: Level-6 transfer: greedy success rate (200 episodes) on the hand-crafted Level 6 and on the in-distribution held-out generated pool, per condition. Runs use QMIX and VDN with one to four seeds per condition. Mean Level-6 return is negative for all trained conditions (agents accrue step penalties without exiting). Source: `results/curriculum_experiment/runs/`.

A.14 Generator gallery — base random generator

Pure random sampling with **no** geometric validation: the generator places agents, exits, walls, and laser sources uniformly at random and accepts any layout the SAT oracle certifies as solvable, regardless of laser geometry. Every level shown here is therefore solvable — the contrast with the Constrained Random generator of the next section is one of geometric **quality**, not of solvability.

The characteristic artefact is visible throughout the pools below: most levels contain a laser source that emits **no active beam**, because the source points immediately off the grid or sits flush against a wall or edge. Such a source is inert — it behaves like an ordinary wall tile rather than a laser — so the level presents none of the beam-crossing structure the laser was meant to introduce. Exits and agent starts may likewise fall on beam tiles (the LLE engine silently relocates an agent start that would be killed on spawn). These are exactly the degeneracies that the geometric filters of the Constrained Random generator (Section A.15) are designed to reject. The same grid, agent, laser, and wall parameters are used here as in that section, so the two galleries can be read side by side. Pools are generated by `src/scripts/generate_appendix_galleries.py`.

A.14.1 Base random — 3×3, 2 agents, 1 laser

Grid	3 × 3
Agents	2
Lasers	1
Horizon $T_{\max}$	6
Wall budget	1
Seed	20260601
Samples in pool	16

Table 31: Parameters and seed for the Base Random 3×3 gallery pool.

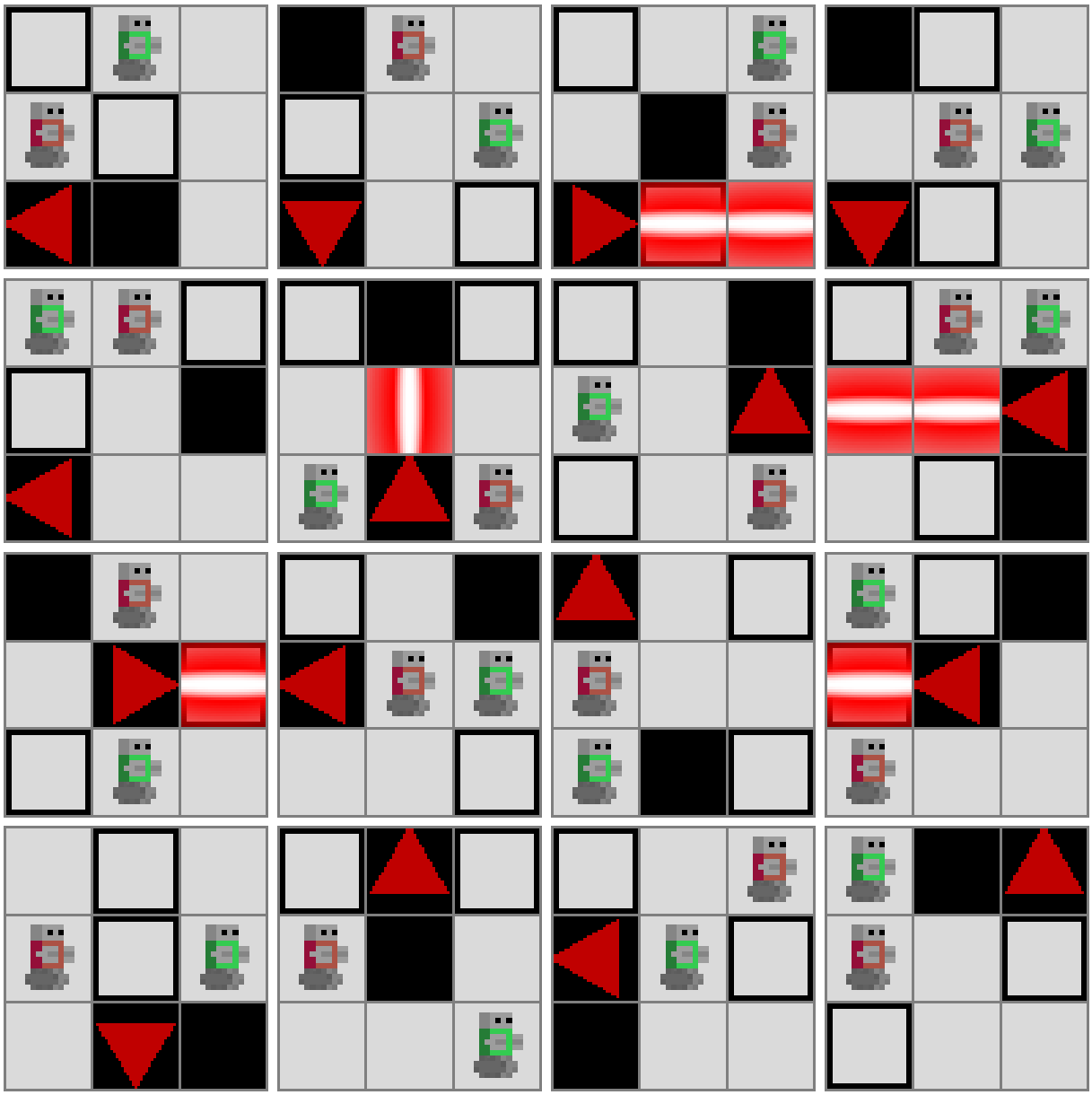

Figure 37: 16 Base Random 3×3 (2 agents, 1 laser) levels, no geometric validation.

A.14.2 Base random — 5×5, 3 agents, 2 lasers

Grid	5 × 5
Agents	3
Lasers	2
Horizon $T_{\max}$	10
Wall budget	2
Seed	20260602
Samples in pool	16

Table 32: Parameters and seed for the Base Random 5×5 gallery pool.

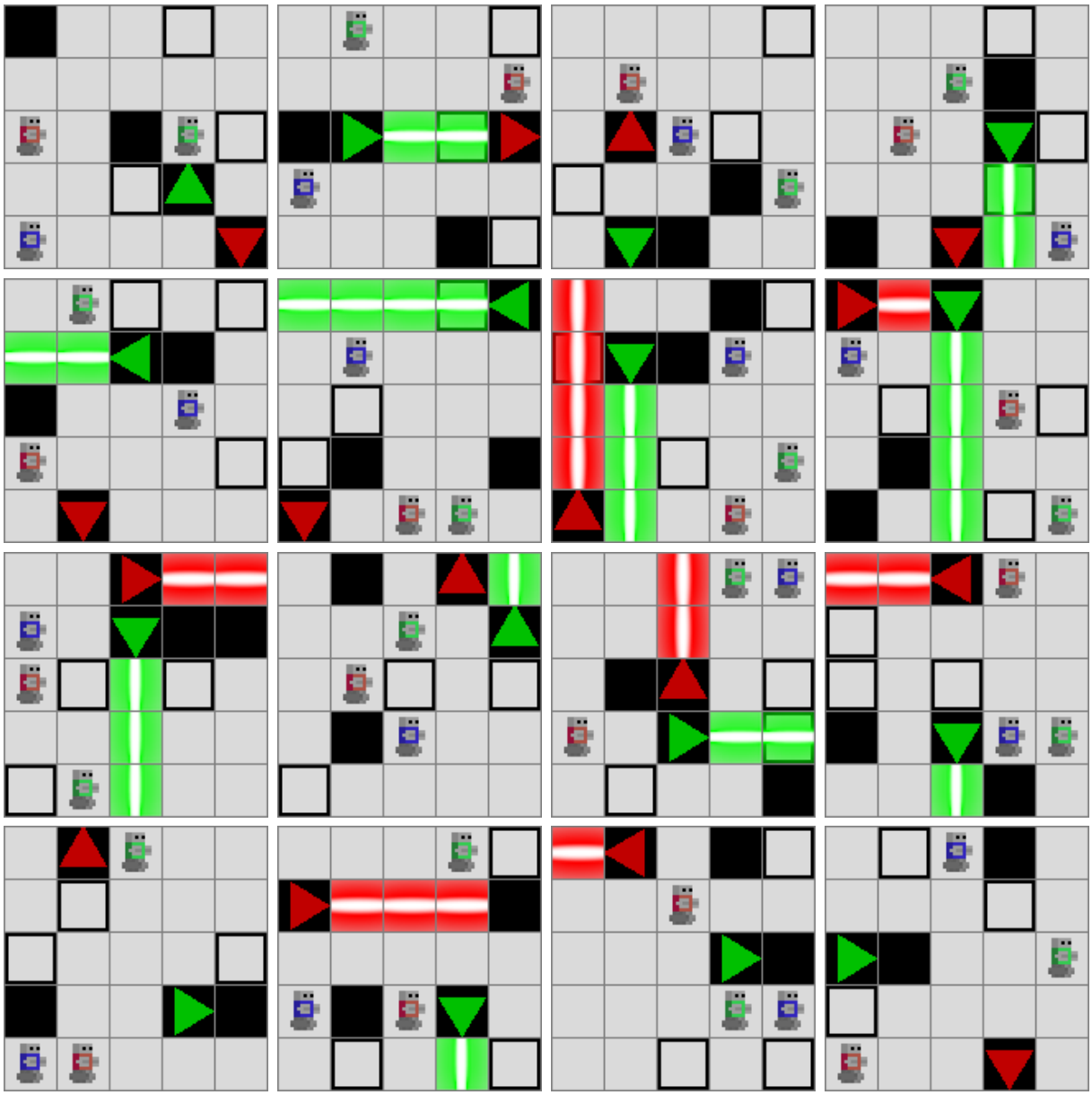

Figure 38: 16 Base Random 5×5 (3 agents, 2 lasers) levels, no geometric validation.

A.14.3 Base random — 7×7, 4 agents, 2 lasers

Grid	7 × 7
Agents	4
Lasers	2
Horizon $T_{\max}$	14
Wall budget	4
Seed	20260603
Samples in pool	16

Table 33: Parameters and seed for the Base Random 7×7 gallery pool.

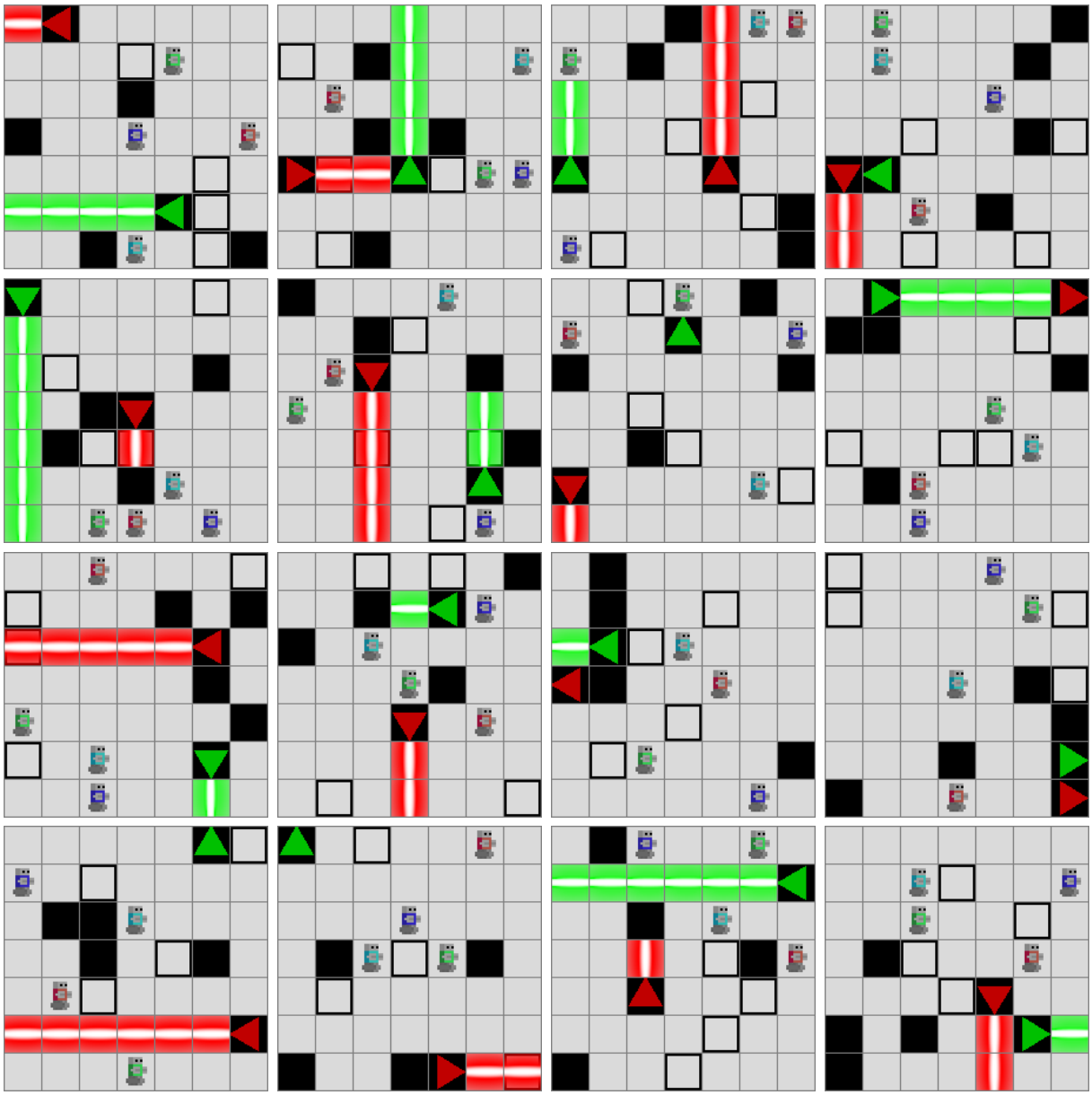

Figure 39: 16 Base Random 7×7 (4 agents, 2 lasers) levels, no geometric validation.

A.15 Generator gallery — constrained random generator

Random sampling plus geometric filters (no laser pointing immediately out of bounds, no zero-length beam, no exit on an unavoidable beam segment, etc.). Compared with the Base Random generator of Section A.14 — which uses the identical grid, agent, laser, and wall parameters but no geometric pre-filter — these filters remove the cosmetically degenerate layouts (inert laser sources that emit no beam, exits stranded on beam tiles) so that accepted levels actually exhibit the beam-crossing structure the experiments rely on. Pools are generated by `src/scripts/generate_appendix_galleries.py`.

A.15.1 Constrained random — 3×3, 2 agents, 1 laser

Grid	3 × 3
Agents	2
Lasers	1
Horizon $T_{\max}$	6
Wall budget	1
Seed	20260611
Samples in pool	16

Table 34: Parameters and seed for the Constrained Random 3×3 gallery pool.

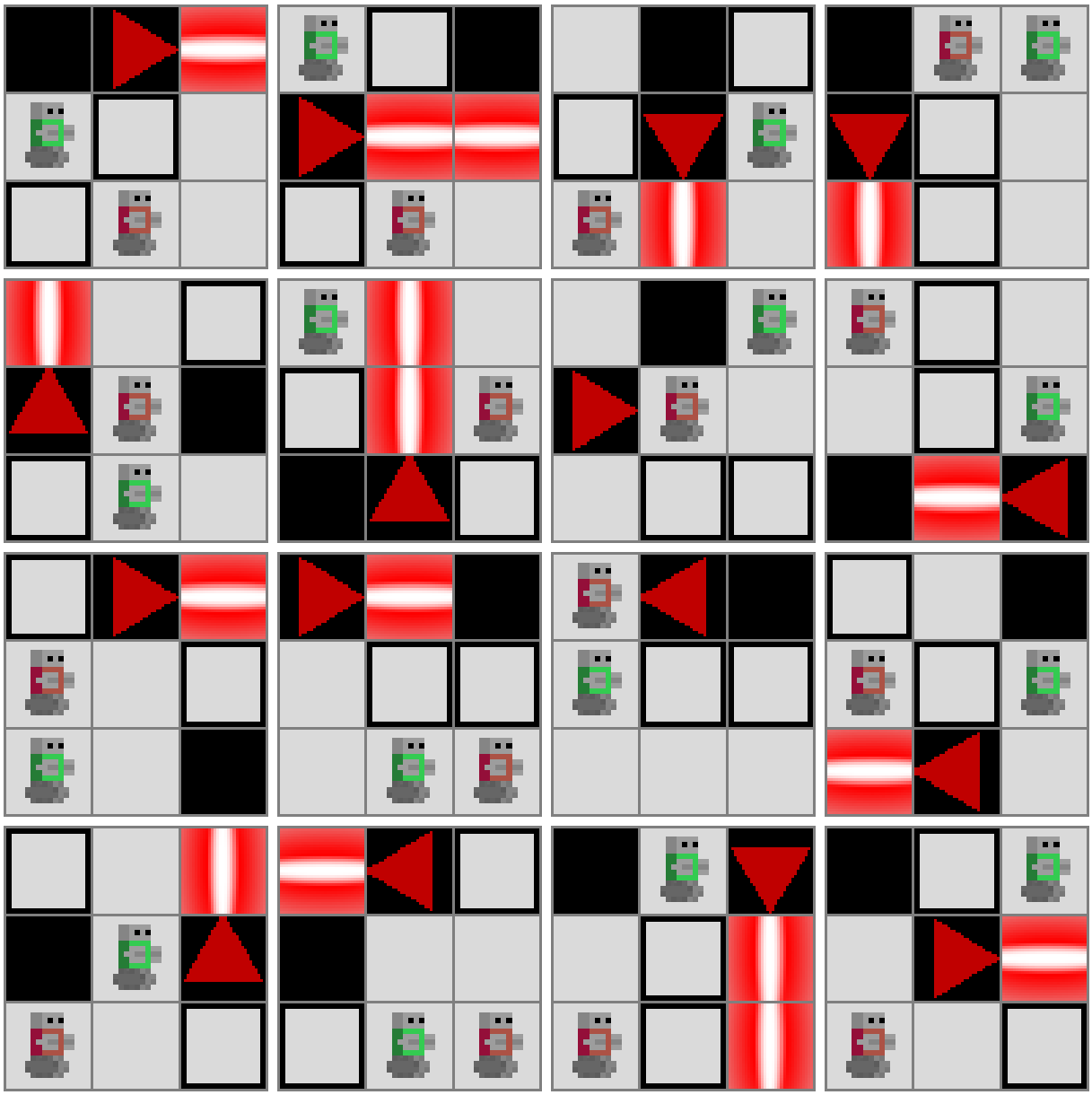

Figure 40: 16 Constrained Random 3×3 (2 agents, 1 laser) levels.

A.15.2 Constrained random — 5×5, 3 agents, 2 lasers

Grid	5 × 5
Agents	3
Lasers	2
Horizon $T_{\max}$	10
Wall budget	2
Seed	20260612
Samples in pool	16

Table 35: Parameters and seed for the Constrained Random 5×5 gallery pool.

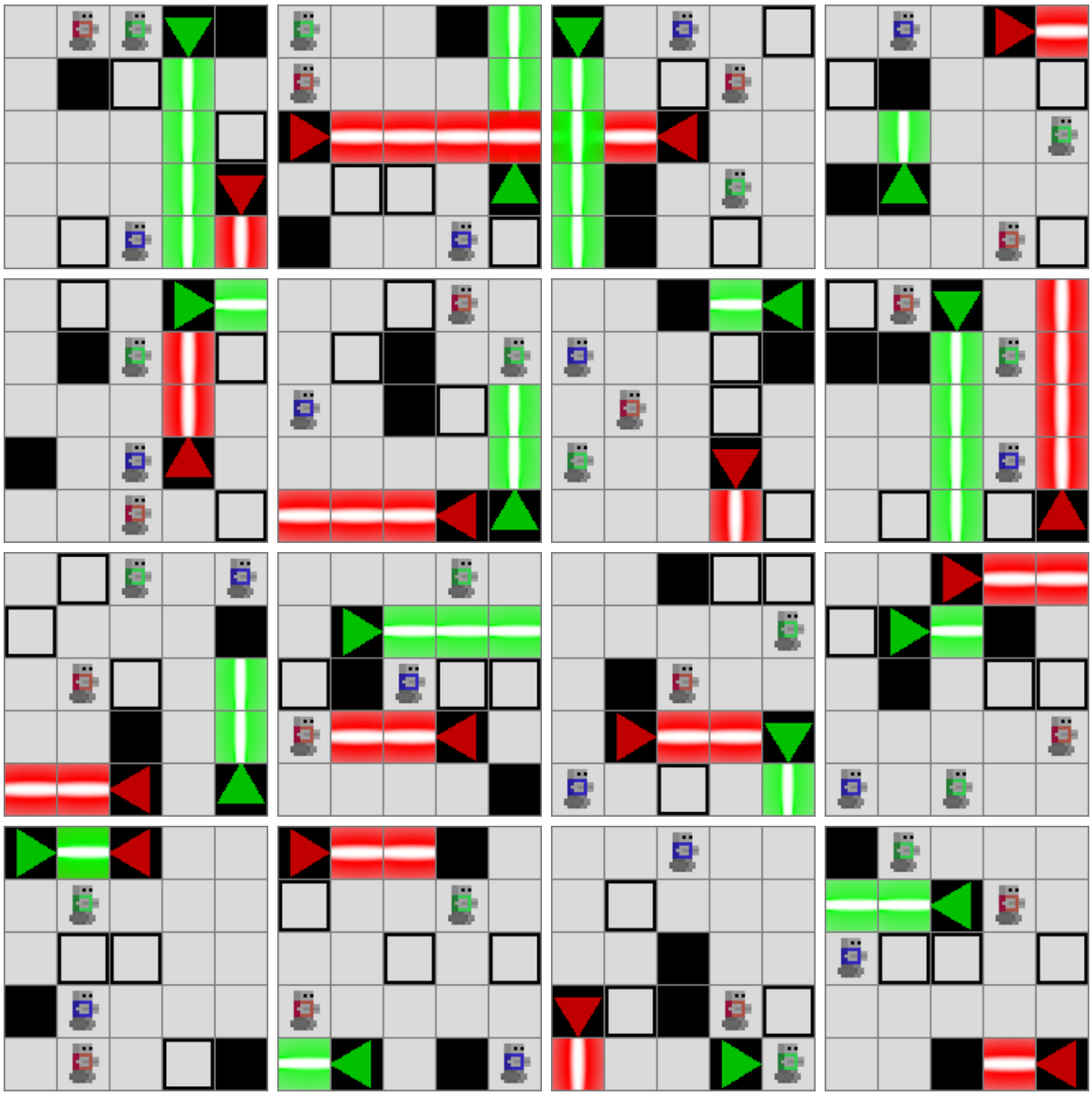

Figure 41: 16 Constrained Random 5×5 (3 agents, 2 lasers) levels.

A.15.3 Constrained random — 7×7, 4 agents, 2 lasers

Grid	7 × 7
Agents	4
Lasers	2
Horizon $T_{\max}$	14
Wall budget	4
Seed	20260613
Samples in pool	16

Table 36: Parameters and seed for the Constrained Random 7×7 gallery pool.

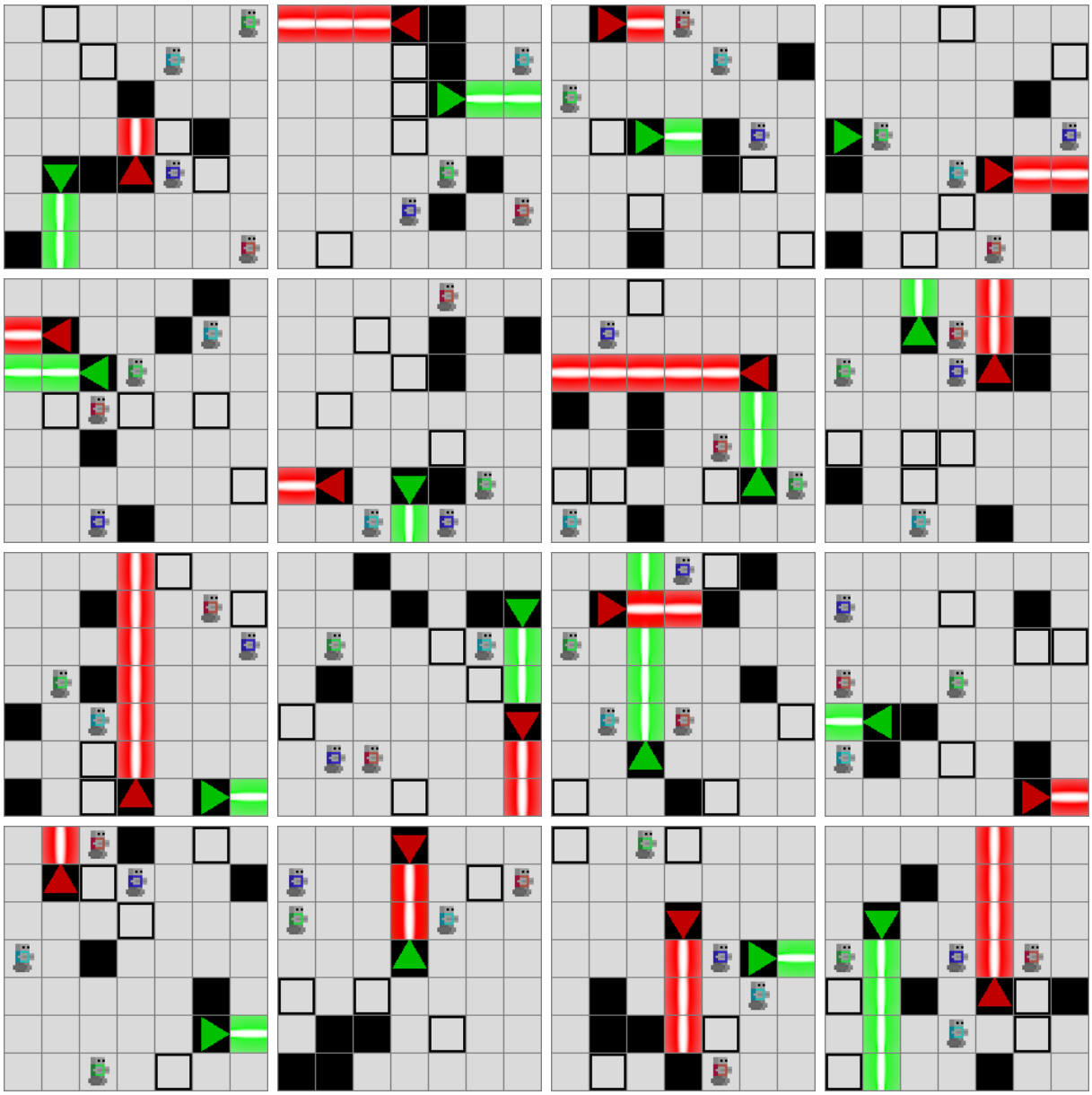

Figure 42: 16 Constrained Random 7×7 (4 agents, 2 lasers) levels.

A.16 Generator gallery — constructive generator (solvable)

Lane-based construction (one disjoint lane per agent) followed by random wall placement on the remaining cells. Cooperation is not required at generation time; only solvability is certified by the SAT oracle.

A.16.1 Constructive solvable — 5×5, 3 agents, 1 laser

Grid	5 × 5
Agents	3
Lasers	1
Horizon $T_{\max}$	10
Wall budget	3
Seed	20260621
Samples in pool	16

Table 37: Parameters and seed for the Constructive Solvable 5×5 gallery pool.

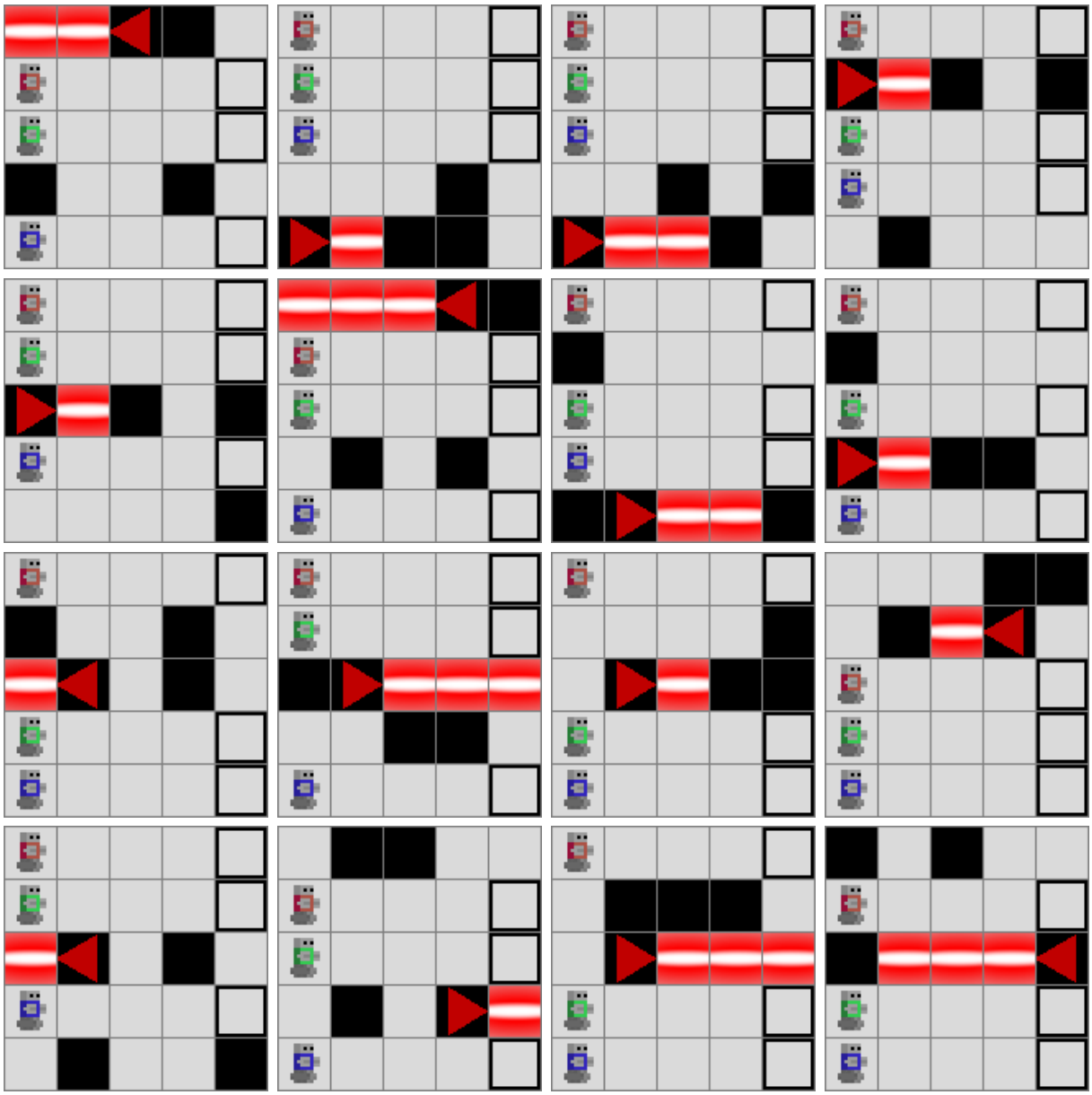

Figure 43: 16 Constructive Solvable 5×5 (3 agents, 1 laser) levels.

A.16.2 Constructive solvable — 7×7, 4 agents, 2 lasers

Grid	7 × 7
Agents	4
Lasers	2
Horizon $T_{\max}$	14
Wall budget	5
Seed	20260622
Samples in pool	16

Table 38: Parameters and seed for the Constructive Solvable 7×7 gallery pool.

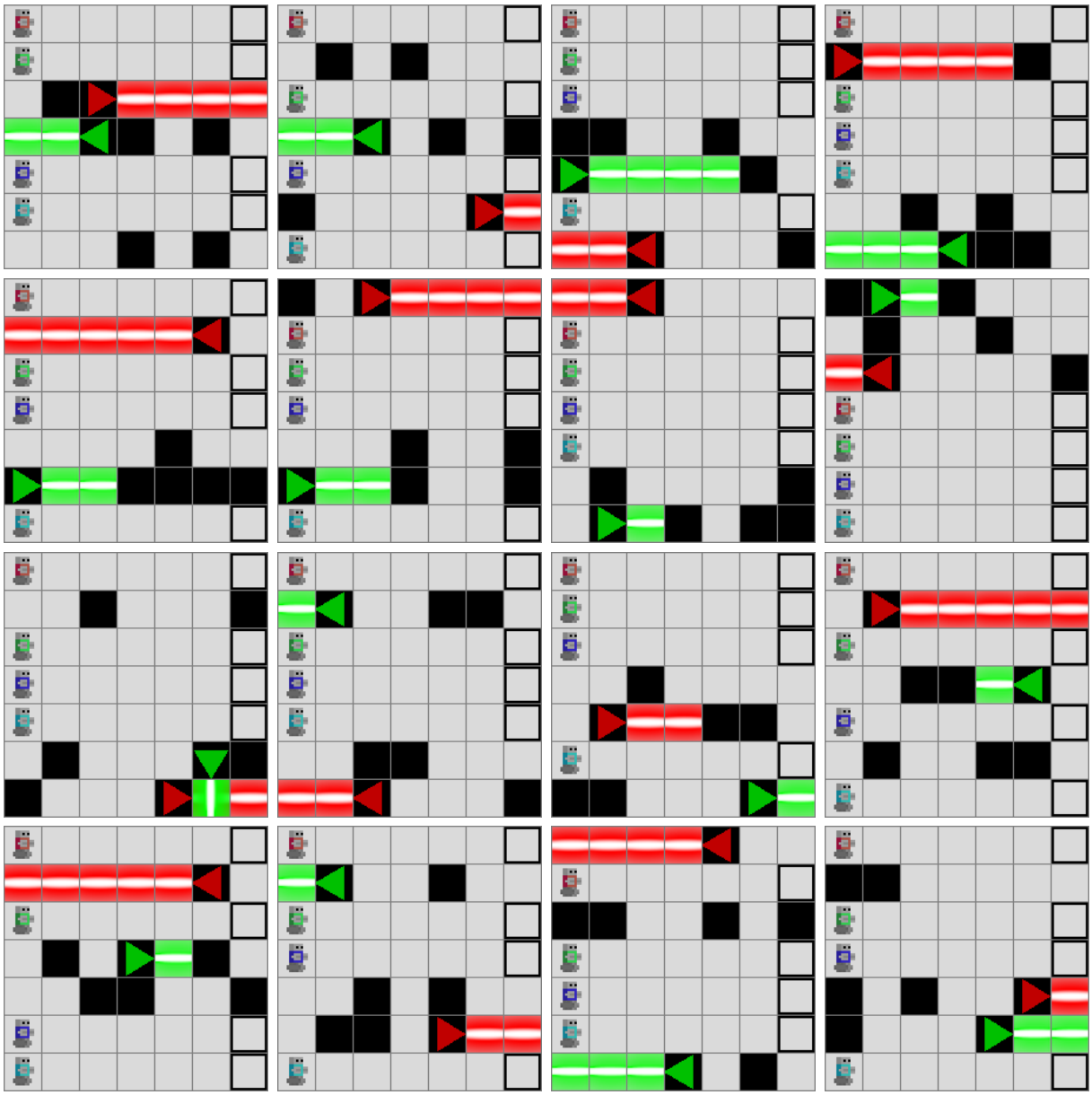

Figure 44: 16 Constructive Solvable 7×7 (4 agents, 2 lasers) levels.

A.16.3 Constructive solvable — 9×9, 4 agents, 3 lasers

Grid	9 × 9
Agents	4
Lasers	3
Horizon $T_{\max}$	18
Wall budget	8
Seed	20260623
Samples in pool	16

Table 39: Parameters and seed for the Constructive Solvable 9×9 gallery pool.

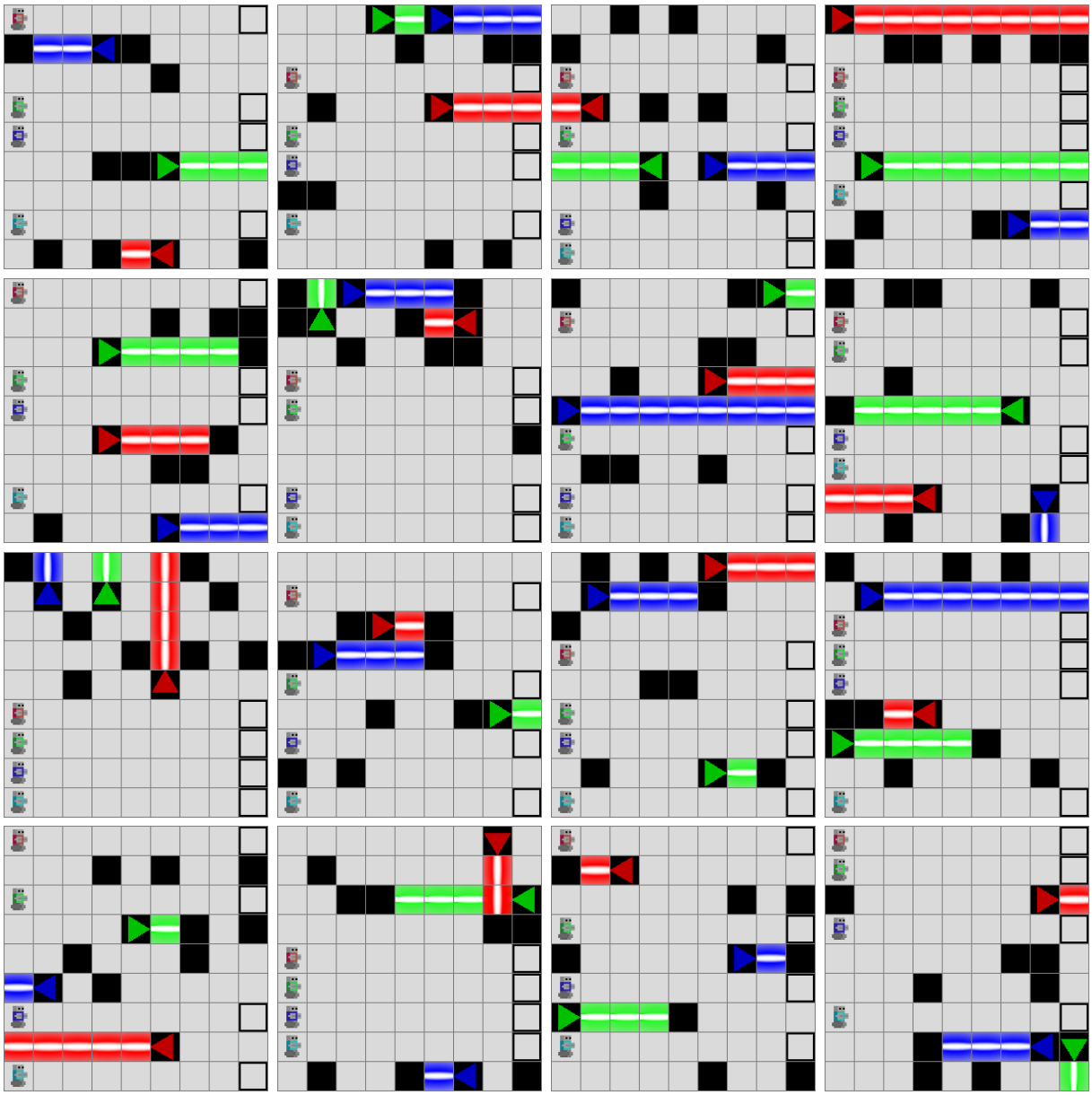

Figure 45: 16 Constructive Solvable 9×9 (4 agents, 3 lasers) levels.

A.17 Generator gallery — constructive generator (cooperative)

Lane-based construction with planted same-colour structural lasers, certified to satisfy the binary cooperation criterion of Theorem 5.1. No profile filter is applied; the gallery shows what the generator produces when any cooperation profile is admissible.

A.17.1 Constructive cooperative — 5×5, 2 agents, 1 laser

Grid	5 × 5
Agents	2
Lasers	1
Horizon $T_{\max}$	10
Wall budget	2
Seed	20260631
Profile filter	cooperative
Samples in pool	16

Table 40: Parameters and seed for the Constructive Cooperative 5×5 gallery pool.

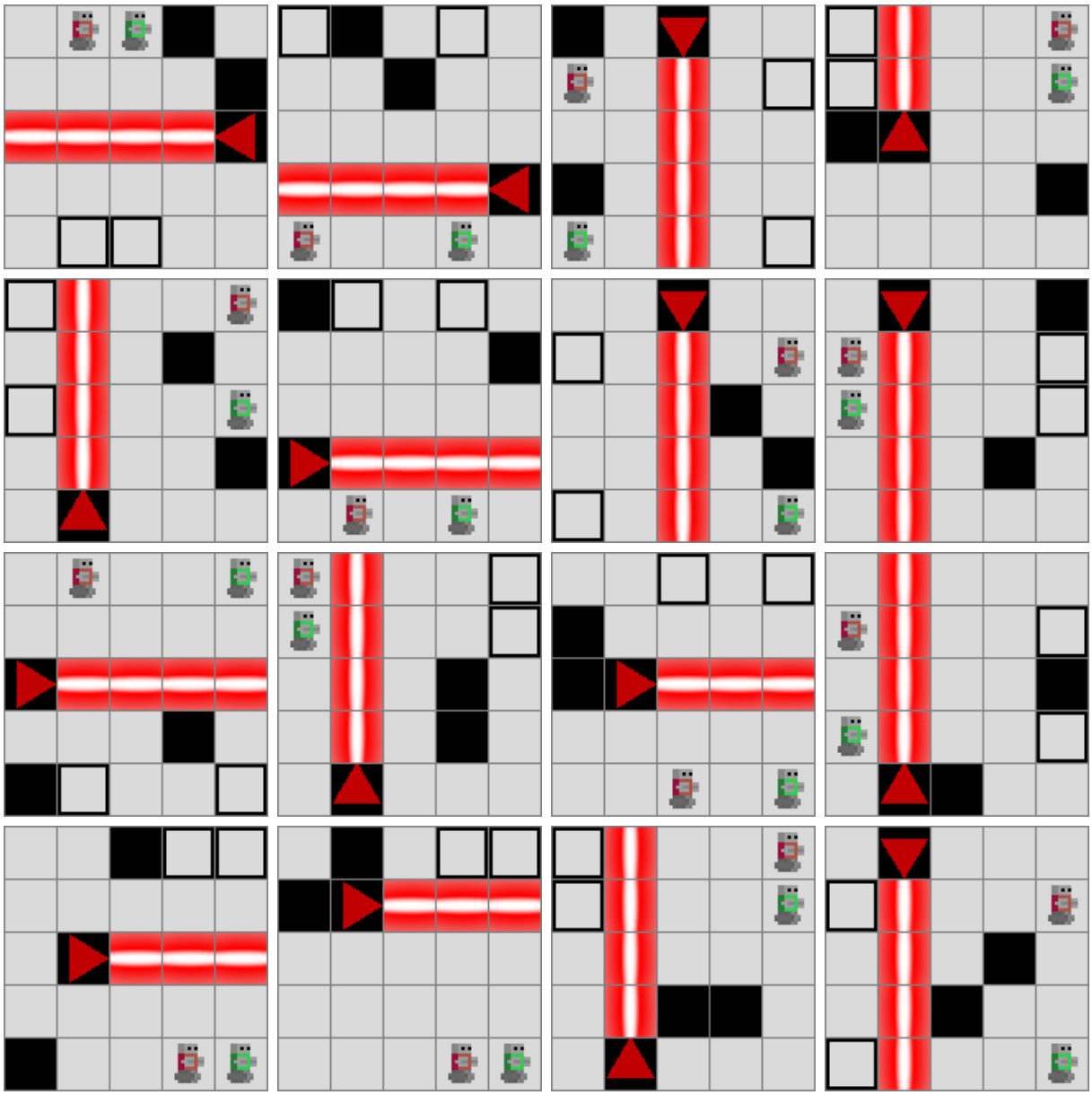

Figure 46: 16 Constructive Cooperative 5×5 (2 agents, 1 laser) levels.

A.17.2 Constructive cooperative — 7×7, 3 agents, 2 lasers

Grid	7 × 7
Agents	3
Lasers	2
Horizon $T_{\max}$	14
Wall budget	4
Seed	20260632
Profile filter	cooperative
Samples in pool	16

Table 41: Parameters and seed for the Constructive Cooperative 7×7 gallery pool.

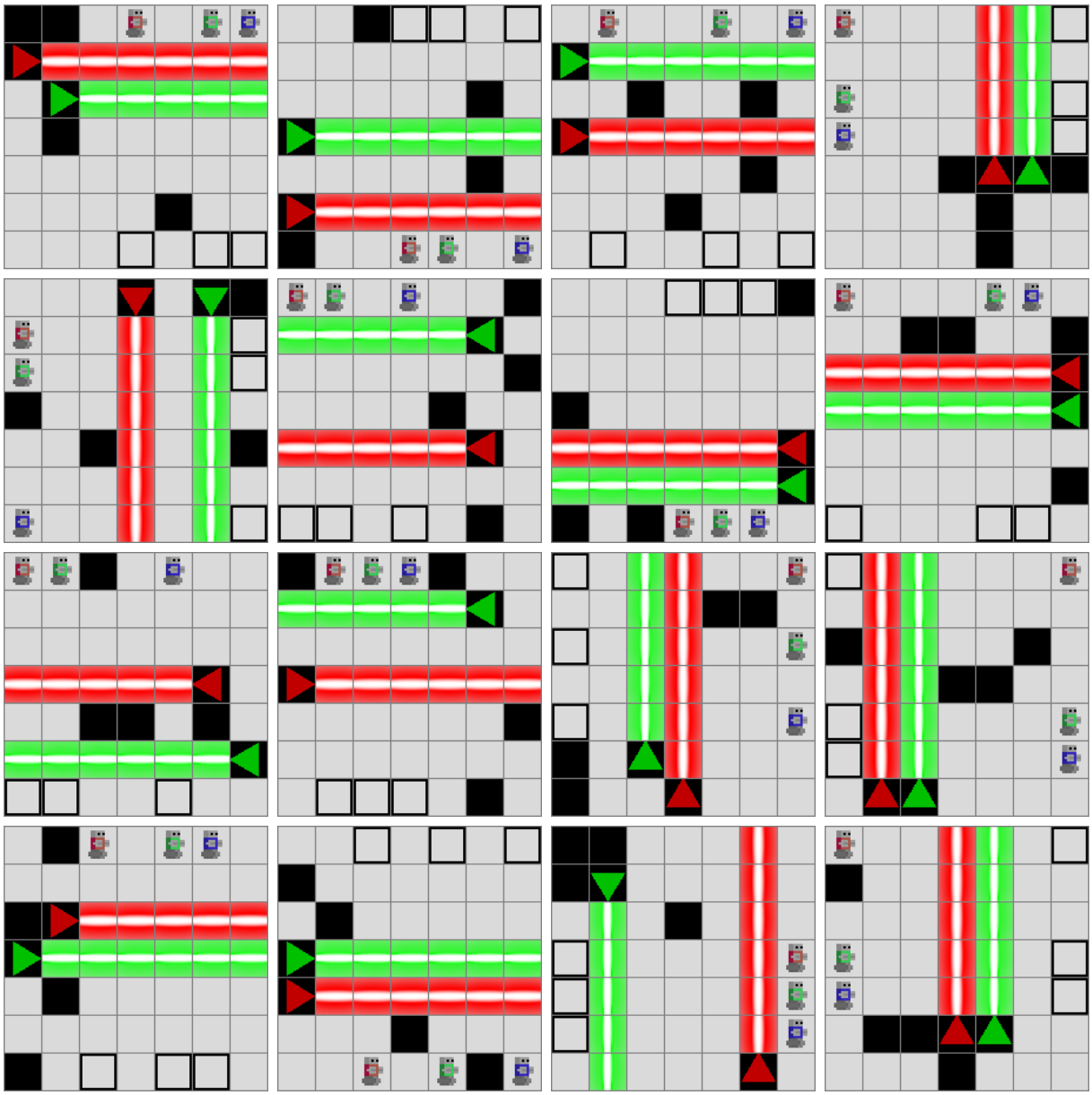

Figure 47: 16 Constructive Cooperative 7×7 (3 agents, 2 lasers) levels.

A.17.3 Constructive cooperative — 9×9, 4 agents, 3 lasers

Grid	9 × 9
Agents	4
Lasers	3
Horizon $T_{\max}$	18
Wall budget	8
Seed	20260633
Profile filter	cooperative
Samples in pool	16

Table 42: Parameters and seed for the Constructive Cooperative 9×9 gallery pool.

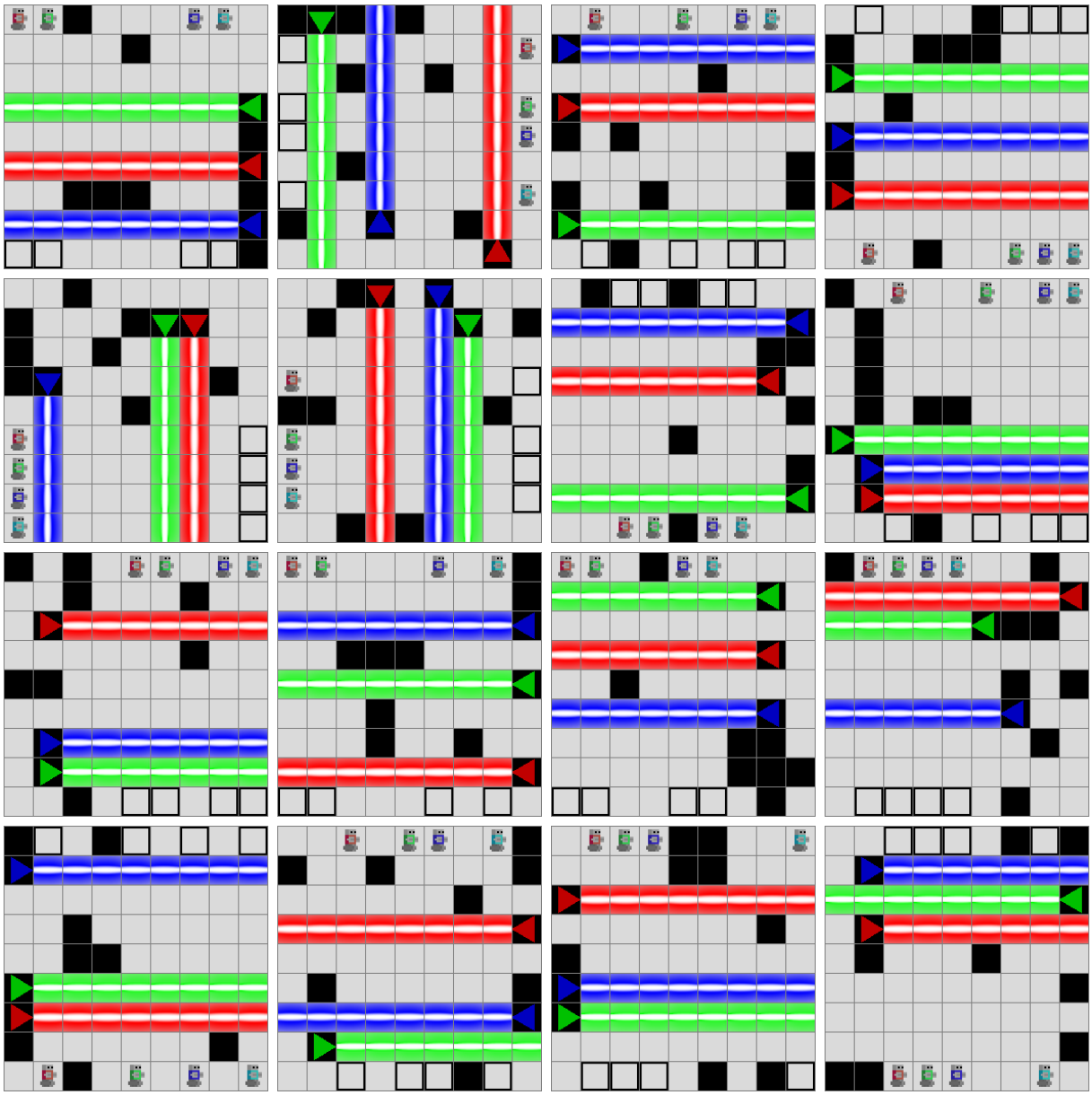

Figure 48: 16 Constructive Cooperative 9×9 (4 agents, 3 lasers) levels.

A.18 Generator gallery — constructive cooperative with profile filter

Same Constructive Cooperative generator as the previous section, but with an explicit profile filter so accepted levels match a single label (e.g. **mutual** or **distributed**).

A.18.1 Constructive cooperative — 8×8, 3 agents, 2 lasers, profile = `mutual`

Grid	8 × 8
Agents	3
Lasers	2
Horizon $T_{\max}$	16
Wall budget	5
Seed	20260641
Profile filter	mutual
Samples in pool	16

Table 43: Parameters and seed for the profile-filtered (mutual) 8×8 gallery pool.

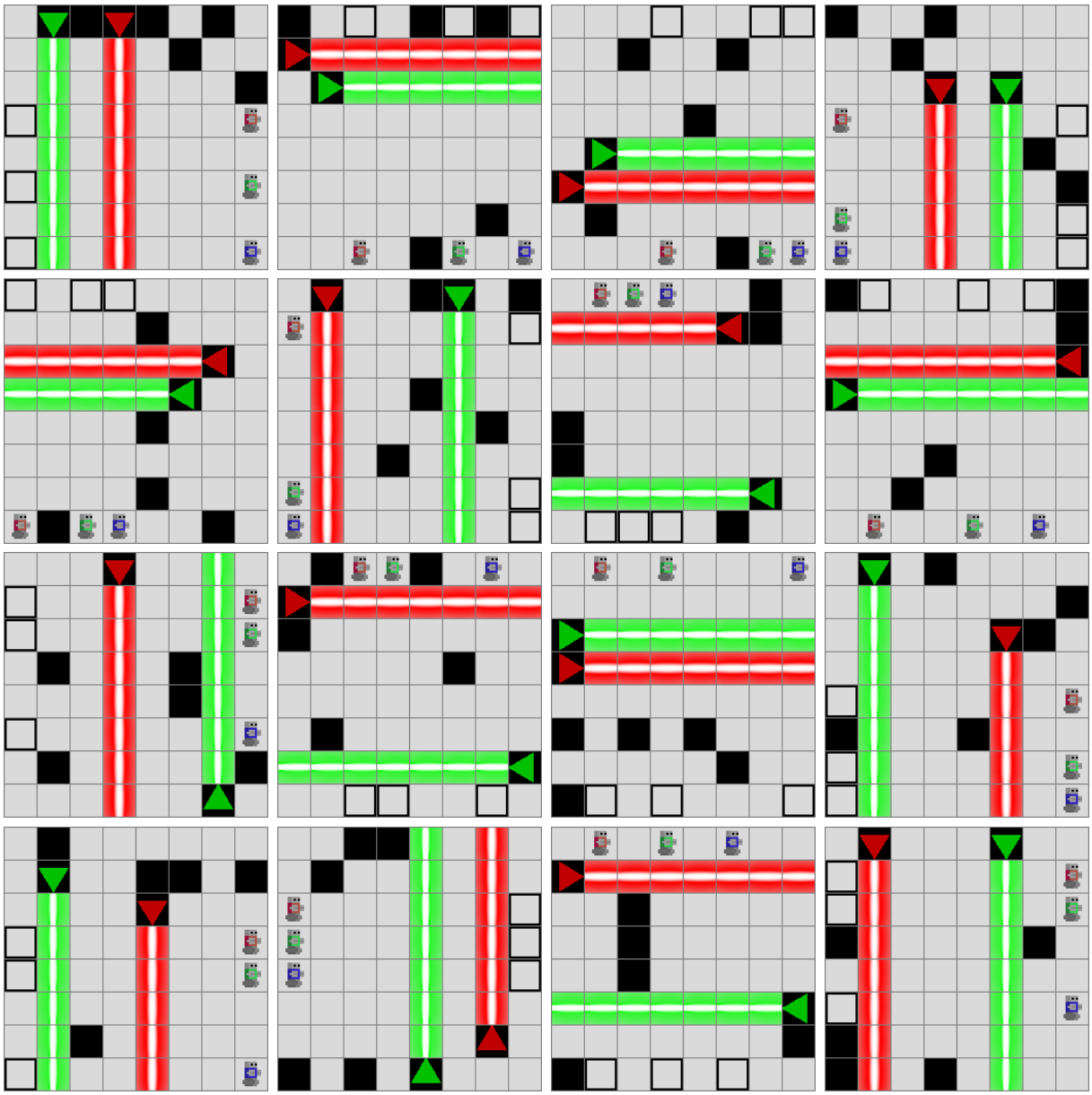

Figure 49: 16 profile-filtered (mutual) 8×8 (3 agents, 2 lasers) levels.

A.18.2 Constructive cooperative — 10×10, 4 agents, 3 lasers, profile = `distributed`

Grid	10 × 10
Agents	4
Lasers	3
Horizon $T_{\max}$	20
Wall budget	8
Seed	20260642
Profile filter	distributed
Samples in pool	16

Table 44: Parameters and seed for the profile-filtered (distributed) 10×10 gallery pool.

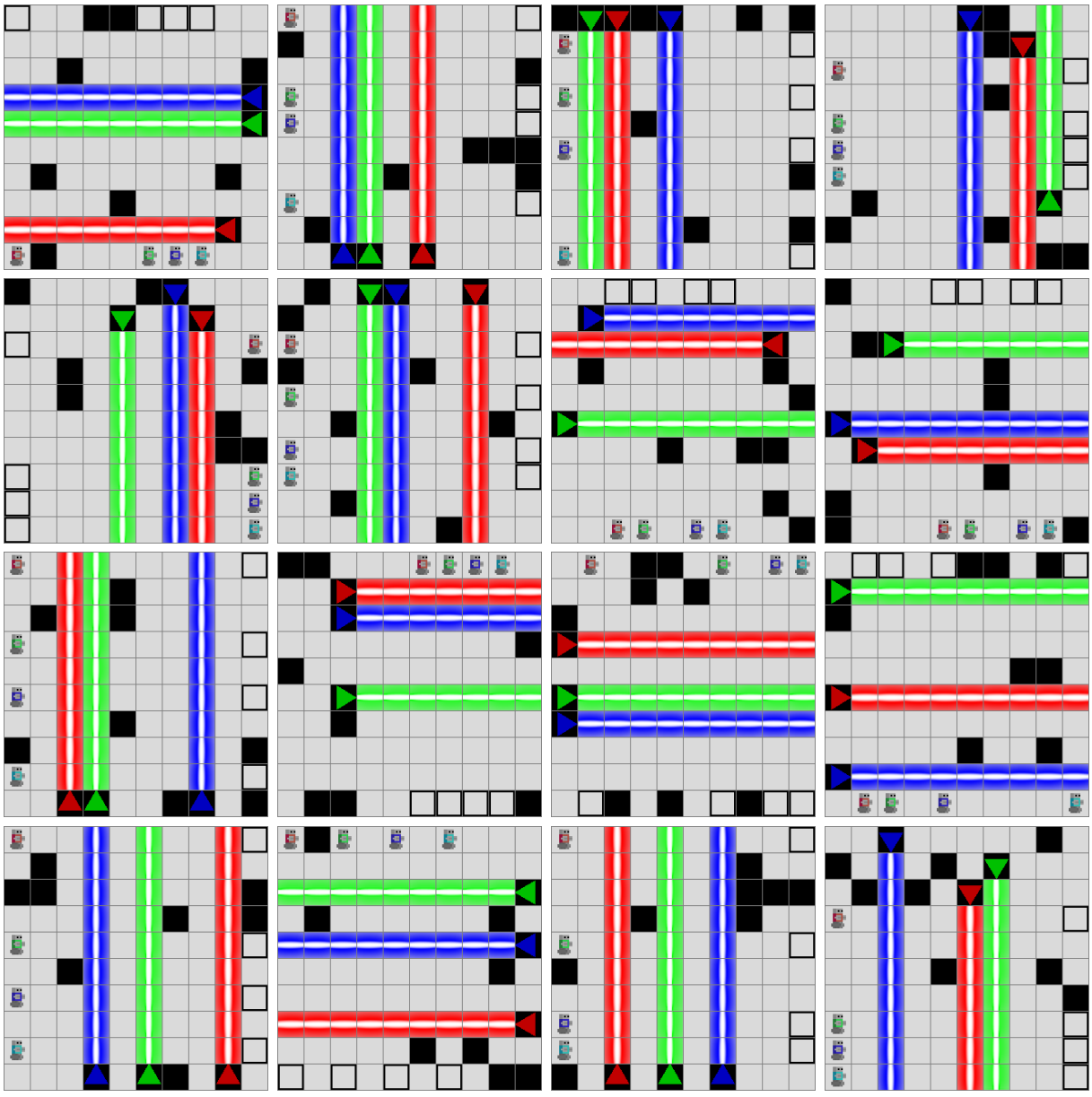

Figure 50: 16 profile-filtered (distributed) 10×10 (4 agents, 3 lasers) levels.

A.19 Generator gallery — Level-6-style generator

Clustered starts and exits on opposing sides of the grid with a corridor of lasers in between, inspired by the hand-crafted LLE Level 6.

A.19.1 Level-6-style — 8×8, 4 agents, 2 lasers

Grid	8 × 8
Agents	4
Lasers	2
Horizon $T_{\max}$	16
Wall budget	6
Seed	20260651
Samples in pool	16

Table 45: Parameters and seed for the Level-6-Style 8×8 gallery pool.

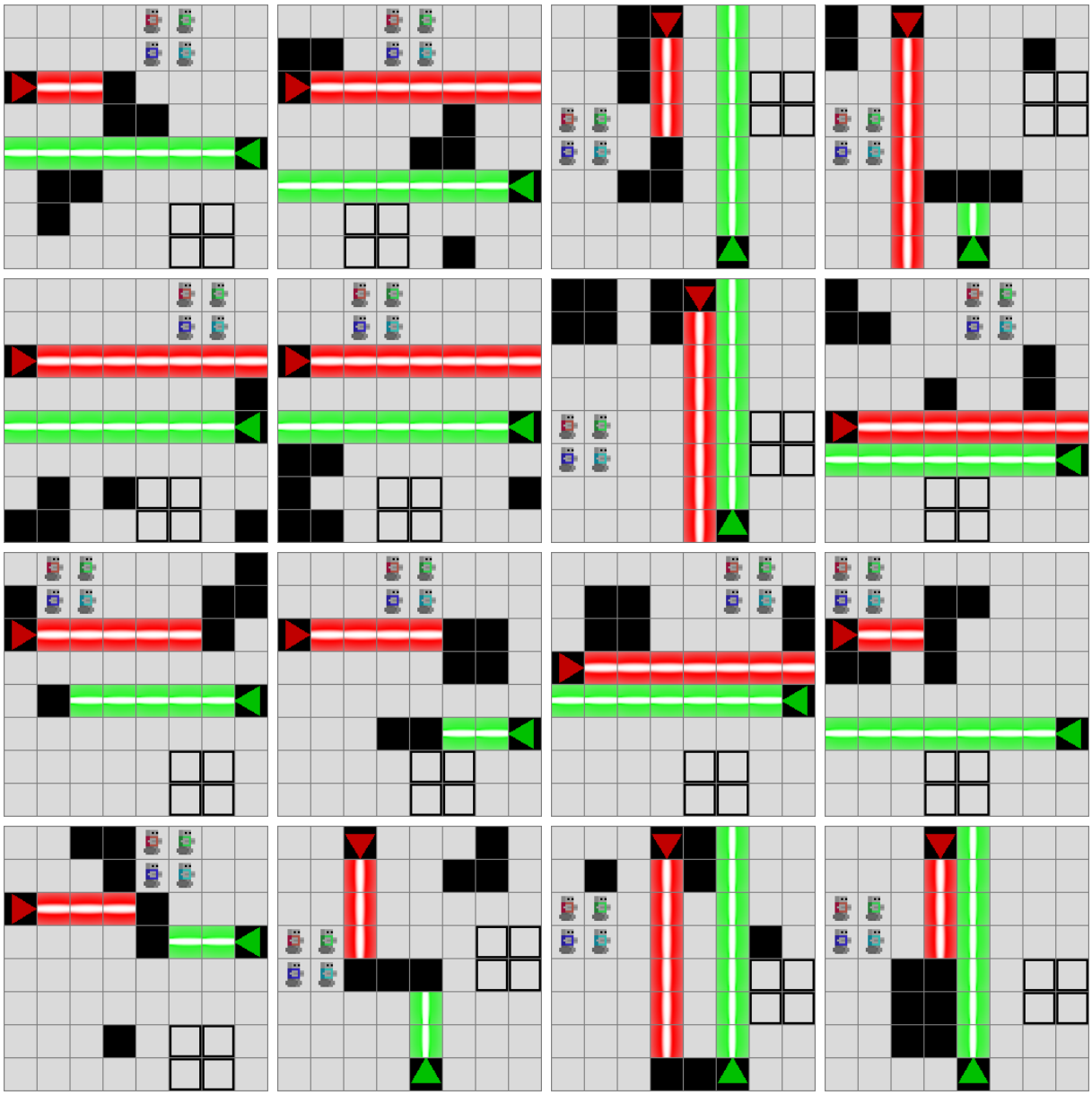

Figure 51: 16 Level-6-Style 8×8 (4 agents, 2 lasers) levels.

A.19.2 Level-6-style — 10×10, 4 agents, 3 lasers

Grid	10 × 10
Agents	4
Lasers	3
Horizon $T_{\max}$	18
Wall budget	8
Seed	20260652
Samples in pool	16

Table 46: Parameters and seed for the Level-6-Style 10×10 gallery pool.

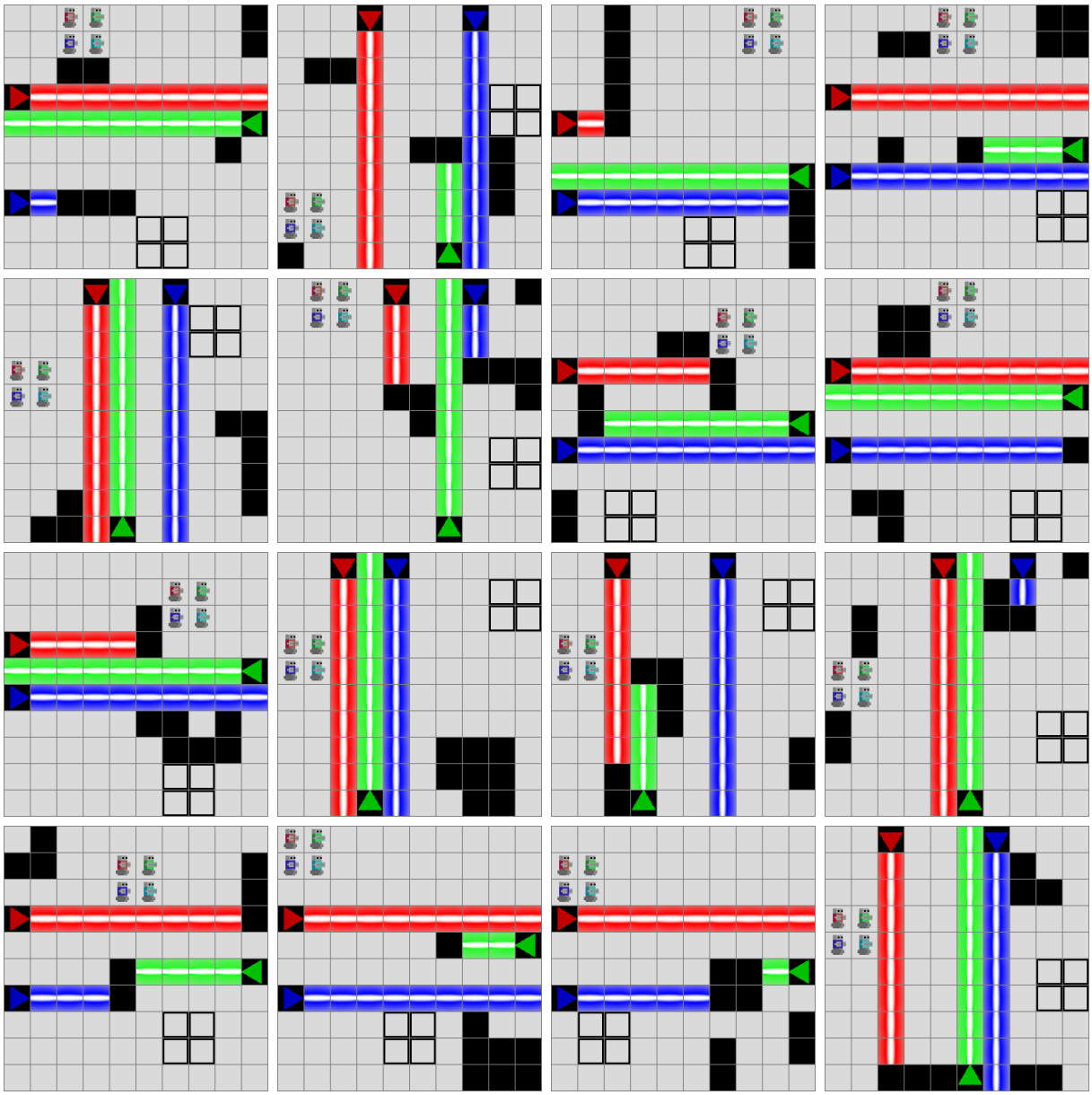

Figure 52: 16 Level-6-Style 10×10 (4 agents, 3 lasers) levels.

A.19.3 Level-6-style — 12×13, 4 agents, 3 lasers (Level 6 footprint)

Grid	12 × 13
Agents	4
Lasers	3
Horizon $T_{\max}$	21
Wall budget	10
Seed	20260653
Samples in pool	16

Table 47: Parameters and seed for the Level-6-Style 12×13 gallery pool.

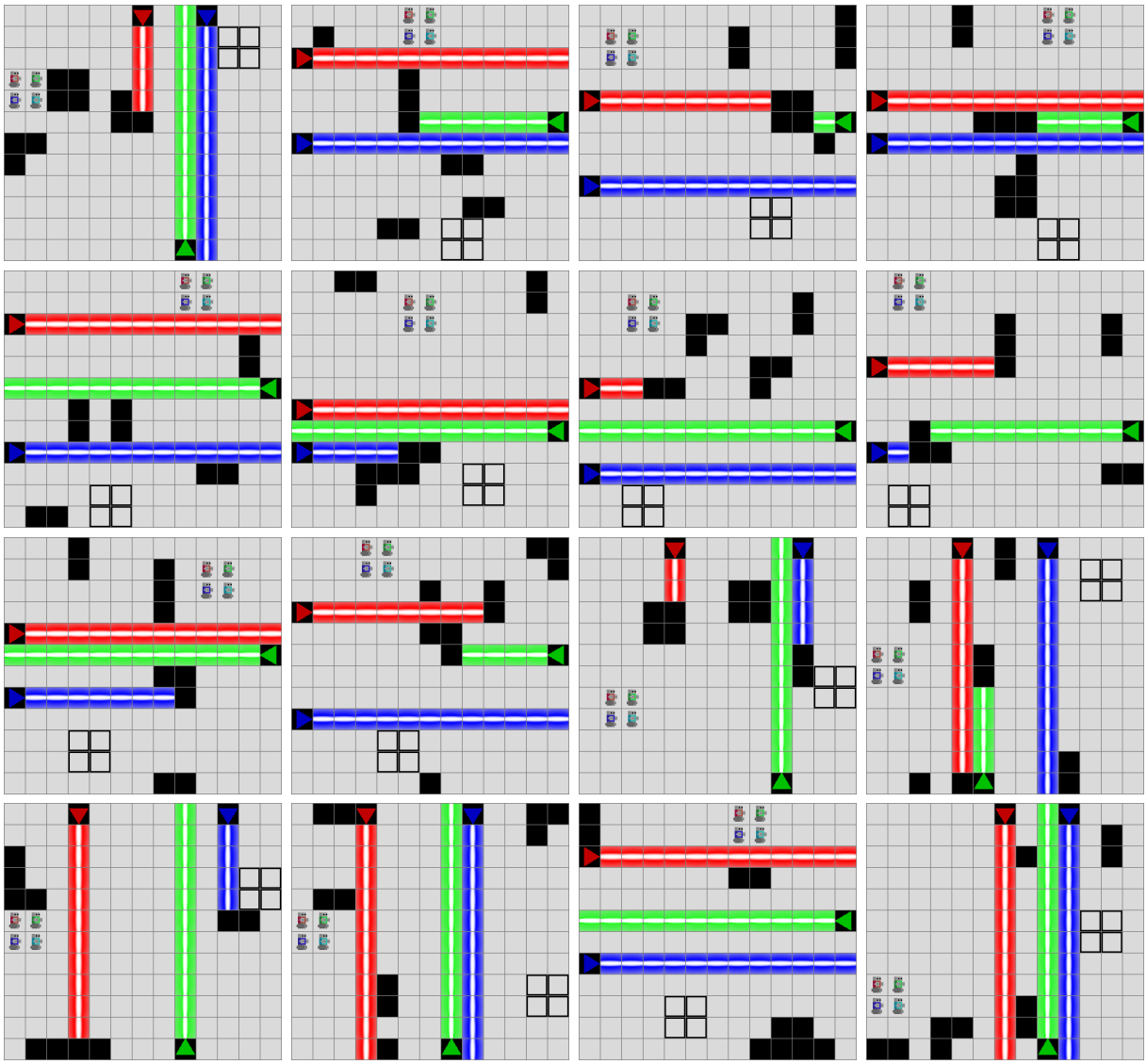

Figure 53: 16 Level-6-Style 12×13 (4 agents, 3 lasers) levels — same footprint as the hand-crafted LLE Level 6.